NextPlay

Documentación de funciones

695 funciones en 60 archivos · generado el 03/11/2026 12:36

Archivo	Funciones
accounts/achievements.py	8
accounts/admin.py	4
accounts/apps.py	1
accounts/context_processors.py	1
accounts/forms.py	17
accounts/images.py	1
accounts/management/commands/nightly.py	1
accounts/management/commands/rematch_titles.py	3
accounts/management/commands/seed_test_profile.py	9
accounts/management/commands/sync_libraries.py	3
accounts/matching.py	16
accounts/models.py	54
accounts/psn.py	13
accounts/signals.py	2
accounts/steam.py	22
accounts/views.py	44
accounts/xbox.py	10
games/admin.py	1
games/history.py	13
games/igdb.py	20
games/management/commands/import_igdb.py	4
games/management/commands/localize_games.py	4
games/management/commands/seed_demo.py	1

games/management/commands/update_prices.py	2
games/media.py	17
games/models.py	38
games/nintendo.py	10
games/prices.py	18
games/spanish.py	8
games/views.py	11
games/xbox.py	10
library/admin.py	10
library/apps.py	1
library/autofill.py	4
library/forms.py	18
library/markup.py	4
library/models.py	38
library/points.py	5
library/reminders.py	2
library/signals.py	1
library/templatetags/reviews.py	6
library/views.py	28
library/wishlist.py	6
social/activity.py	12
social/admin.py	2
social/apps.py	1
social/forms.py	3
social/models.py	21
social/signals.py	2
social/views.py	28
lists/admin.py	3
lists/forms.py	10
lists/models.py	28
lists/presets.py	14
lists/views.py	45
challenges/admin.py	0
challenges/forms.py	5
challenges/models.py	14
challenges/progress.py	8
challenges/views.py	10

App «accounts»

Usuarios: registro, inicio de sesión, perfil público o privado, vinculación con Steam, Xbox y PlayStation y cuentas de otras plataformas.

accounts/achievements.py

Logros importados de Steam, de PlayStation (sus trofeos) y de Xbox, juntados por juego.

Cada plataforma los apunta en los títulos importados del usuario (models.ImportedTitle.set_achievements): Steam los lee juego a juego (steam.import_achievements), PlayStation con su lista de trofeos (psn.save_trophies) y Xbox con su historial de juegos (xbox.save_achievements). Aquí se juntan por juego para la ficha, las portadas y las cifras del perfil.

De Steam se guardan además uno a uno: los logros de cada juego (models.Achievement, con save_info) y cuáles tiene cada usuario y cuándo los consiguió (models.UnlockedAchievement, con save_unlocked). De ahí salen «Tus logros» de la ficha (detail) y la sección «Logros» del perfil (showcase).

```
def progress_by_game(user, game_ids)
```

Cómo va user con los logros de los juegos game_ids: {id del juego: [progreso en cada plataforma]}, solo de los juegos que tienen logros y ya se han leído.

Cada progreso es un dict con la plataforma («platform»), su nombre («label») y cómo llama a sus logros («noun»: «trofeos» en PlayStation), los conseguidos («unlocked») y los que hay («total»), el porcentaje («percent», sin redondear hacia arriba: el 100 % es tenerlos todos), si los tiene todos («complete») y si tiene el platino de PlayStation («platinum»). Con varias cuentas o versiones del juego en una plataforma (la de PS4 y la de PS5), cuenta la que más lleva. «detailed» dice si se conocen también uno a uno (detail), y «imported_id» y «title_id» son su título importado y el de la plataforma.

accounts/achievements.py, línea 24

```
def badge(progress)
```

Insignia de la portada de un juego según su progreso (un valor de progress_by_game): «Platino» si tiene el de PlayStation, «100 %» si tiene todos los logros en alguna plataforma y, si no, vacía.

accounts/achievements.py, línea 63

```
def totals(progress)
```

Cifras del perfil a partir de progress_by_game: cuántos platinos tiene («platinums») y en cuántos juegos tiene todos los logros de alguna plataforma («complete»).

accounts/achievements.py, línea 73

```
def save_info(title, items, percentages, at)
```

Guarda los logros del juego de title (un ExternalTitle) tal como los da su plataforma y apunta que se han leído en at.

items son sus logros en orden, cada uno con «api_name», «name», «description», «icon», «icon_locked» y «hidden»; percentages, el porcentaje de los jugadores que tiene cada uno ({clave: porcentaje}), o None si no se ha podido leer: entonces se queda el que había. Los logros que ya no da la plataforma no se borran: alguien puede tenerlos.

accounts/achievements.py, línea 83

```
def save_unlocked(found)
```

Apunta uno a uno los logros conseguidos en unos títulos importados: found es [(ImportedTitle, logros)], con todos los logros de su juego como [(clave, conseguido, fecha o None)], que es lo que da steam.player_achievements.

Los que no salían en los logros del juego (save_info) se añaden con su clave por nombre, para no perderlos; de los que ya estaban apuntados se borran los que ya no tiene y se corrige la fecha si ha cambiado.

accounts/achievements.py, línea 108

```
def _by_rarity(achievement)
```

Clave para ordenar del logro más raro al más común; los que no se sabe, al final.

accounts/achievements.py, línea 165

```
def detail(user, game, order='')
```

«Tus logros» de user en game, uno a uno, de la plataforma en que se conocen así (de momento, Steam; con varias versiones, la que más lleva), o None si no hay ninguna.

Devuelve un dict con la plataforma («label»), los que tiene («unlocked», cada logro con su «unlocked_at») y los que le faltan («locked»), cada grupo cortado en DETAIL_SHOWN («…_shown») y el resto aparte («…_more»), cuántos hay («total»), el más raro de los que tiene («rarest»), las fechas del primero y del último («first» y «last») y los órdenes (ORDERS) con el elegido («order»). Con order «raros» los dos grupos van del más raro al más común y con «juego», en el orden del juego; si no, los que tiene del más reciente al más antiguo (los que no tienen fecha, al final) y los que le faltan del más común al más raro, que suelen ser los más fáciles.

accounts/achievements.py, línea 170

```
def showcase(user)
```

Sección «Logros» del perfil de user: sus últimos logros conseguidos («latest», SHOWCASE_SHOWN, del más reciente al más antiguo y con su juego en achievement.title.game), el más raro («rarest») y cuántos tiene guardados uno a uno («count»); None si no tiene ninguno.

accounts/achievements.py, línea 218

accounts/admin.py

Clase ExternalTitleAdmin

Revisión de los emparejamientos: los cambiados por usuarios se filtran por «emparejado por».

```
@admin.display(description='usuarios')
def owners(self, obj)
```

Cuántos usuarios tienen este título importado.

accounts/admin.py, línea 71

```
def save_model(self, request, obj, form, change)
```

Si cambia el juego, lo marca como corregido por el administrador y recoloca ya las horas de todos los usuarios que tienen el título (las de un usuario normal esperan a su próxima importación).

accounts/admin.py, línea 75

```
@admin.action(description='Bloquear: los usuarios ya no podrán cambiarlos')
def lock(self, request, queryset)
```

Bloquea los títulos elegidos.

accounts/admin.py, línea 87

```
@admin.action(description='Desbloquear')
def unlock(self, request, queryset)
```

Desbloquea los títulos elegidos.

accounts/admin.py, línea 92

Clase AchievementAdmin

Logros de cada juego tal como los da su plataforma; se rellenan solos al leer los de los jugadores.

accounts/apps.py

Clase AccountsConfig

```
def ready(self)
```

Registra las señales de la app al arrancar Django (crean el perfil de cada usuario nuevo).

accounts/apps.py, línea 7

accounts/context_processors.py

```
def notifications(request)
```

Número de avisos sin leer del usuario con sesión, para la campana de la cabecera.

accounts/context_processors.py, línea 4

accounts/forms.py

Clase SignupForm

```
def clean_email(self)
```

Pasa el correo a minúsculas y rechaza el registro si ya hay una cuenta con ese correo.

accounts/forms.py, línea 37

Clase ProfileForm

Todo lo que se cambia en Editar perfil salvo las tiendas y las cuentas vinculadas.

Datos (nombre, estado, pronombres, ubicación, biografía), apariencia (foto, cabecera, fondo, color y diseño), escaparate (favoritos, reseña destacada y secciones), redes sociales y privacidad. Las redes llegan en campos link_<red> y se guardan juntas en Profile.links. Los favoritos y las secciones llegan como listas en el orden en que están en la página.

```
def __init__(self, *args, user, **kwargs)
```

Prepara el formulario para user.

Los juegos de la cabecera son los de su biblioteca (y el que ya tenga puesto), las reseñas que puede destacar son las suyas y las fotos que puede elegir son las de las cuentas que tiene vinculadas. Añade un campo por red social con lo que tenga guardado.

accounts/forms.py, línea 96

```
@property
def privacy_fields(self)
```

Los campos de «Privacidad y avisos», que la plantilla pinta como interruptores.

accounts/forms.py, línea 161

```
def _clean_image(self, name)
```

Pasa la imagen nueva del campo name por accounts.images; sin imagen nueva deja lo que había (False = quitarla).

accounts/forms.py, línea 165

```
def clean_avatar(self)
```

Foto de perfil nueva, recortada en cuadrado y en WebP.

accounts/forms.py, línea 172

```
def clean_banner(self)
```

Imagen de cabecera nueva, recortada en apaisado (3:1) y en WebP.

accounts/forms.py, línea 176

```
def clean_background(self)
```

Imagen de fondo nueva, reducida a 1920 × 1200 como mucho y en WebP.

accounts/forms.py, línea 180

```
def clean_accent_color(self)
```

Color en minúsculas; vacío si es el violeta por defecto. Rechaza lo que no sea #rrggbb.

accounts/forms.py, línea 184

```
def clean_banner_art(self)
```

ID de la imagen de IGDB elegida para la cabecera; solo minúsculas y números, como los de IGDB.

accounts/forms.py, línea 191

```
def clean(self)
```

Comprueba las redes sociales y que la cabecera y el fondo elegidos tengan su imagen o su juego.

Deja en cleaned_data["links"] el diccionario que se guardará en Profile.links.

accounts/forms.py, línea 198

```
def save(self, commit=True)
```

Guarda el perfil con sus redes y el orden de sus secciones.

Con commit también borra del disco las imágenes sustituidas o quitadas y guarda el orden de los favoritos (quitando los marcados).

accounts/forms.py, línea 234

```
def preview(self)
```

Perfil y favoritos tal como quedarían con lo enviado, para la vista previa de Editar perfil.

No guarda nada: valida el formulario y lo que no sea válido se queda como estaba. Las imágenes nuevas no llegan, porque no se suben hasta guardar; en su lugar vienen en local_avatar, local_banner y local_background las direcciones blob: con las que el navegador ya las enseña. Devuelve (perfil sin guardar, favoritos en su nuevo orden).

accounts/forms.py, línea 248

```
def _apply_extras(self, profile)
```

Pone en profile lo que no se guarda campo a campo: las redes (en links) y el orden y la visibilidad de las secciones.

Si las secciones enviadas no son válidas, se dejan como estaban.

accounts/forms.py, línea 264

```
def _delete_replaced_images(self, profile)
```

Borra del disco las imágenes que tenía el perfil y ya no usa (sustituidas por otra o quitadas).

accounts/forms.py, línea 274

```
def _save_favorites(self)
```

Quita los favoritos marcados y guarda el orden en que llegaron los demás (los que no llegaron, al final).

accounts/forms.py, línea 281

```
def _favorite_changes(self)
```

(ids de los juegos que se quitan, favoritos que quedan) según el envío, sin guardar nada.

Los que quedan van en el orden en que llegaron en favoritos y los que no llegaron, al final.

accounts/forms.py, línea 291

Clase PlatformAccountForm

```
def clean(self)
```

Normaliza el nombre o código y comprueba que tiene el formato de la plataforma elegida.

accounts/forms.py, línea 308

accounts/images.py

Tratamiento de las imágenes que suben los usuarios: la foto, la cabecera y el fondo del perfil y las de las reseñas y publicaciones.

Cada imagen se comprueba con Pillow, se gira según su EXIF (las fotos del móvil vienen tumbadas), se reduce hasta caber en el tamaño de su uso y se guarda en WebP con un nombre aleatorio. No se recorta: se guarda entera y el usuario elige qué parte se ve, o que se vea entera, con el encuadre del perfil (Profile.avatar_x, avatar_y, avatar_zoom, avatar_fit y los de la cabecera y el fondo). De las imágenes de las reseñas y de las publicaciones se guardan dos tamaños: la grande, para el visor, y una miniatura. Al volver a codificarla se pierden los metadatos EXIF, que en las fotos del móvil pueden llevar la ubicación.

Clase Kind

Medidas máximas (en píxeles) y peso máximo de la subida (en MB) de un tipo de imagen.

```
def process(upload, kind)
```

Devuelve la imagen subida lista para guardar: un ContentFile en WebP con nombre «<tipo>-<aleatorio>.webp».

kind es una clave de KINDS. Lanza ValidationError con un mensaje para el usuario si no es una imagen, es de otro formato, pesa más de lo permitido o tiene demasiados píxeles. De los GIF animados se queda con el primer fotograma.

accounts/images.py, línea 45

accounts/management/commands/nightly.py

Clase Command

```
def handle(self, *args, **options)
```

Lanza cada tarea aunque falle la anterior y da error al final si alguna ha fallado.

accounts/management/commands/nightly.py, línea 15

accounts/management/commands/rematch_titles.py

Clase Command

```
def add_arguments(self, parser)
```

Opción: --dry-run, que enseña lo que haría sin guardar nada.

accounts/management/commands/rematch_titles.py, línea 19

```
def handle(self, *args, **options)
```

Con --dry-run lo hace todo dentro de una transacción que se deshace, también los juegos que traiga de IGDB.

accounts/management/commands/rematch_titles.py, línea 23

```
def rematch(self)
```

Busca los títulos pendientes, apunta su juego (para todos, como una importación) y actualiza las bibliotecas.

Las horas pasan como con una corrección del emparejamiento: sin apuntar actividad ni sesiones, porque no son de ahora.

accounts/management/commands/rematch_titles.py, línea 33

accounts/management/commands/seed_test_profile.py

Clase Command

```
def add_arguments(self, parser)
```

Opción --vacio para crear la cuenta vacía en vez de la de ejemplo.

accounts/management/commands/seed_test_profile.py, línea 136

```
def handle(self, *args, **options)
```

Crea el usuario de prueba con su biblioteca y reseñas, y añade reseñas de los usuarios demo.

Solo funciona con DJANGO_DEBUG=1. Si el usuario ya existía, rehace su biblioteca desde cero. Con --vacio crea (o deja vacía) la cuenta «tester_steam».

accounts/management/commands/seed_test_profile.py, línea 142

```
def achievements(self, user, games)
```

Rehace los títulos importados de prueba del usuario con sus logros (ACHIEVEMENTS), sin tocar los de verdad. Los de Steam van también uno a uno (demo_achievements), para ver «Tus logros» desplegados en la ficha y la sección «Logros» del perfil.

accounts/management/commands/seed_test_profile.py, línea 230

```
def demo_achievements(self, imported, unlocked, total, now)
```

Rehace los logros de ejemplo del título de Steam de imported: total, con los nombres de DEMO_ACHIEVEMENTS, uno de cada ocho oculto y del más común al más raro, y los unlocked primeros conseguidos a lo largo de los dos últimos años, del más antiguo al más reciente.

accounts/management/commands/seed_test_profile.py, línea 249

```
def review(self, user, entry, text, spoilers, when)
```

Publica una reseña de user sobre el juego de entry con fecha when, con su nota, horas y estado.

accounts/management/commands/seed_test_profile.py, línea 273

```
def lists(self, user, catalog)
```

Rehace las listas de ejemplo (LISTS) del usuario de prueba y de los usuarios demo que existan, con sus juegos (catalog: los de su biblioteca y los extra, en orden), sus comentarios y sus niveles, cada una con su actividad (creada hace unas horas, con sus juegos), con los «me gusta» y comentarios de LIST_REACTIONS y con quien colabora en ellas (LIST_COLLABORATORS).

accounts/management/commands/seed_test_profile.py, línea 280

```
def challenges(self, user, games)
```

Rehace los retos de ejemplo (CHALLENGES) con quienes se unen a ellos, y las sesiones de juego del usuario de prueba (SESSIONS, sobre games, los juegos de su biblioteca) para que el reto de horas tenga progreso.

accounts/management/commands/seed_test_profile.py, línea 320

```
def activity(self, user, game, when, status, rating, review)
```

Apunta en la actividad de user lo que hizo con game en el momento when: su estado, su nota y su reseña (o None).

accounts/management/commands/seed_test_profile.py, línea 350

```
def create_empty_user(self)
```

Crea «tester_steam» o lo deja como recién creado: sin juegos ni cuentas de Steam, PlayStation, Xbox u otras plataformas.

accounts/management/commands/seed_test_profile.py, línea 357

accounts/management/commands/sync_libraries.py

Clase Command

```
def add_arguments(self, parser)
```

Opciones: --pause (segundos entre usuarios), --min-hours (saltar a los recién sincronizados) y --user.

accounts/management/commands/sync_libraries.py, línea 26

```
def handle(self, *args, **options)
```

Recorre los usuarios con cuentas de Steam, del que lleva más tiempo sin sincronizar al que menos.

Un usuario que falla (perfil privado, Steam o IGDB caídos) no para a los demás: se apunta en el log y se sigue. Solo da error al final si no ha podido sincronizar a nadie. Que falle la lista de deseos de alguien no cuenta como fallo de su biblioteca.

accounts/management/commands/sync_libraries.py, línea 38

```
def sync_wishlist(self, user)
```

Trae a «Lo quiero» la lista de deseos de Steam del usuario y lo cuenta; si falla, lo apunta en el log.

accounts/management/commands/sync_libraries.py, línea 94

accounts/matching.py

Emparejamiento de los juegos importados (Steam, PlayStation, Xbox) con el catálogo y su paso a la biblioteca.

Cada título importado se guarda en ExternalTitle (compartido por todos los usuarios) y en ImportedTitle (del usuario, con sus minutos). Los títulos ya emparejados —por una importación anterior o porque alguien lo corrigió a mano— no se vuelven a buscar; los demás se buscan con la función de cada plataforma: Steam por su ID de app y las demás por nombre, sin fijarse en cómo está escrito (name_key), en el catálogo local y, si falta, en IGDB. Las horas de cada plataforma en la biblioteca salen después de todos los ImportedTitle del usuario.

```
def clean_name(name)
```

Nombre de un juego sin ™ ® © ni coletillas de plataforma, para buscarlo en el catálogo.

accounts/matching.py, línea 49

```
def normalize(name)
```

Nombre para comparar: sin ™ ® ©, en minúsculas y con los espacios simplificados.

accounts/matching.py, línea 56

```
def name_key(name)
```

Nombre para comparar sin fijarse en cómo está escrito: sin ™ ® ©, signos, espacios, tildes ni mayúsculas, para que «WATCH_DOGS 2» y «Watch Dogs 2», «Tom Clancy’s» y «Tom Clancy's» o «Ghostrunner2» y «Ghostrunner 2» sean el mismo.

accounts/matching.py, línea 61

```
def _name_keys(name, name_es, search_names)
```

Claves (name_key) de todos los nombres de un juego: el suyo, el de España y los otros que guarda para el buscador.

accounts/matching.py, línea 69

```
def _catalog_games(keys)
```

Juegos del catálogo que tienen alguna de las claves keys entre sus nombres, con sus plataformas ya cargadas.

La comparación no cabe en una consulta, así que recorre los nombres de todo el catálogo. Solo se hace cuando hay títulos que buscar, no en cada visita: si un día el catálogo tiene cientos de miles de juegos, habrá que guardar las claves.

accounts/matching.py, línea 74

```
def _title_parts(name)
```

Lo que se busca de un título: (clave de su nombre, clave sin el año o la región del final, nombre sin ellos, año o None).

accounts/matching.py, línea 85

```
def add_title(titles, external_id, name, minutes=0, last_played=None, platforms=(),
first_played=None, deck_minutes=0)
```

Añade un título importado al dict titles ({ID externo: datos}).

Si el ID ya estaba (el mismo juego en dos cuentas de Steam, o dos versiones con el mismo nombre) suma los minutos (y los jugados en una Steam Deck), se queda con la última partida más reciente y con la primera más antigua, y junta las plataformas, que son nombres de plataforma de IGDB («PlayStation 5», «Xbox One»...): sirven para desempatar al buscar por nombre y para rellenar la plataforma de las reseñas.

accounts/matching.py, línea 92

```
def _platform_names(game)
```

Nombres de las plataformas del juego (de IGDB), aprovechando el prefetch si lo hay.

accounts/matching.py, línea 113

```
def _fits(game, platforms)
```

Indica si el juego puede ser uno jugado en esas plataformas.

Sin plataformas conocidas (del título o del juego) cualquiera vale.

accounts/matching.py, línea 118

```
def _share_platform(names, platforms)
```

Indica si un juego que sale en las plataformas names puede ser uno jugado en platforms (ver _fits).

accounts/matching.py, línea 126

```
def _seen_games(keys)
```

Juegos de los títulos ya emparejados, de cualquier plataforma, que se llaman como alguna de las claves keys, como {clave: [Game]}: Steam los empareja por su ID de app y los demás pueden estar corregidos a mano, así que son una pista fiable.

accounts/matching.py, línea 131

```
def _score(game, group, platforms, year)
```

Lo bueno que es game para un título, para quedarse con el mayor: primero que sea del año que dice el título, después que salga en sus plataformas, después cómo se ha encontrado (group: cuanto más alto, más se parece el nombre), después el año más cercano y, por último, el más popular.

accounts/matching.py, línea 146

```
def match_by_name(titles)
```

Busca en el catálogo los juegos de {clave: (nombre limpio, plataformas)} y devuelve {clave: Game}.

Los nombres se comparan sin fijarse en cómo están escritos (name_key), con el nombre de cada juego, con sus otros nombres (el de España y los alternativos que guarda para el buscador) y con los títulos ya emparejados de otras plataformas (_seen_games). Un título que acaba en un año o una región («resident evil 4 (2005)», «LittleBigPlanet3 (LATAM)») también vale sin ellos. Primero mira lo que ya hay; después pide a IGDB por nombre exacto (tal cual y con mayúsculas de título, porque a veces vienen en mayúsculas) los que no tengan ningún juego que se llame así en sus plataformas y, para los que sigan sin ningún candidato, hace hasta MAX_SEARCHES búsquedas sueltas quedándose solo con los resultados que tienen ese nombre entre los suyos y salen en sus plataformas. Los lanzadores (LAUNCHERS) no se buscan. Si varios juegos encajan (remakes como God of War o Resident Evil 4), decide _score.

accounts/matching.py, línea 160

```
def save_titles(user, platform, titles, auto_match)
```

Guarda los títulos importados de un usuario en una plataforma y devuelve {ID externo: Game o None}.

titles es el dict que llena add_title. Los títulos que ya tienen juego, o que alguien dejó sin juego a mano, se respetan; los demás se buscan con auto_match(ids), que devuelve {ID: Game}. Los minutos, las partidas y las plataformas del usuario se sobrescriben con los de esta importación.

accounts/matching.py, línea 233

```
def update_library(user, platform, opened_only=False, record_activity=False)
```

Pasa a la biblioteca las horas de una plataforma a partir de todos los títulos importados del usuario.

Por juego suma los minutos de sus títulos (puede venir de varias cuentas o de la versión de PS4 y la de PS5) y toma la última partida más reciente. Los juegos nuevos entran como «Jugando» si se jugaron en los últimos días y como «Jugado» si no; sin horas ni partida, como «Pendiente», salvo con opened_only (plataformas que solo listan juegos abiertos alguna vez). Los que ya estaban conservan su estado (salvo «Pendiente», que pasa a jugado si hay horas). Si un título ha cambiado de juego desde la última vez (porque se corrigió el emparejamiento), al juego anterior se le quitan sus horas de esta plataforma. Si han cambiado horas, avisa al usuario de las reseñas que se le han quedado atrás (library.reminders). Con record_activity (lo piden las importaciones, no las correcciones del emparejamiento), los juegos nuevos se apuntan en su actividad como una sola importación (social.activity.games_imported) y lo que ha subido el tiempo de los que ya tenían horas de esta plataforma se apunta como sesiones de juego (library.models.PlaySession), con las que se mira si ha conseguido algún reto de horas (challenges.progress.check). Devuelve (juegos añadidos, juegos actualizados).

accounts/matching.py, línea 285

```
def reassign(imported, game, by)
```

Cambia el juego de un título importado (None = ninguno) para todos y recoloca las horas de quien lo cambia.

Lo hecho por un administrador queda como suyo («admin»); lo de los demás, como «user». Los demás usuarios con ese título lo verán en su próxima importación. Si el juego anterior se queda en la biblioteca de quien lo cambia sin horas, valoración ni reseña, se quita: solo estaba por el error. Lanza PermissionError si el título está bloqueado y quien lo cambia no es administrador.

Devuelve un dict con los juegos añadidos («added») y actualizados («updated») en su biblioteca y si se ha quitado el juego anterior («removed»).

accounts/matching.py, línea 357

accounts/models.py

```
def normalize_handle(network, value)
```

Limpia el usuario de una red social tal como lo escribe la gente.

Quita los espacios y la @ del principio y, si pega la URL de su perfil, se queda con el usuario (en Bluesky, lo que va después de /profile/). El usuario de Discord se pasa a minúsculas.

accounts/models.py, línea 48

```
def text_on(color)
```

Color del texto encima de un fondo del color color («#rrggbb»): blanco, como en el resto de la web, salvo en los colores claros (contraste con el blanco por debajo de 3:1, el mínimo de WCAG para texto en negrita), que llevan texto casi negro.

accounts/models.py, línea 64

```
def profile_upload_to(instance, filename)
```

Ruta de las imágenes de un perfil: perfiles/<id del usuario>/<nombre aleatorio que pone accounts.images>.

accounts/models.py, línea 79

Clase Profile

```
def __str__(self)
```

Texto del perfil en el admin y en la consola: su nombre visible.

accounts/models.py, línea 205

```
@property
def name(self)
```

Nombre que se muestra: el nombre visible si lo tiene, si no el nombre de usuario.

accounts/models.py, línea 210

```
def get_absolute_url(self)
```

URL del perfil público del usuario (/u/<usuario>/).

accounts/models.py, línea 214

```
def visible_to(self, viewer)
```

Indica si viewer puede ver el perfil: siempre si es público, y si es privado solo su dueño.

Es la regla de quién ve qué en todo NextPlay (perfil, biblioteca, reseñas, seguidores...); en las consultas la aplica visible_filter, que tiene que decir lo mismo.

accounts/models.py, línea 218

```
@staticmethod
def visible_filter(viewer, path='user')
```

La regla de visible_to como condición de consulta (Q): lo de los perfiles públicos y lo de viewer.

path es el camino desde lo que se filtra hasta su usuario: «user» en las reseñas (el de por defecto), «follower» en los seguidores...; vacío para filtrar los propios usuarios.

accounts/models.py, línea 227

```
@property
def steam_accounts(self)
```

Cuentas de Steam vinculadas, de la primera a la última (aprovecha el prefetch si lo hay).

accounts/models.py, línea 240

```
@property
def steam_linked(self)
```

Indica si el usuario tiene al menos una cuenta de Steam vinculada.

accounts/models.py, línea 245

```
@property
def steam_avatar(self)
```

Avatar de la primera cuenta de Steam vinculada, o cadena vacía si no tiene ninguna.

accounts/models.py, línea 250

```
@property
def psn(self)
```

Cuenta de PlayStation vinculada, o None si no tiene.

accounts/models.py, línea 256

```
@property
def platforms(self)
```

Cuentas de otras plataformas (Xbox, Switch, Epic...) que muestra en su perfil.

accounts/models.py, línea 264

```
@property
def xbox(self)
```

Cuenta de Xbox vinculada, o None si no tiene.

accounts/models.py, línea 269

```
@property
def avatar_url(self)
```

Foto del perfil según avatar_source: la subida o la de Steam, PlayStation o Xbox; vacía para la inicial.

En automático (o si la elegida ya no existe, p. ej. porque desvinculó la cuenta) prueba por orden la subida, la de Steam, la de PlayStation y la de Xbox. Cada una se mira solo si hace falta.

accounts/models.py, línea 277

```
@property
def accent(self)
```

Color del perfil en hexadecimal; el violeta de NextPlay si no ha elegido otro.

accounts/models.py, línea 297

```
@property
def accent_rgb(self)
```

El color del perfil como «r, g, b», para usarlo con transparencia en el CSS: rgba(var(--accent-rgb), .3).

accounts/models.py, línea 302

```
@property
def accent_text(self)
```

Color del texto encima del color del perfil (ver text_on).

accounts/models.py, línea 307

```
@property
def banner_url(self)
```

Imagen de la cabecera según banner_mode, o vacía para el degradado con el color del perfil.

Con un juego usa el arte de IGDB elegido o, si no hay, su portada (que se enseña desenfocada).

accounts/models.py, línea 312

```
@property
def banner_blurred(self)
```

Indica si la cabecera es la portada de un juego: es vertical y se desenfoca para que no se vea pixelada.

accounts/models.py, línea 324

```
@property
def background_url(self)
```

Imagen de fondo de la página del perfil según background_mode, o vacía si no tiene.

accounts/models.py, línea 329

```
def _uploaded_url(self, name)
```

URL de la imagen subida del campo name (avatar, banner o background), o vacía si no hay.

En la vista previa de Editar perfil manda la que se acaba de elegir y aún no se ha subido.

accounts/models.py, línea 337

```
@property
def background_blurred(self)
```

Indica si el fondo es la portada de un juego (la de la cabecera) y hay que desenfocarlo.

accounts/models.py, línea 346

```
@property
def background_dim_ratio(self)
```

Lo que se oscurece el fondo como número entre 0 y 1 con punto decimal, para el CSS.

accounts/models.py, línea 351

```
@property
def avatar_image(self)
```

La foto que se enseña, para accounts/_avatar.html: {"url", "framing", "fit"}, o {} si va la inicial.

El encuadre (ver _framing) solo vale para la foto subida: las de Steam, PlayStation y Xbox se enseñan tal cual.

accounts/models.py, línea 356

```
@property
def banner_framing(self)
```

Encuadre de la cabecera como variables de CSS (ver _framing).

accounts/models.py, línea 370

```
@property
def background_framing(self)
```

Encuadre del fondo de la página como variables de CSS (ver _framing).

accounts/models.py, línea 375

```
def _framing(self, name)
```

Encuadre de la imagen name (avatar, banner o background) como variables de CSS para su atributo style.

«--focus-x: 30%; --focus-y: 20%; --zoom: 1.5;», que app.css pasa a object-position y a la escala de la imagen. Vacío con el encuadre por defecto: centrada y sin zoom.

accounts/models.py, línea 379

```
@property
def section_layout(self)
```

Secciones del escaparate en el orden elegido: [(clave, nombre, visible)].

Las que no estén en sections (nuevas o nunca configuradas) van al final y visibles; las claves que ya no existen o repetidas se ignoran.

accounts/models.py, línea 391

```
@property
def visible_sections(self)
```

Claves de las secciones del escaparate que se enseñan, en su orden.

accounts/models.py, línea 406

```
@property
def social_links(self)
```

Redes del perfil para enseñarlas: [(clave, nombre, usuario o dominio, URL o vacía)].

Van en el orden de SOCIAL_LINKS y la web personal al final, con su dominio como texto.

accounts/models.py, línea 411

Clase FavoriteGame

Juego que el usuario enseña en «Juegos favoritos» de su perfil; como mucho MAX_PER_USER, en el orden que elija.

```
def __str__(self)
```

Texto en el admin: «usuario · juego».

accounts/models.py, línea 443

Clase SteamAccount

Cuenta de Steam vinculada a un usuario; un usuario puede tener varias.

```
def __str__(self)
```

Texto de la cuenta en el admin: su nombre en Steam, o el SteamID si no se conoce.

accounts/models.py, línea 468

```
@property
def name(self)
```

Nombre que se muestra: el de Steam, o el SteamID si aún no se ha podido leer.

accounts/models.py, línea 473

```
@property
def url(self)
```

URL del perfil en Steam; la construye con el SteamID si Steam no la ha dado.

accounts/models.py, línea 478

Clase PSNAccount

Cuenta de PlayStation Network vinculada a un usuario (una por usuario).

No se guarda ningún código ni token de Sony: el usuario pega su código NPSSO cada vez que sincroniza.

```
def __str__(self)
```

Texto de la cuenta en el admin: su PSN ID, o el ID de cuenta si no se conoce.

accounts/models.py, línea 501

```
@property
def name(self)
```

Nombre que se muestra: el PSN ID, o «PlayStation» si Sony no lo ha devuelto.

accounts/models.py, línea 506

```
@property
def url(self)
```

Perfil público en PSNProfiles (Sony no tiene página pública de perfiles), o vacío sin PSN ID.

accounts/models.py, línea 511

Clase PlatformAccount

Cuenta de otra plataforma que el usuario muestra en su perfil (sin importar juegos).

No se verifica que sea suya: es lo que escribe, como en la mayoría de webs. Una por plataforma.

```
def __str__(self)
```

Texto de la cuenta en el admin: «Plataforma: nombre».

accounts/models.py, línea 566

```
@property
def logo_template(self)
```

Ruta de la plantilla SVG con el logo de la plataforma.

accounts/models.py, línea 571

```
@property
def url(self)
```

Perfil público en la web de la plataforma cuando existe (solo GOG); si no, vacío.

accounts/models.py, línea 576

```
@classmethod
def normalize(cls, platform, handle)
```

Limpia lo que escribe el usuario: quita espacios de los extremos y pone el código de Switch en formato SW-1234-5678-9012 aunque lo escriba solo con números o con espacios.

accounts/models.py, línea 583

```
@classmethod
def is_valid_handle(cls, platform, handle)
```

Indica si el nombre o código cumple el formato de esa plataforma.

accounts/models.py, línea 594

Clase XboxAccount

Cuenta de Xbox (cuenta Microsoft) vinculada a un usuario (una por usuario).

No se guarda ningún token de Microsoft: para sincronizar el usuario vuelve a iniciar sesión.

```
def __str__(self)
```

Texto de la cuenta en el admin: su gamertag.

accounts/models.py, línea 617

```
@property
def name(self)
```

Nombre que se muestra: el gamertag, o «Xbox» si Microsoft no lo ha devuelto.

accounts/models.py, línea 622

```
@property
def url(self)
```

Perfil público del gamertag en xbox.com, o vacío si no se conoce.

accounts/models.py, línea 627

Clase ExternalTitle

Juego tal como lo identifica una plataforma (app de Steam, titleId de PlayStation o de Xbox) y el juego del catálogo que le corresponde (vacío si no se encontró).

Es compartido entre usuarios: la importación lo mira antes de buscar por nombre, así que una corrección hecha a mano sirve para todos. Un administrador puede bloquearlo para que nadie más lo cambie.

```
def __str__(self)
```

Texto del título en el admin: «Plataforma: nombre».

accounts/models.py, línea 679

Clase ImportedTitle

Título que tiene un usuario en una plataforma, con los minutos y la última partida de su última importación.

De aquí salen las horas de cada plataforma en la biblioteca, así que al corregir el juego de un título se pueden recolocar sin volver a importar.

```
def __str__(self)
```

Texto en el admin: «usuario · título».

accounts/models.py, línea 723

```
@property
def hours(self)
```

Horas jugadas redondeadas a una décima.

accounts/models.py, línea 728

```
def set_achievements(self, unlocked, total, platinum=False, at=None)
```

Apunta los logros leídos de la plataforma (sin guardar): unlocked de total, con o sin platino, leídos en at (ahora, si no se da). Devuelve True si ha cambiado algo de lo que se enseña.

accounts/models.py, línea 732

Clase Achievement

Logro de un juego en su plataforma (de momento, solo de Steam), compartido por todos los que lo tienen: su nombre y su descripción en castellano, sus iconos, si es oculto y qué parte de los jugadores de la plataforma lo tiene (percent, de donde sale su rareza). Lo guarda steam.import_achievements la primera vez que alguien tiene logros de ese juego; lo que ha conseguido cada uno va en UnlockedAchievement.

```
def __str__(self)
```

Texto en el admin: «Plataforma: juego · logro».

accounts/models.py, línea 778

```
@property
def rarity(self)
```

Nombre de su rareza (RARITY_TIERS): «Rarísimo», «Muy raro» o «Raro», o vacío si es común o no se sabe.

accounts/models.py, línea 783

Clase UnlockedAchievement

Logro que ha conseguido un usuario en uno de sus títulos importados y cuándo. Algunos logros muy antiguos de Steam no tienen fecha. Lo apunta steam.import_achievements cada vez que lee los logros de ese juego.

```
def __str__(self)
```

Texto en el admin: «usuario · logro».

accounts/models.py, línea 812

Clase Notification

Aviso para un usuario: se ve en la campana de la cabecera hasta que abre la página de avisos.

```
def __str__(self)
```

Texto del aviso en el admin: «usuario · texto».

accounts/models.py, línea 856

accounts/psn.py

Importación de juegos y horas de PlayStation (PS4/PS5) con la API no oficial de la app de PlayStation.

Sony no tiene API pública: se usa la misma que la app oficial, a la que se accede con el código NPSSO de la sesión del usuario en playstation.com. El código no se guarda; se pide en cada sincronización. Es experimental: Sony puede cambiarla sin avisar.

Clase PSNError

Fallo al hablar con PlayStation (red, respuesta inesperada o API cambiada).

Clase PSNAuthError

Sony no acepta el código NPSSO (mal copiado o caducado).

Clase PSNRejectedError

PlayStation rechaza lo que se le pide (400 o 404): p. ej. un ID de juego que no conoce.

```
def clean_npsso(value)
```

Saca el código NPSSO de lo que pegue el usuario: el código suelto o el JSON entero de la página de Sony.

Devuelve el código o None si no tiene el formato esperado (64 letras y números).

accounts/psn.py, línea 58

```
def access_token(npsso)
```

Cambia el código NPSSO por un token de acceso de la API de PlayStation.

Lanza PSNAuthError si Sony no da el código de autorización (NPSSO incorrecto o caducado).

accounts/psn.py, línea 72

```
def _get(url, token, **params)
```

GET autenticado contra la API de PlayStation; devuelve el JSON. Lanza PSNRejectedError si PlayStation rechaza lo que se le pide (400 o 404) y PSNError con cualquier otro fallo.

accounts/psn.py, línea 114

```
def _account_id_from_token(token)
```

ID de cuenta guardado dentro del token JWT, por si el perfil no lo devuelve; None si no está.

accounts/psn.py, línea 130

```
def profile(token)
```

Datos de la cuenta del token: {"account_id", "online_id", "avatar"}.

Si el perfil no responde, intenta sacar al menos el ID de cuenta del propio token.

accounts/psn.py, línea 141

```
def played_games(token)
```

Juegos de PS4 y PS5 que ha jugado la cuenta, con «name», «playDuration», «lastPlayedDateTime»...

accounts/psn.py, línea 161

```
def trophy_titles(token)
```

Juegos con trofeos de la cuenta (también de PS3 y PS Vita): cada uno con su nombre («trophyTitleName») y, por tipo (bronze, silver, gold y platinum), los trofeos que tiene («definedTrophies») y los conseguidos («earnedTrophies»).

accounts/psn.py, línea 174

```
def trophies_by_title_id(token, title_ids)
```

Listas de trofeos de los juegos title_ids a partir de su ID de PSN: {ID: [listas como las de trophy_titles]}.

Solo se piden los ID de verdad (TITLE_ID: los títulos que llegan sin ID se importan con su nombre), TITLE_IDS_PER_REQUEST por petición y TROPHY_WORKERS peticiones a la vez. Una colección trae una lista por cada juego que incluye. Si PlayStation rechaza una petición (un ID que no conoce la tumba entera), sus juegos se piden de uno en uno; los que aun así fallan, y los de una petición que falla por otra cosa (la red, el token), se quedan fuera y se buscan por nombre (save_trophies).

accounts/psn.py, línea 187

```
def trophy_key(name)
```

Clave para emparejar un juego con su lista de trofeos por el nombre: sin coletillas de plataforma y sin fijarse en cómo está escrito (name_key), para que «Tom Clancy’s» y «Tom Clancy's» o «Ghostrunner2» y «Ghostrunner 2» sean el mismo.

accounts/psn.py, línea 213

```
def trophy_count(lists)
```

(conseguidos, totales, platino) de una o varias listas de trofeos sumadas (las de los juegos de una colección), o None si no tienen ningún trofeo.

accounts/psn.py, línea 219

```
def save_trophies(user, trophies, by_title=None)
```

Apunta en los títulos de PlayStation importados del usuario sus trofeos y devuelve en cuántos.

Primero, los de su ID de juego (by_title, de trophies_by_title_id): una colección suma las listas de sus juegos y cuenta como platino si tiene alguno. Los que no salen así se buscan por nombre (trophy_key) en la lista de trofeos de la cuenta (trophies, de trophy_titles): el del título o el del juego del catálogo que tiene y, si con ellos no aparece, sin el año o la región del final (YEAR_OR_REGION). Si un juego tiene trofeos aparte en PS4 y en PS5, por nombre cuenta la versión que más lleva (primero el platino). Los títulos sin trofeos leídos se quedan como estaban.

accounts/psn.py, línea 229

```
def duration_minutes(value)
```

Minutos de una duración ISO 8601 de PSN («PT228H56M33S» → 13736); 0 si no se entiende.

accounts/psn.py, línea 263

```
def import_library(user, npsso)
```

Vincula (o actualiza) la cuenta de PSN del NPSSO e importa sus juegos de PS4/PS5 con sus horas.

Los nuevos entran como «Jugando» si se jugaron en los últimos 14 días y como «Jugado» si no; los que ya tenía conservan su estado y solo se les actualizan las horas de PlayStation. La versión de PS4 y la de PS5 de un juego se suman. Lanza PSNAuthError si el código no vale y ValueError si esa cuenta de PSN ya es de otro usuario.

Con el mismo token lee después los trofeos de cada juego (trophies_by_title_id, trophy_titles y save_trophies); si fallan, la importación de los juegos sigue valiendo.

Devuelve un dict con la cuenta («account»), cuántos juegos se han añadido («added»), actualizado («updated») y no se han encontrado en el catálogo («unmatched»), y de cuántos se han leído los trofeos («trophies»).

accounts/psn.py, línea 276

accounts/signals.py

```
@receiver(post_save, sender=settings.AUTH_USER_MODEL)
def create_profile(sender, instance, created, raw=False, **kwargs)
```

Crea el perfil de un usuario justo después de crearlo.

No hace nada al guardar un usuario ya existente ni al cargar fixtures (raw=True).

accounts/signals.py, línea 9

```
@receiver(post_delete, sender=Profile)
def delete_profile_images(sender, instance, **kwargs)
```

Borra del disco la foto, la cabecera y el fondo de un perfil eliminado (p. ej. al borrar la cuenta).

accounts/signals.py, línea 19

accounts/steam.py

Vinculación con Steam (inicio de sesión OpenID) e importación de los juegos de su biblioteca y de su lista de deseos.

Clase SteamError

Fallo al hablar con Steam (red, clave de la API o respuesta no válida).

Clase SteamPrivateError

Steam no devuelve los juegos porque los detalles del perfil del usuario no son públicos.

```
def is_configured()
```

Indica si está puesta la clave de la Web API de Steam, necesaria para leer la biblioteca.

accounts/steam.py, línea 56

```
def login_url(return_to, realm)
```

URL de Steam donde el usuario inicia sesión; Steam lo devuelve después a return_to.

accounts/steam.py, línea 61

```
def verify_login(params, return_to)
```

Comprueba con Steam la respuesta de OpenID y devuelve el SteamID64 del usuario, o None si no vale.

Rechaza respuestas de otro proveedor, para otra URL de vuelta o que Steam no confirme como suyas (así nadie puede inventarse una URL de vuelta con el SteamID de otro).

accounts/steam.py, línea 74

```
def _api(path, **params)
```

Llama a un método GET de la Web API de Steam con la clave de settings y devuelve su «response».

accounts/steam.py, línea 99

```
def player_summary(steam_id)
```

Nombre, avatar y URL del perfil de Steam: {"persona", "avatar", "profile_url"}, o {} si no lo encuentra.

accounts/steam.py, línea 113

```
def owned_games(steam_id)
```

Juegos de la cuenta de Steam: [{"appid", "name", "playtime_forever", "rtime_last_played"...}].

Lanza SteamPrivateError si el perfil no deja ver sus juegos (Steam responde sin «games»).

accounts/steam.py, línea 126

```
def wishlist(steam_id)
```

Lista de deseos de la cuenta de Steam: [{"appid", "priority", "date_added"}], o None si no se puede leer.

No necesita la clave de la API (se manda igual). Steam contesta lo mismo, sin «items», a una lista privada que a una vacía, así que las dos dan None.

accounts/steam.py, línea 144

```
def refresh_account(account)
```

Actualiza el nombre, el avatar y la URL guardados de una cuenta de Steam (sin fallar si Steam no responde).

accounts/steam.py, línea 153

```
def _match_games(app_ids)
```

Busca los juegos del catálogo de una lista de IDs de Steam y devuelve {ID de Steam: Game}.

Primero mira el catálogo local; los que falten los busca en IGDB y los importa (si IGDB está configurado).

accounts/steam.py, línea 165

```
def _timestamp(value)
```

Fecha de una marca de tiempo de Steam (segundos desde 1970), o None si no viene o es 0.

accounts/steam.py, línea 188

```
def _played_at(data)
```

Fecha de la última partida de un juego de Steam, o None si nunca se ha jugado.

accounts/steam.py, línea 193

```
def import_library(user, include_unplayed=False)
```

Importa a la biblioteca del usuario los juegos que ha jugado en todas sus cuentas de Steam vinculadas.

Si un juego está en varias cuentas (o en varias apps de Steam, como un relanzamiento) se suman sus horas y se toma la última partida más reciente. Los nuevos entran como «Jugando» si se jugaron en los últimos 14 días y como «Jugado» si no; los que ya tenía conservan su estado y solo se les actualizan las horas. Con include_unplayed también entran los que nunca ha abierto, como «Pendiente». Cada app queda guardada como título importado, así que las que no se encuentren se pueden emparejar a mano. Las cuentas que no dejan ver sus juegos se saltan; si no deja ninguna, lanza SteamPrivateError.

Después lee los logros de los juegos jugados hace menos (import_achievements).

Devuelve un dict con cuántos juegos se han añadido («added»), actualizado («updated») y no se han encontrado en el catálogo («unmatched»), los nombres de las cuentas privadas («private») y de cuántos juegos se han leído los logros («achievements»).

accounts/steam.py, línea 198

```
def player_achievements(steam_id, app_id)
```

Logros de un juego en una cuenta de Steam, todos y en el orden de Steam: [(clave, conseguido, fecha)], con la fecha en que lo consiguió o None si no lo tiene (o si es de los muy antiguos, de los que Steam no guarda la fecha); [] si el juego no tiene logros o None si no se pueden leer (perfil o detalles de juego privados, o Steam no responde).

A un juego sin logros Steam le contesta con un 400 y «Requested app has no stats», así que no se mira el código de la respuesta sino su «playerstats».

accounts/steam.py, línea 254

```
def _icon(url)
```

Dirección actual de un icono de logro de Steam (ICON_URL en vez de OLD_ICON_PATH), o vacía si no viene.

accounts/steam.py, línea 279

```
def achievement_schema(app_id)
```

Logros de un juego de Steam con sus textos en castellano, en el orden de Steam: [{"api_name", "name", "description", "icon", "icon_locked", "hidden"}], [] si no tiene o None si Steam no responde.

De los ocultos Steam no da la descripción (tampoco en los logros de cada jugador), así que se queda vacía. Los iconos van con su dirección actual (_icon).

accounts/steam.py, línea 284

```
def global_percentages(app_id)
```

Porcentaje de los jugadores de Steam que tiene cada logro de un juego, redondeado a una décima: {clave: porcentaje}, o None si Steam no lo da. No necesita la clave de la API, y el porcentaje llega como texto («63.1»).

accounts/steam.py, línea 314

```
def achievement_info(app_id)
```

Logros de un juego de Steam con su rareza, para achievements.save_info: (esquema, porcentajes) como los dan achievement_schema y global_percentages (los porcentajes, None si no se han podido leer), o None si Steam no da el esquema.

accounts/steam.py, línea 326

```
def _needs_info(title, result, known, now)
```

Si hay que (volver a) leer los logros del juego de title: si nunca se han leído, si se leyeron hace más de ACHIEVEMENTS_INFO_DAYS días o si el jugador trae alguno que no está. result es lo que da player_achievements y known, las (título, clave) que ya hay.

accounts/steam.py, línea 336

```
def import_achievements(user, limit=ACHIEVEMENTS_PER_IMPORT)
```

Lee de Steam los logros de los juegos del usuario que ha jugado desde la última vez que se leyeron (o que nunca se han leído), de la partida más reciente a la más antigua y como mucho limit (None: todos), y los apunta en sus títulos importados (ImportedTitle): cuántos tiene y, uno a uno, cuáles y cuándo los consiguió (achievements.save_unlocked). Con varias cuentas de Steam se queda con la que más lleva de cada juego.

De los juegos cuyos logros aún no se conocen (o con más de ACHIEVEMENTS_INFO_DAYS días, o si el jugador trae uno nuevo) lee también cuáles tienen, con sus textos en castellano y su rareza (achievement_info). Lo que no se puede leer se intenta la próxima vez: también los logros de un jugador si no se sabe aún cuáles son los de su juego. Van ACHIEVEMENTS_WORKERS peticiones a la vez.

Devuelve de cuántos juegos se han leído.

accounts/steam.py, línea 345

```
def _first(values)
```

El menor de los valores que no son None (el mejor puesto o la fecha más antigua), o None si no hay ninguno.

accounts/steam.py, línea 409

```
def _wishlist_games(app_ids)
```

Juegos del catálogo de unos IDs de Steam de la lista de deseos: {ID: Game o None}.

Si alguien corrigió a mano el juego de esa app (ExternalTitle, también para decir que no es un juego), manda la corrección; los demás se buscan como los de la biblioteca (_match_games).

accounts/steam.py, línea 415

```
def import_wishlist(user)
```

Trae a «Lo quiero» los juegos de la lista de deseos de todas las cuentas de Steam del usuario.

Los juegos se buscan por su ID de Steam como los de la biblioteca. No se añaden los que ya están en su biblioteca ni los que quitó a mano de «Lo quiero» (Profile.steam_wishlist_skip); de los que ya estaban en «Lo quiero» solo se apuntan su puesto y su fecha en Steam. Si se han podido leer todas sus listas, se quitan los que trajo una importación anterior y ya no están en Steam (los ha comprado o quitado allí); los que añadió él en NextPlay nunca se quitan. Un juego en varias cuentas (o en varias apps, como un relanzamiento) se queda con el mejor puesto y la fecha más antigua.

Devuelve un dict con los deseados de Steam («total»), los WishlistItem nuevos («added», una lista), cuántos se han quitado («removed»), cuántos no se traen porque ya los tiene («owned») o porque los quitó él («skipped»), cuántos no están en el catálogo («unmatched») y los nombres de las cuentas cuya lista no se pudo leer, porque es privada o está vacía («unreadable»).

accounts/steam.py, línea 431

accounts/views.py

```
def signup(request)
```

Registro de cuenta nueva.

Si el usuario ya ha iniciado sesión lo manda a su perfil. Al registrarse bien inicia su sesión y lo lleva al catálogo para que añada sus primeros juegos.

accounts/views.py, línea 52

```
def profile(request, username)
```

Perfil público de un usuario: cabecera personalizada, escaparate, su biblioteca y sus reseñas.

El escaparate (favoritos, lo que está jugando, reseña destacada, estadísticas y géneros) sale en el orden y con las secciones que haya elegido. La biblioteca tiene pestañas por estado (Jugando, Completado...) con su contador y se filtra con ?estado= y por idioma con ?idioma= (voces o textos en castellano); se ordena con ?orden= (recientes, nota, horas o nombre). Con ?vista=resenas muestra sus reseñas en vez de la biblioteca, con ?vista=listas sus listas, con ?vista=tier-lists sus tier lists y con ?vista=actividad lo que ha hecho (desde ?antes=, el cursor de «Cargar más»). Un perfil privado da 404 a cualquiera que no sea su dueño.

accounts/views.py, línea 69

```
@require_POST
@login_required
def profile_preview(request)
```

Vista previa de Editar perfil: la página del perfil tal como quedaría con lo que hay en el formulario.

La pide el JavaScript de Editar perfil cada vez que se cambia algo y la enseña a la vez en un marco del ancho de un ordenador y en otro del de un móvil. No guarda nada (ver ProfileForm.preview). Sale como la ven los demás, sin los botones del dueño, aunque el perfil sea privado.

accounts/views.py, línea 89

```
def _profile_owners()
```

Usuarios con lo que enseña la cabecera del perfil ya cargado: perfil, juego de la cabecera y cuentas vinculadas.

accounts/views.py, línea 102

```
def _profile_context(request, owner, favorites=None, preview=False)
```

Variables de la plantilla del perfil de owner, con la biblioteca filtrada y ordenada según ?estado=, ?idioma= y ?orden=.

favorites son sus juegos favoritos en orden (si no se dan, los guardados). Con preview es la vista previa de Editar perfil: se ve como la ven los demás (también sus listas: solo las públicas), el usuario de la barra superior es owner con los cambios sin guardar y la biblioteca se corta en PREVIEW_LIBRARY juegos. También lleva cuántos seguidores y seguidos tiene owner y si quien mira sigue a owner o al revés (social.views.follow_context), y sus listas y, aparte, sus tier lists (lists.views.profile_lists), para sus pestañas y el escaparate; las tier lists que se enseñan llevan su tablero en miniatura (lists.views.mark_boards). Con sus logros importados (accounts.achievements), cada juego de la biblioteca lleva su insignia de «Platino» o «100 %» (achievement_badge) y las cifras de la cabecera, sus platinos y sus juegos al 100 %; si el escaparate enseña la sección «Logros», lleva también sus últimos logros y el más raro (achievements.showcase).

accounts/views.py, línea 109

```
def _profile_stats(owner, entries, counts, reviews)
```

Cifras de la cabecera del perfil: juegos, completados, horas jugadas (Steam + PlayStation + Xbox) y las de cada plataforma (hours_by_platform, de la que tiene más a la que menos y solo las que tienen alguna), nota media (en estrellas: las notas van en medias estrellas) y reseñas publicadas (reviews, ya contadas).

También calcula los tres géneros más repetidos de su biblioteca.

accounts/views.py, línea 238

```
@login_required
def edit_profile(request)
```

Editar perfil: datos, apariencia (foto, cabecera, fondo, color y diseño), escaparate, redes y privacidad.

El formulario trabaja con una copia del perfil para que, si no es válido, la página (las imágenes subidas, la barra superior) siga enseñando lo guardado. Al lado va la vista previa del perfil en un ordenador y en un móvil, que pinta profile_preview con lo que hay en el formulario. Las tiendas y las cuentas vinculadas tienen sus propios formularios en la misma página.

accounts/views.py, línea 266

```
def _posted_sections(form)
```

Secciones tal como venían en un envío no válido (orden y visibles), para no perder lo que movió; [] si no hay.

accounts/views.py, línea 305

```
def favorite_context(user, game, error='')
```

Variables del botón «Añadir a mis favoritos» de la ficha de game para user.

accounts/views.py, línea 315

```
@require_POST
@login_required
def favorite_toggle(request, game_id)
```

Añade el juego a los favoritos del perfil del usuario o lo quita si ya estaba.

Como mucho FavoriteGame.MAX_PER_USER: con el cupo lleno avisa de que quite alguno. Los nuevos van al final. Con HTMX devuelve el botón actualizado; si no, vuelve a la ficha del juego.

accounts/views.py, línea 328

```
def game_arts(game)
```

Imágenes apaisadas de IGDB (arte y capturas) de un juego para la cabecera: [{"id", "thumb", "full"}].

«thumb» es la miniatura para elegirla y «full», el tamaño con el que se ve en la cabecera.

Se guardan un día en caché. Lista vacía si no hay juego, el juego no viene de IGDB, IGDB no está configurado o no responde.

accounts/views.py, línea 355

```
@login_required
def game_achievements(request, game_id)
```

«Tus logros» de un juego, uno a uno (fragmento HTML que la ficha pide con HTMX al desplegarlos y al cambiar de orden con ?orden=): los que tiene quien mira, con cuándo los consiguió, y los que le faltan, con su rareza (accounts.achievements.detail). Da 404 si no tiene logros de ese juego leídos uno a uno.

accounts/views.py, línea 381

```
@login_required
def banner_arts(request)
```

Opciones de imagen para la cabecera con el juego elegido en Editar perfil (fragmento HTML).

El formulario lo pide con HTMX al cargar y al cambiar de juego (?banner_game=). Solo vale con juegos de la biblioteca del usuario o con el que ya tiene en la cabecera.

accounts/views.py, línea 393

```
@require_POST
@login_required
def banner_from_game(request, game_id)
```

Pone de cabecera del perfil una imagen de la ficha de un juego (el botón del visor del carrusel).

Como en Editar perfil, solo con juegos de la biblioteca del usuario, y solo con imágenes apaisadas de la ficha (games.media.banner_arts). El encuadre vuelve al centro, como con cualquier imagen nueva. Con HTMX devuelve el aviso del visor; si no, vuelve a la ficha con un mensaje.

accounts/views.py, línea 422

```
def linking_context()
```

Variables que necesitan los paneles de vinculación de cuentas (en Editar perfil y en el modal del perfil).

accounts/views.py, línea 456

```
def _back(request, default)
```

Redirige a la página de la que vino el formulario (campo «next») si es de esta web; si no, a default.

Así las acciones lanzadas desde el modal del perfil vuelven al perfil y no a Editar perfil.

accounts/views.py, línea 471

```
def _steam_return_to(request, state)
```

URL absoluta a la que Steam devuelve al usuario tras iniciar sesión, con el código anti-CSRF state.

accounts/views.py, línea 482

```
def _report_import(request, result)
```

Avisa al usuario de cuántos juegos se han importado de Steam, cuántos no estaban en el catálogo y qué cuentas se han saltado por ser privadas.

accounts/views.py, línea 487

```
def _achievements_note(count, noun='logros')
```

Coletilla del mensaje de una importación con los juegos de los que se han leído los logros (o los trofeos).

accounts/views.py, línea 502

```
def _with_unmatched(text, unmatched)
```

Mensaje de resultado de una importación, con el aviso de los juegos no encontrados y un enlace para emparejarlos a mano (HTML seguro: el texto se escapa).

accounts/views.py, línea 507

```
def _report_wishlist(request, result)
```

Avisa de lo que se ha traído de la lista de deseos de Steam y, con los precios de hoy, de cuántos de sus deseados están en su mínimo histórico o son más baratos fuera de Steam (HTML seguro: el texto se escapa).

Los precios de los juegos nuevos se piden en ese momento (games.prices.fetch_missing).

accounts/views.py, línea 518

```
def _fetch_library_prices(user)
```

Pide ya los precios de los juegos de la biblioteca del usuario que nunca se han buscado (games.prices.fetch_missing), para que lo recién importado tenga precio sin esperar a la tarea de cada noche.

Si una tienda no responde, la importación vale igual: lo que falte llega con update_prices. La sincronización de cada noche (sync_libraries) no lo hace, porque justo después va update_prices con todo el catálogo.

accounts/views.py, línea 552

```
def _import_steam(request, include_unplayed=False)
```

Lanza la importación de todas las cuentas de Steam del usuario y le avisa del resultado o del fallo.

Si lo tiene activado (Profile.steam_wishlist), trae también su lista de deseos de Steam a «Lo quiero»; que falle eso no estropea lo de la biblioteca. Si falla por Steam o IGDB deja el error completo en el log del servidor para poder revisarlo. Si sale bien, pide los precios de los juegos que nunca los han tenido (_fetch_library_prices).

accounts/views.py, línea 564

```
@require_POST
@login_required
def steam_link(request)
```

Manda al usuario a iniciar sesión en Steam para vincular una cuenta más (Steam nunca nos da su contraseña).

No deja pasar de SteamAccount.MAX_PER_USER cuentas.

accounts/views.py, línea 595

```
@login_required
def steam_callback(request)
```

Vuelta desde Steam: verifica el inicio de sesión, añade la cuenta a las del usuario e importa sus juegos.

Rechaza la vuelta si el código anti-CSRF no coincide, si Steam no la confirma o si esa cuenta de Steam ya está vinculada a otro usuario. Si ya era suya, solo avisa.

accounts/views.py, línea 608

```
@require_POST
@login_required
def steam_import(request)
```

Vuelve a importar los juegos de todas las cuentas de Steam vinculadas (con «unplayed» también los no jugados).

accounts/views.py, línea 650

```
@require_POST
@login_required
def steam_auto_sync(request)
```

Activa (auto_sync=1) o desactiva la sincronización nocturna de la biblioteca de Steam del usuario.

accounts/views.py, línea 664

```
@require_POST
@login_required
def steam_wishlist(request)
```

Activa (steam_wishlist=1) o desactiva que la importación de Steam traiga la lista de deseos a «Lo quiero».

Con restore=1, olvida los juegos que quitó a mano para que la próxima importación los vuelva a traer.

accounts/views.py, línea 678

```
@require_POST
@login_required
def steam_unlink(request, account_id)
```

Desvincula una cuenta de Steam del usuario; los juegos importados se quedan en su biblioteca.

Solo puede desvincular cuentas suyas: con el ID de una cuenta ajena responde 404.

accounts/views.py, línea 700

```
def _psn_error(request, text)
```

Avisa del error de PlayStation y vuelve a donde estaba el formulario (por defecto, Editar perfil).

accounts/views.py, línea 713

```
@require_POST
@login_required
def psn_sync(request)
```

Vincula la cuenta de PlayStation del código NPSSO pegado (o la actualiza) e importa sus juegos de PS4/PS5.

El código solo se usa durante esta petición: no se guarda. Avisa si el código no tiene el formato esperado, si Sony no lo acepta o si esa cuenta de PlayStation ya es de otro usuario. Después pide los precios de los juegos que nunca los han tenido (_fetch_library_prices).

accounts/views.py, línea 721

```
@require_POST
@login_required
def psn_unlink(request)
```

Desvincula la cuenta de PlayStation del usuario; los juegos importados se quedan en su biblioteca.

accounts/views.py, línea 757

```
@require_POST
@login_required
def platform_add(request)
```

Añade la cuenta de otra plataforma (Xbox, Switch, Epic, GOG, Battle.net) o cambia la que ya tenía.

Si el formato no es el de esa plataforma avisa del error y no guarda nada.

accounts/views.py, línea 766

```
@require_POST
@login_required
def platform_remove(request, platform)
```

Quita del perfil del usuario su cuenta de esa plataforma (no da error si no la tenía).

accounts/views.py, línea 787

```
def _xbox_redirect_uri(request)
```

URL absoluta de vuelta desde Microsoft; tiene que coincidir con la registrada en la aplicación de Azure.

accounts/views.py, línea 797

```
@require_POST
@login_required
def xbox_link(request)
```

Manda al usuario a iniciar sesión con su cuenta Microsoft para vincular Xbox o volver a sincronizarla.

Sirve para las dos cosas porque no se guarda ningún token: cada sincronización pide iniciar sesión.

accounts/views.py, línea 804

```
@login_required
def xbox_callback(request)
```

Vuelta desde Microsoft: comprueba el código anti-CSRF, vincula la cuenta de Xbox e importa sus juegos.

Si el usuario canceló el inicio de sesión, si el código no coincide o si Xbox Live falla, avisa y vuelve a Editar perfil. Después pide los precios de los juegos que nunca los han tenido (_fetch_library_prices).

accounts/views.py, línea 817

```
@require_POST
@login_required
def xbox_unlink(request)
```

Desvincula la cuenta de Xbox del usuario; los juegos importados se quedan en su biblioteca.

accounts/views.py, línea 853

```
def _search_games(query)
```

Juegos del catálogo cuyo nombre contiene query, de más a menos popular, completando con IGDB si salen pocos.

accounts/views.py, línea 866

```
@login_required
def imported_titles(request)
```

Juegos importados para revisar a mano.

Sin parámetros, los títulos de Steam, PlayStation o Xbox del usuario que no se encontraron en el catálogo (con ?ocultos=1, los que ocultó). Con ?juego=<id>, los que han llevado a ese juego de su biblioteca, para corregirlo si no es el que jugó. Con ?titulo=<id>&q= busca candidatos para ese título (por defecto, por su nombre).

accounts/views.py, línea 880

```
@require_POST
@login_required
def imported_title_assign(request, pk)
```

Asigna a un título importado del usuario el juego elegido (o ninguno si «game» llega vacío).

La corrección sirve para todos los usuarios con ese título y sus horas pasan al juego nuevo en su biblioteca. Responde 404 si el título no es suyo o el juego no existe.

accounts/views.py, línea 922

```
@require_POST
@login_required
def imported_title_hide(request, pk)
```

Oculta (hidden=1) un título sin emparejar del usuario —herramientas, demos, apps— o lo vuelve a mostrar.

accounts/views.py, línea 949

```
@login_required
def notifications(request)
```

Últimos avisos del usuario (bajadas de precio, nuevos seguidores...). Al abrir la página, los no leídos pasan a leídos.

Los que tienen quién los ha provocado (actor) traen ya lo que hace falta para enseñar su foto.

accounts/views.py, línea 963

```
@require_POST
@login_required
def shops_update(request)
```

Guarda las tiendas en las que compra el usuario (casillas «shops»). Ninguna o todas = enseñar todas, así las tiendas que aparezcan más adelante también se ven.

accounts/views.py, línea 979

accounts/xbox.py

Vinculación con Xbox (inicio de sesión con cuenta Microsoft) e importación de sus juegos y minutos jugados.

Flujo: el usuario inicia sesión en Microsoft (OAuth 2.0 de una aplicación de Azure con el permiso XboxLive.signin) → con el token de Microsoft se pide uno de usuario de Xbox Live → con ese, uno XSTS que trae el XUID y el gamertag → con el XSTS se leen el historial de juegos y la estadística MinutesPlayed. Ningún token se guarda: se usan durante la petición y se tiran.

Clase XboxError

Fallo al hablar con Microsoft o Xbox Live (red, credenciales o respuesta inesperada).

```
def is_configured()
```

Indica si están puestas las credenciales de la aplicación de Azure para iniciar sesión con Microsoft.

accounts/xbox.py, línea 51

```
def login_url(redirect_uri, state)
```

URL de Microsoft donde el usuario inicia sesión; Microsoft lo devuelve a redirect_uri con un código.

accounts/xbox.py, línea 56

```
def _post(url, **kwargs)
```

POST a Microsoft o Xbox Live que devuelve el JSON; lanza XboxError si falla.

accounts/xbox.py, línea 69

```
def xbox_tokens(code, redirect_uri)
```

Cambia el código de Microsoft por los datos de Xbox Live: {"xuid", "gamertag", "auth"}.

«auth» es la cabecera Authorization (XBL3.0) para llamar a las APIs de Xbox Live.

accounts/xbox.py, línea 89

```
def _get(url, auth, contract, **params)
```

GET autenticado contra una API de Xbox Live con su versión de contrato; devuelve el JSON.

accounts/xbox.py, línea 127

```
def avatar(xuid, auth)
```

URL del avatar (gamerpic) de la cuenta, o vacío si Xbox Live no la da.

accounts/xbox.py, línea 142

```
def played_titles(xuid, auth)
```

Juegos del historial de la cuenta (solo de tipo «Game», sin apps como Netflix), con su última partida.

accounts/xbox.py, línea 152

```
def minutes_played(xuid, auth, title_ids)
```

Minutos jugados por título según la estadística MinutesPlayed: {titleId: minutos}.

Los juegos que no publican esa estadística (muchos de Xbox 360 y algunos de PC) no aparecen.

accounts/xbox.py, línea 158

```
def save_achievements(user, played)
```

Apunta en los títulos de Xbox importados del usuario los logros que llegan con su historial (played, de played_titles: «currentAchievements» de «totalAchievements») y devuelve en cuántos.

accounts/xbox.py, línea 183

```
def import_library(user, code, redirect_uri)
```

Vincula (o actualiza) la cuenta de Xbox del inicio de sesión e importa sus juegos con los minutos jugados.

Los juegos se emparejan con el catálogo por nombre (y por los dispositivos en los que se jugaron). Los nuevos entran como «Jugando» si se jugaron en los últimos 14 días y como «Jugado» si no; los que ya tenía conservan su estado (salvo «Pendiente», que pasa a jugado si hay horas) y se les actualizan los minutos de Xbox. Si el usuario tenía puesto a mano un gamertag en «Otras plataformas», se quita porque ya está la cuenta verificada. Lanza ValueError si esa cuenta de Xbox ya es de otro usuario.

Devuelve un dict con la cuenta («account»), cuántos juegos se han añadido («added»), actualizado («updated») y no se han encontrado en el catálogo («unmatched»), y de cuántos se han leído los logros («achievements»: llegan con el historial, ver save_achievements).

accounts/xbox.py, línea 202

App «games»

Catálogo de juegos, importación desde IGDB y precios de PC (IsThereAnyDeal), Xbox y Nintendo.

games/admin.py

Clase GamePriceInline

```
def has_add_permission(self, request, obj=None)
```

Impide añadir precios a mano: solo los escribe update_prices.

games/admin.py, línea 13

games/history.py

Historial de precios de PC de un juego (de IsThereAnyDeal), veredicto «¿compro ahora o espero?» y gráfica.

El historial se guarda en Game.price_history por tienda, para poder enseñarlo de todas o solo de algunas (las del usuario, o una concreta):

{"since": fecha ISO, "shops": {"61": {"name": "Steam", "points": [[fecha ISO, precio o None, % de rebaja], ...]}}}

Cada tienda tiene un punto por cada cambio de precio, de más viejo a más nuevo; precio None es que dejó de venderlo. Se pide a ITAD al abrir la ficha y se reutiliza durante MAX_AGE.

```
def needs_update(game, now=None)
```

Indica si hay que volver a pedir el historial: nunca se pidió, tiene más de MAX_AGE o es del formato antiguo (solo el más barato, sin separar por tienda).

games/history.py, línea 37

```
def _amount(value)
```

Importe de un precio de ITAD como Decimal con dos decimales.

games/history.py, línea 48

```
def _decimal(value)
```

Precio guardado en el JSON (float) como Decimal con dos decimales.

games/history.py, línea 53

```
def shop_series(events, seed, start)
```

Historial de cada tienda a partir de los cambios de ITAD: {"since", "shops"} (ver el docstring del módulo).

seed es {id de tienda: (precio, rebaja, nombre)} con lo que cuesta hoy en cada tienda; se usa como precio de partida de las tiendas sin cambios en el periodo. Las que sí tienen cambios empiezan con el precio normal de su primer cambio (antes de una rebaja se vendía a precio normal); si su primer cambio es que dejó de venderlo, no se sabe a cuánto estaba y empiezan ahí.

games/history.py, línea 58

```
def update_history(game, client=None, now=None)
```

Pide a ITAD el historial de los dos últimos años del juego y lo guarda. Lanza ITADError si falla.

games/history.py, línea 83

```
def lowest_series(history, shops=None)
```

Precio más barato entre esas tiendas (ids; None = todas) a lo largo del tiempo.

Devuelve [(fecha, precio, rebaja, tienda)] de más viejo a más nuevo, con un punto solo cuando cambia el más barato; si varias tiendas cambian a la vez, queda el resultado final.

games/history.py, línea 101

```
def _year(points, now)
```

Puntos (fecha, precio, rebaja, tienda) del último año, empezando con el que estaba vigente hace un año (con esa fecha) si se sabe.

games/history.py, línea 131

```
def verdict(points, current, low, now, all_shops=True)
```

Veredicto para el precio de hoy: {"code", "title", "text", "year_low"}.

«minimo» si está en su mínimo (low: el histórico con todas las tiendas, o el de los dos años de historial con unas tiendas concretas); «bueno» si en el último año no ha estado más barato o está cerca de lo más barato del año; «espera» si ha bajado de ese precio varias veces.

games/history.py, línea 140

```
def last_big_sale(points)
```

Última vez que estuvo rebajado al menos BIG_SALE_CUT %: {"date", "cut", "price", "shop"}, o None.

games/history.py, línea 177

```
def chart(points, now, current=None, low_line=None, width=320, height=90, pad=6)
```

Datos de la gráfica escalonada del último año para dibujarla en SVG, o None si no hay historial.

Termina en current (el GamePrice más barato de hoy, el que enseña la ficha) si se da: el historial de ITAD puede contar alguna tienda que no sale entre los precios de hoy. low_line es el precio de la línea del mínimo.

Devuelve el tamaño, los puntos de la línea («line») y del área bajo ella («area»), el precio más alto y más bajo de la gráfica, la altura de la línea del mínimo («low_y», None si queda fuera) y, en JSON («hover»), cada tramo con su fecha, posición, precio, rebaja y tienda para la etiqueta que sale al pasar por encima. Las coordenadas van ya en texto con punto decimal: en una plantilla en español, floatformat pondría coma y el SVG no sería válido.

games/history.py, línea 186

```
def shop_options(game, user_shops)
```

Opciones del selector de tienda de la gráfica: [(valor de ?tienda=, etiqueta)].

«Tus tiendas» (si el usuario las tiene elegidas), «Todas las tiendas» y cada tienda con historial, las más conocidas primero.

games/history.py, línea 243

```
def selected_shops(value, user_shops)
```

Tiendas que enseñar según ?tienda= y las del usuario: (ids o None para todas, valor elegido).

Sin valor: las del usuario si las tiene, si no todas. «todas»: todas. Un id: solo esa tienda.

games/history.py, línea 262

```
def summary(game, now=None, shops=None)
```

Todo lo que enseña la ficha sobre el historial de esas tiendas (ids; None = todas).

{"current": precio más barato de hoy en esas tiendas de PC, "verdict", "chart", "last_big_sale", "all_shops"}, o None si no hay historial, el juego es gratis o ninguna de esas tiendas lo vende.

games/history.py, línea 275

games/igdb.py

Cliente de IGDB y la función que vuelca sus juegos en el catálogo local.

Clase IGDBError

Fallo al hablar con IGDB o con Twitch (credenciales, red o respuesta no válida).

```
def is_configured()
```

Indica si están puestas las credenciales de Twitch para usar IGDB.

games/igdb.py, línea 60

Clase IGDBClient

```
def __init__(self, client_id=None, client_secret=None)
```

Prepara el cliente con las credenciales dadas o las de settings; falla si no hay ninguna.

games/igdb.py, línea 66

```
def _token(self)
```

Token de acceso de Twitch: lo saca de la caché o pide uno nuevo y lo guarda hasta que caduque.

games/igdb.py, línea 73

```
def query(self, endpoint, body)
```

Lanza una consulta en Apicalypse (el lenguaje de IGDB) contra endpoint y devuelve el JSON.

Si IGDB responde 401 borra el token de la caché para que la próxima llamada pida uno nuevo.

games/igdb.py, línea 96

```
def top_games(self, limit=MAX_PAGE_SIZE, offset=0)
```

Una página de los juegos más valorados por número de votos, sin ediciones sueltas, con sus ediciones.

games/igdb.py, línea 119

```
def search(self, term, limit=20)
```

Busca juegos por texto en IGDB (hasta limit resultados, sin ediciones sueltas), con sus ediciones.

games/igdb.py, línea 128

```
def games_by_steam_ids(self, app_ids)
```

Juegos de IGDB (con sus ediciones) de una lista de IDs de Steam y un dict {ID de Steam: id de IGDB}.

Los IDs que IGDB no conoce (herramientas, bandas sonoras, servidores...) no aparecen en el dict.

games/igdb.py, línea 137

```
def games_by_ids(self, igdb_ids)
```

Juegos de IGDB (con sus ediciones) a partir de sus ids, en lotes de MAX_PAGE_SIZE.

games/igdb.py, línea 156

```
def games_by_names(self, names)
```

Juegos de IGDB (con sus ediciones) cuyo nombre es exactamente uno de names, sin ediciones sueltas.

Va en lotes para no hacer una petición por juego. Si dos juegos se llaman igual vienen los dos; quien llama decide con cuál quedarse (p. ej. el más popular).

games/igdb.py, línea 165

```
def spanish_data(self, igdb_ids)
```

Solo los campos de castellano (idiomas, nombres y versiones regionales) de esos juegos de IGDB, en lotes.

Sirve para completar los juegos que se importaron antes de que se pidieran esos campos.

games/igdb.py, línea 184

```
def artworks(self, igdb_id, limit=8)
```

IDs de imagen del arte promocional de un juego y, si no llegan a limit, de sus capturas.

Son imágenes apaisadas que sirven de cabecera o de fondo del perfil (la portada es vertical). Del arte se quitan los logos, iconos, portadas e infografías (NOT_ART_TYPES) y va primero el principal.

games/igdb.py, línea 198

```
def game_media(self, igdb_id)
```

Artes, capturas y vídeos de un juego en una sola consulta: {"artworks", "screenshots", "videos"}.

Son las listas tal como las da IGDB (los artes con su tipo y tamaño, los vídeos con su ID de YouTube); vacío si IGDB no conoce el juego.

games/igdb.py, línea 215

```
def _with_editions(self, games)
```

Añade a cada juego sus ediciones (GOTY, Definitive...) en la clave «editions», para tener sus IDs de tienda.

games/igdb.py, línea 224

```
def sort_arts(artworks)
```

Artes de IGDB que son ilustraciones (sin los tipos de NOT_ART_TYPES), con el arte principal primero.

Los que no traen tipo se quedan: IGDB no los ha clasificado, pero suelen ser ilustraciones.

games/igdb.py, línea 246

```
def _upsert_ref(model, data, **extra)
```

Busca o crea un Genre o Platform de IGDB, aprovechando uno que no venga de IGDB con el mismo slug.

games/igdb.py, línea 255

```
def _normalize(name)
```

Nombre para comparar: sin ™ ® ©, en minúsculas y con los espacios simplificados.

games/igdb.py, línea 271

```
def _store_ids(data)
```

IDs de tienda de un juego de IGDB: {"steam": [...], "gog": [...]}.

IGDB puede tener varias entradas por tienda: relanzamientos con el mismo nombre que el juego (se quedan todos y sus ofertas se juntan) y otras raras como «Mac Portal» (solo se usa la primera, y únicamente si ninguna se llama como el juego).

games/igdb.py, línea 276

```
def _store_alternatives(data, own_ids)
```

IDs de tienda de las ediciones del juego y después de las fichas de RELATED_STORES (sus ports, sus ediciones ampliadas y sus remasters), sin repetir los suyos.

Sirven para encontrar precios cuando el juego original ya no se vende o cuando IGDB guarda sus IDs de tienda en otra ficha, como la versión de PC de Red Dead Redemption o la edición ampliada de The Witcher 2.

games/igdb.py, línea 297

```
def _xbox_product_ids(data)
```

IDs de producto de Microsoft Store del juego y después de sus ediciones y remasters, sin repetidos.

games/igdb.py, línea 318

```
@transaction.atomic
def upsert_games(payload)
```

Crea o actualiza los juegos locales a partir de los de IGDB y devuelve la lista de Game.

Guarda datos, nota, portada, IDs de tienda, géneros, plataformas y lo de castellano (idiomas, nombre en España, otros nombres y portada europea, ver games.spanish). Aprovecha juegos de demo o creados a mano con el mismo nombre o slug en vez de duplicarlos. Si cambian los IDs de tienda, borra los precios de PC, que se buscaron con los IDs viejos. Todo va en una transacción.

games/igdb.py, línea 331

games/management/commands/import_igdb.py

Clase Command

```
def add_arguments(self, parser)
```

Opciones del comando: --limit (cuántos populares), --search (importar una búsqueda) y --sin-ids-de-tienda (repasar los del catálogo sin IDs de Steam ni de GOG).

games/management/commands/import_igdb.py, línea 15

```
def handle(self, *args, **options)
```

Punto de entrada: importa una búsqueda, los más populares o repasa los que no tienen IDs de tienda, y dice cuántos juegos guardó.

games/management/commands/import_igdb.py, línea 25

```
def _import_top(self, client, limit)
```

Importa los limit juegos más populares en páginas de hasta 500. Devuelve cuántos importó.

games/management/commands/import_igdb.py, línea 44

```
def _refresh_without_store_ids(self, client)
```

Vuelve a pedir a IGDB los juegos del catálogo que no tienen con qué buscar sus precios de PC (sin IDs de Steam ni de GOG, ni suyos ni de sus ediciones), por si los tienen en un port o una edición ampliada (igdb.RELATED_STORES). Devuelve (cuántos tienen ahora con qué buscarlos, cuántos se han pedido); sus precios llegan con update_prices.

games/management/commands/import_igdb.py, línea 57

games/management/commands/localize_games.py

Clase Command

```
def add_arguments(self, parser)
```

Opciones: --limit, --dias, --igdb, --sin-tiendas y --pausa.

games/management/commands/localize_games.py, línea 21

```
def handle(self, *args, **options)
```

Primero IGDB (rápido, en lotes de 500) y después las tiendas (Steam de uno en uno, con pausas).

games/management/commands/localize_games.py, línea 35

```
def _from_igdb(self, everything)
```

Pide a IGDB los idiomas, los nombres y la portada europea de los juegos que no los tienen (o de todos).

Los que IGDB no devuelve se quedan con los datos de IGDB vacíos, para no volver a pedirlos cada noche.

games/management/commands/localize_games.py, línea 41

```
def _from_stores(self, limit, days, pause)
```

Pide la sinopsis en español y los idiomas de Steam (y la descripción de Microsoft) de los más populares.

games/management/commands/localize_games.py, línea 65

games/management/commands/seed_demo.py

Clase Command

```
def handle(self, *args, **options)
```

Crea los 12 juegos de GAMES con sus géneros y plataformas; se salta los que ya existen.

games/management/commands/seed_demo.py, línea 37

games/management/commands/update_prices.py

Clase Command

```
def add_arguments(self, parser)
```

Opciones del comando: --limit (solo los N más populares) y --game (un solo juego por slug).

games/management/commands/update_prices.py, línea 15

```
def handle(self, *args, **options)
```

Actualiza los precios de PC (si hay ITAD_API_KEY), los de Xbox y los de Nintendo de los juegos elegidos, y después crea los avisos de la lista de deseos.

games/management/commands/update_prices.py, línea 20

games/media.py

Imágenes y vídeos de cada juego para la cabecera y el carrusel de la ficha.

Los artes, capturas, logos y vídeos se piden a IGDB en una sola consulta la primera vez que alguien abre la ficha (y otra vez pasado MAX_AGE) y se guardan en Game.media:

{"arts": [{"id", "type", "w", "h"}], "logos": [{"id", "type", "crop"}], "shots": [{"id", "w", "h"}], "videos": [{"id", "name"}], "steam": {"app_id", "hero", "logo"}}

«arts» son solo ilustraciones (sin logos, iconos, portadas ni infografías) con el arte principal primero; «logos», el blanco y el de color, que pueden ir encima de la cabecera; «videos», tráileres de YouTube con el nombre que les da IGDB (en inglés). Si el juego está en Steam, «steam» dice si su biblioteca tiene imagen de cabecera (True, False o None si no se pudo mirar) y el recorte de su logo. Los recortes («crop», ver _logo_crop) quitan el margen transparente de los logos para que encajen en la cabecera.

```
def steam_app_id(game)
```

ID de Steam de la imagen y el logo de la cabecera: el manual si lo hay; None si no está en Steam.

games/media.py, línea 39

```
def needs_update(game, now=None)
```

Indica si hay que pedir a IGDB las imágenes y vídeos del juego.

Hay que pedirlos si el juego viene de IGDB y nunca se pidieron, tienen más de MAX_AGE o se pidieron con otro ID de Steam (o sin él).

games/media.py, línea 44

```
def update_media(game, client=None, now=None)
```

Pide a IGDB los artes, capturas, logos y vídeos del juego y los guarda en game.media (formato arriba).

Si el juego está en Steam, mira también si su biblioteca tiene imagen de cabecera y logo. Del logo que se vaya a usar (el de Steam o, si no hay, el de IGDB) guarda el recorte sin el margen transparente.

games/media.py, línea 58

```
def _size(image)
```

ID y tamaño de una imagen de IGDB tal como se guardan en Game.media.

games/media.py, línea 94

```
def _video_order(video)
```

Orden de los vídeos: el tráiler de lanzamiento (o el de la fecha de salida), los demás tráileres, los de gameplay y el resto; dentro de cada grupo, el que antes subió IGDB.

games/media.py, línea 99

```
def _exists(url)
```

Indica si la imagen de url existe (True o False), o None si no se pudo comprobar.

games/media.py, línea 112

```
def _logo_crop(url)
```

Recorte del logo de url sin su margen transparente, o None si no existe, no se puede leer o no tiene transparencia (lleva fondo y taparía la cabecera con un recuadro).

Son fracciones de la imagen entera: izquierda «l», arriba «t», ancho «w» y alto «h» de lo que no es transparente, y la proporción ancho/alto de ese trozo («ratio»). Los logos de la biblioteca de Steam vienen en un lienzo de 16:9 con mucho margen arriba y abajo.

games/media.py, línea 121

```
def _igdb_logo_url(logo)
```

URL del logo de IGDB en PNG (con su transparencia), a un tamaño que se ve nítido en la cabecera.

games/media.py, línea 154

```
def _is_wide(image)
```

Indica si la imagen es apaisada (o no se sabe su tamaño); las verticales se enseñan enteras, sin recortar.

games/media.py, línea 159

```
def hero(game)
```

Imagen y logo de la cabecera de la ficha: {"image", "fallback", "credit", "fallback_credit", "logo", "logo_style"}.

La imagen es la de la biblioteca de Steam si el juego está en Steam y la tiene (o aún no se sabe: la página pasa a «fallback» si no carga); si no, el arte principal de IGDB sin logo, el arte apaisado más ancho o la primera captura. Sin imagen, la ficha se queda con la carátula desenfocada. El logo, el de Steam o el blanco o de color de IGDB, solo va cuando se sabe su recorte (logo_style, variables CSS); si no, el nombre en texto. «credit» dice de dónde salen; «fallback_credit», si se pasa a «fallback».

games/media.py, línea 164

```
def _credit(image_source, logo_source)
```

Texto que dice de dónde salen la imagen y el logo de la cabecera («Imagen y logo: biblioteca de Steam»).

games/media.py, línea 200

```
def _hero_image(media)
```

URL de la mejor imagen de IGDB para la cabecera, o vacía si no hay ninguna.

Es el arte principal sin logo; si no hay, el arte apaisado más ancho que no lleve el logo (repetiría el título); y si tampoco, la primera captura.

games/media.py, línea 208

```
def carousel(game)
```

Miniaturas y filtros del carrusel de la ficha: {"items": [...], "filters": [(tipo, nombre, cuántas)]}.

Orden: el primer vídeo, el arte principal, las capturas, el resto de artes y el resto de vídeos. Cada miniatura es un dict con kind («video», «captura» o «arte»), tag (su etiqueta), heading y detail (el título del visor), label (para lectores de pantalla), thumb y full (URLs de la miniatura y de la imagen grande), tall (vertical: se enseña entera), youtube (ID del vídeo) y art (ID de IGDB que se puede poner de cabecera del perfil; vacío en los vídeos y en las imágenes verticales). Solo hay filtros si hay más de un tipo.

games/media.py, línea 224

```
def official_images(game)
```

Capturas y artes de la ficha que se pueden poner en una reseña sin subir nada, primero las capturas.

Son miniaturas como las del carrusel (ver carousel), con «id», el ID de la imagen en IGDB. Vacía si aún no se han pedido las imágenes del juego.

games/media.py, línea 244

```
def _image_item(kind, image, n, total)
```

Miniatura del carrusel de un arte o una captura (ver carousel), con su ID de IGDB en «id».

games/media.py, línea 257

```
def _video_item(video, n, total)
```

Miniatura del carrusel de un vídeo de YouTube (ver carousel); el vídeo se carga al pulsarlo en el visor.

games/media.py, línea 277

```
def banner_arts(game)
```

IDs de IGDB de las imágenes de la ficha que valen de cabecera del perfil: artes y capturas apaisados.

games/media.py, línea 294

games/models.py

```
def spanish_genre_name(name, slug='')
```

Nombre en español de un género de IGDB; si no está en GENRE_NAMES_ES, el original.

games/models.py, línea 36

```
def igdb_image_url(image_id, size='t_cover_big', ext='jpg')
```

URL de una imagen del CDN de IGDB (portada, arte o captura) en ese tamaño; vacía si no hay imagen.

Con ext «png» conserva la transparencia, que hace falta en los logos.

games/models.py, línea 44

Clase Spanish

Cuánto español tiene un juego en voces o en textos. Vacío = no se sabe (ninguna fuente da sus idiomas).

```
def language_filter(key, prefix='')
```

Condición (Q) del filtro de idioma ?idioma=key, o None si key no es uno de LANGUAGE_FILTERS.

prefix va delante de los campos para filtrar desde otro modelo (p. ej. «game__» en la lista de deseos). Con voces en castellano también se cuentan los textos: si está doblado, está traducido.

games/models.py, línea 82

```
def search_filter(query)
```

Condición (Q) de la búsqueda de juegos por nombre: el de IGDB, el de España y sus otros nombres (siglas como «tw3»).

games/models.py, línea 97

Clase Genre

```
def __str__(self)
```

Nombre del género.

games/models.py, línea 110

Clase Platform

```
def __str__(self)
```

Nombre completo de la plataforma.

games/models.py, línea 131

```
@property
def short_name(self)
```

Nombre corto para las etiquetas (p. ej. «PS5»); el completo si no tiene abreviatura.

games/models.py, línea 136

Clase Game

```
def __str__(self)
```

Nombre del juego.

games/models.py, línea 233

```
def get_absolute_url(self)
```

URL de la ficha del juego (/juegos/<slug>/).

games/models.py, línea 237

```
def review_platforms(self)
```

Plataformas que se pueden elegir en una reseña del juego: las suyas y, si sale en PC, la Steam Deck.

games/models.py, línea 241

```
@property
def cover_url(self)
```

URL de la portada en el CDN de IGDB (la europea si IGDB la tiene), o cadena vacía si no hay portada.

games/models.py, línea 249

```
def merged_languages(self)
```

Idiomas de todas las fuentes juntos: {código: {"audio", "text"}}. Si una fuente dice que tiene voces, las tiene.

games/models.py, línea 253

```
def refresh_spanish(self)
```

Recalcula spanish_audio y spanish_text con los idiomas de languages (sin guardar el juego).

Castellano si está el español de España, latino si solo está el de México y «sin español» si hay idiomas pero ninguno es español; vacío si ninguna fuente da idiomas. Las voces cuentan también como textos.

games/models.py, línea 261

```
@property
def spain_name(self)
```

Nombre con el que se vende en España si es distinto del original («Los Sims 4»); vacío si no.

games/models.py, línea 282

```
@property
def language_sources(self)
```

Fuentes que han dado idiomas del juego, para citarlas en la ficha: p. ej. ["IGDB", "Steam"].

games/models.py, línea 287

```
@property
def language_rows(self)
```

Idiomas para la tabla de la ficha: [(nombre en español, voces, textos)], con los españoles y el inglés primero.

games/models.py, línea 293

```
@property
def release_year(self)
```

Año de lanzamiento, o None si no se conoce la fecha.

games/models.py, línea 308

```
@property
def price_lookups(self)
```

Grupos ([(tienda, ID en la tienda), ...], nombre de edición) con los que buscar precios, por preferencia.

Primero los IDs del propio juego y después los de sus ediciones y remasters. Los IDs de un mismo grupo son el mismo producto, así que sus ofertas se juntan. Si hay un ID de Steam manual, solo se usa ese.

games/models.py, línea 313

```
@property
def xbox_lookup_ids(self)
```

IDs de producto de Xbox con los que buscar el precio: el manual si lo hay, si no los de IGDB.

games/models.py, línea 331

```
@property
def on_current_xbox(self)
```

Indica si el juego está en Xbox One o Xbox Series X|S.

games/models.py, línea 336

```
@property
def nintendo_lookup_nsuid(self)
```

NSUID con el que pedir el precio de la eShop: el manual si lo hay, si no el encontrado por el nombre.

games/models.py, línea 341

```
@property
def best_price(self)
```

Oferta más barata de todas las tiendas (PC, Xbox y Nintendo), o None si no hay precios.

games/models.py, línea 346

```
def cheapest(self, shops=None)
```

Oferta más barata de esas tiendas (ids; None = todas), o None si ninguna lo vende.

games/models.py, línea 350

```
@property
def is_free(self)
```

Indica si alguna tienda lo da gratis (precio y precio normal a 0), es decir, si es free-to-play.

games/models.py, línea 359

```
def price_groups(self, shops=None)
```

Reparte los precios de esas tiendas (ids; None = todas) para la ficha en (destacados, listados, plegados, mejor).

destacados: las tiendas más conocidas (Steam, Epic, Microsoft, GOG, Xbox, Nintendo) en un orden fijo. listados: las demás más baratas, solo si el juego no está en ninguna destacada. plegados: el resto por precio, dentro de «Ver N tiendas más». mejor: la oferta más barata cuando ha quedado plegada, para poder señalarla igualmente.

games/models.py, línea 363

```
def store_searches(self)
```

(tienda, plantilla del logo, URL de búsqueda) de las consolas en las que está el juego pero sin precio.

Son PlayStation Store y, si no se encontró su precio, Nintendo eShop y Xbox Store.

games/models.py, línea 380

```
def community_rating(self)
```

Media de las valoraciones de los usuarios de NextPlay: {"average": de 0,5 a 5 estrellas o None, "count": n}, y la misma solo con quienes tienen horas importadas de Steam, PlayStation o Xbox («verified_average» y «verified_count»), que no pueden valorarlo sin haberlo jugado.

Las notas se guardan en medias estrellas (de 1 a 10), así que las medias se dividen entre dos.

games/models.py, línea 393

```
@property
def critic_score(self)
```

Nota de la crítica redondeada (0-100), o None si no tiene.

games/models.py, línea 416

```
@property
def critic_tier(self)
```

Franja de la nota para colorearla: «high» (75 o más), «mid» (50-74), «low» o vacío sin nota.

games/models.py, línea 421

```
@property
def historical_low_symbol(self)
```

Símbolo de la moneda del mínimo histórico (€, $, £...).

games/models.py, línea 429

```
def currency_symbol(code)
```

Símbolo de una moneda a partir de su código ISO («EUR» → «€»); el propio código si no lo conoce.

games/models.py, línea 478

```
def known_shops()
```

Tiendas con algún precio en el catálogo, [(id, nombre)]: las destacadas primero y después por nombre.

games/models.py, línea 483

```
def format_price(amount, symbol='€')
```

Precio escrito como en España para los textos que se generan fuera de las plantillas: «2,99 €».

games/models.py, línea 495

Clase GamePrice

Precio actual de un juego en una tienda.

```
def __str__(self)
```

Texto del precio en el admin: «juego · tienda: precio moneda».

games/models.py, línea 525

```
@property
def symbol(self)
```

Símbolo de la moneda del precio.

games/models.py, línea 530

```
@property
def is_free(self)
```

Indica si el juego es gratis en esta tienda (no solo rebajado a 0 por una oferta).

games/models.py, línea 535

```
@property
def is_featured(self)
```

Indica si la tienda es una de las destacadas (Steam, Epic, Microsoft, GOG, Xbox, Nintendo).

games/models.py, línea 540

```
@property
def logo_template(self)
```

Ruta de la plantilla SVG con el logo de la tienda; solo para tiendas destacadas.

games/models.py, línea 545

games/nintendo.py

Precios de la eShop de Nintendo (Switch y Switch 2) en la tienda europea (sin documentar, sin clave).

IGDB no tiene el NSUID de los juegos (su ID en la eShop), así que se busca por el nombre en el buscador de Nintendo Europa, el de su web, y se guarda en el juego. El precio sale de la API de precios de la eShop, la misma que usa la web de Nintendo. Los NSUID de la tienda europea valen en todos sus países.

Clase NintendoError

Fallo al consultar la eShop de Nintendo (red o respuesta no válida).

```
def _country()
```

País de la tienda, el mismo que ITAD_COUNTRY (p. ej. «ES»); tiene que ser de la tienda europea.

games/nintendo.py, línea 36

```
def _amount(value)
```

Importe de la API de precios («14.99») como Decimal con dos decimales.

games/nintendo.py, línea 41

```
def fetch_prices(nsuids)
```

Devuelve {NSUID: (precio, precio normal, moneda)} de los juegos que se pueden comprar.

Pide los precios de 50 en 50; los que no están a la venta (sin salir o retirados) no aparecen.

games/nintendo.py, línea 46

```
def _offer(item)
```

(precio, precio normal, moneda) de un juego de la respuesta de precios, o None si no se vende.

El precio es el rebajado si tiene una oferta en marcha.

games/nintendo.py, línea 71

```
def _normalize(title)
```

Título para comparar: en minúsculas, sin tildes ni signos (™, dos puntos, guiones...) y con los espacios simplificados, porque la tienda los pone a su manera: «South Park™ : La vara de la verdad™», «ABZÛ».

games/nintendo.py, línea 85

```
def search_game(names, systems)
```

NSUID del juego de esas consolas («HAC», «BEE») que se llama exactamente como uno de names, o None.

Busca con cada nombre hasta encontrarlo y compara con el título de la tienda española, el original en inglés y sus traducciones. Si dos juegos de una consola se llaman igual no elige ninguno; si es el mismo título en Switch y en Switch 2 se queda con el de Switch, que también funciona en Switch 2.

games/nintendo.py, línea 94

```
def store_url(nsuid)
```

Enlace al juego en la eShop, que lleva a su página en nintendo.com.

games/nintendo.py, línea 135

```
def _systems(game)
```

Códigos del buscador de Nintendo de las consolas del juego: «HAC» (Switch) y «BEE» (Switch 2).

games/nintendo.py, línea 140

```
@transaction.atomic
def _store(game, nsuid, offer)
```

Sustituye el precio de Nintendo del juego por offer y calcula el % de descuento; sin offer lo deja sin precio.

games/nintendo.py, línea 146

```
def update_nintendo_prices(games, sleep=time.sleep)
```

Guarda el precio de la eShop de cada juego y devuelve cuántos acabaron con precio.

Los juegos de Switch o Switch 2 sin NSUID (ni manual ni encontrado antes) lo buscan primero por su nombre y su nombre en España, salvo que otro juego del catálogo se llame igual; el que se encuentra se guarda para no volver a buscarlo.

games/nintendo.py, línea 164

games/prices.py

Cliente de IsThereAnyDeal y la sincronización que guarda los precios actuales de PC de cada juego.

También está aquí la búsqueda al momento, en todas las tiendas, de los precios de los juegos que nunca se han buscado (fetch_missing), para no esperar a la tarea de cada noche.

Clase ITADError

Fallo al hablar con IsThereAnyDeal (clave, red o respuesta no válida).

```
def is_configured()
```

Indica si está puesta la clave de IsThereAnyDeal.

games/prices.py, línea 33

Clase ITADClient

```
def __init__(self, api_key=None, country=None, retries=MAX_RETRIES)
```

Prepara el cliente con la clave y el país dados o los de settings; falla si no hay clave.

retries son las veces que se repite una petición cuando la API pide calma (429). Con 0 no se espera nada, que es lo que conviene cuando alguien está esperando la página.

games/prices.py, línea 39

```
def _post(self, path, body, params=None)
```

POST a la API con la clave y devuelve el JSON.

games/prices.py, línea 51

```
def _get(self, path, params)
```

GET a la API con la clave y devuelve el JSON.

games/prices.py, línea 55

```
def _send(self, method, path, **kwargs)
```

Llama a la API con method (requests.get o requests.post) y la clave, y devuelve el JSON.

Si la API responde 429 (demasiadas peticiones) espera lo que diga Retry-After (máximo 5 minutos) y reintenta hasta self.retries veces (3, si no se dice otra cosa).

games/prices.py, línea 59

```
def lookup(self, store, store_ids)
```

Traduce IDs de una tienda (de Steam o de GOG) a los UUID de juego de ITAD (None si no lo conoce).

games/prices.py, línea 79

```
def prices(self, itad_ids)
```

Ofertas actuales y mínimo histórico de cada juego de ITAD en el país configurado.

games/prices.py, línea 85

```
def history(self, itad_id, since)
```

Cambios de precio de un juego de ITAD desde la fecha since, en todas las tiendas del país.

Cada cambio es {"timestamp", "shop": {"id", "name"}, "deal": {"price", "regular", "cut"}}; «deal» es None cuando la tienda deja de venderlo. Vienen del más nuevo al más viejo.

games/prices.py, línea 89

```
def _amount(value)
```

Importe de un precio de ITAD como Decimal con dos decimales.

games/prices.py, línea 99

```
def _batches(items)
```

Parte la lista en trozos de BATCH_SIZE, lo máximo que acepta la API por petición.

games/prices.py, línea 104

```
def _cheapest_per_shop(deals)
```

Se queda con la oferta más barata de cada tienda.

games/prices.py, línea 110

```
@transaction.atomic
def _store_prices(game, data, edition='', itad_ids=())
```

Sustituye los precios de PC del juego por las ofertas de data y guarda su mínimo histórico.

Con data vacío deja el juego sin precios de PC. edition es el nombre de la edición de la que salen los precios (vacío si son del propio juego) e itad_ids, los juegos de ITAD de los que salen; si cambian, se borra el historial guardado, que era de otros.

games/prices.py, línea 122

```
def _lookup(client, keys)
```

Traduce un conjunto de (tienda, ID) a UUID de ITAD, tienda por tienda y por lotes.

games/prices.py, línea 158

```
def _fetch_prices(client, itad_ids)
```

Precios de todos los UUID de ITAD pedidos por lotes: {uuid: datos de precios}.

games/prices.py, línea 168

```
def _merge(items)
```

Junta los precios de varios juegos de ITAD que son el mismo producto (p. ej. un relanzamiento).

Suma todas sus ofertas y se queda con el mínimo histórico más bajo. Devuelve None si no hay datos.

games/prices.py, línea 176

```
def update_prices(games, client=None)
```

Actualiza los precios de PC de los juegos y devuelve cuántos acabaron con precios.

Va por rondas: en la primera busca con los IDs del propio juego; los juegos que se quedan sin ofertas pasan a la siguiente ronda con su primera edición o remaster, y así sucesivamente.

Todos quedan con la fecha de la búsqueda (prices_updated_at), también los que no tienen IDs de Steam ni de GOG con los que buscarlos en ITAD, que se quedan sin precios de PC: así consta que ya se han mirado.

games/prices.py, línea 191

```
def fetch_missing(games, consoles=False)
```

Busca ya, sin esperar a la tarea de cada noche, los precios de los juegos que nunca se han buscado (prices_updated_at vacío) y devuelve cuántos han quedado buscados.

Los de PC se piden a IsThereAnyDeal de todos a la vez y sin esperas si la API pide calma, y con eso quedan como buscados. Después van los de Xbox y Nintendo: con consoles (la ficha de un juego), de todos; si no (una importación), solo de los que se han quedado sin ningún precio, porque esas tiendas se buscan por el nombre juego a juego y con muchos la importación tardaría demasiado; a los demás se los trae update_prices por la noche.

Si una tienda falla se apunta en el log y se sigue. Si falla IsThereAnyDeal, los juegos siguen sin buscar y se volverán a pedir la próxima vez; en una importación tampoco se buscan entonces sus precios de consola. Sin ITAD_API_KEY no hace nada.

games/prices.py, línea 226

```
def lookup_ids(game, client=None)
```

UUID de ITAD de los IDs de tienda con los que se encontraron los precios actuales del juego.

Sirve para los juegos cuyos precios se guardaron antes de que se apuntaran sus itad_ids.

games/prices.py, línea 262

games/spanish.py

Datos en castellano de los juegos: idiomas (voces y textos), nombre en España, portada europea y sinopsis en español.

Los idiomas, los nombres y la portada europea vienen de IGDB en la misma consulta de la importación (SPANISH_FIELDS, que se añade a GAME_FIELDS). La sinopsis en español y los idiomas de la tienda vienen de Steam (appdetails en español) y, si el juego no está en Steam o no tiene sinopsis, de la descripción de la tienda de Microsoft. Steam admite pocas peticiones (unas 200 cada 5 minutos), así que esa parte la hace el comando localize_games con pausas, nunca al abrir una ficha.

Clase StoreError

Fallo al pedir a Steam los datos de un juego (red, límite de peticiones o respuesta no válida).

```
def igdb_languages(data)
```

Idiomas de un juego de IGDB: {código: ["audio", "text"]}, con «text» si tiene interfaz o subtítulos.

games/spanish.py, línea 56

```
def spanish_names(data)
```

(nombre en España, otros nombres para el buscador) de un juego de IGDB.

El nombre en España es el primer nombre alternativo que IGDB marca como «Spanish title». Para el buscador se juntan, en minúsculas y sin repetir, los demás nombres alternativos (siglas, títulos de otros países...) y los de las versiones regionales, salvo los de los ejecutables.

games/spanish.py, línea 70

```
def europe_cover(data)
```

ID de imagen de la portada europea de un juego de IGDB (versión regional de Europa), o vacío si no tiene.

games/spanish.py, línea 91

```
def apply_igdb(game, data)
```

Copia en game (sin guardarlo) los idiomas, el nombre en España, los otros nombres y la portada europea de IGDB.

games/spanish.py, línea 99

```
def steam_languages(text)
```

Idiomas de la cadena supported_languages de Steam (pedida en español): {código: ["text"] o ["audio", "text"]}.

Steam marca con un asterisco los idiomas con voces; lo que va después del primer
 es la nota que explica el asterisco. Los idiomas que no están en STEAM_LANGUAGES se ignoran.

games/spanish.py, línea 107

```
def clean_text(value)
```

Texto plano a partir del HTML de una tienda: sin etiquetas, con las entidades traducidas y sin espacios de más.

games/spanish.py, línea 122

```
def steam_details(app_id, sleep=time.sleep)
```

Idiomas y sinopsis en español de una app de Steam: (idiomas, sinopsis); ({}, "") si Steam no la conoce.

Los cortes de conexión y los errores de Steam (también el 429 por pedir demasiado) se reintentan tras esperar RETRY_WAITS; si sigue fallando, lanza StoreError.

games/spanish.py, línea 129

```
def localize_games(games, pause=STEAM_PAUSE, sleep=time.sleep)
```

Pide a las tiendas la sinopsis en español y los idiomas de Steam de esos juegos y los guarda.

Steam va de uno en uno, con pause segundos entre petición y petición. Los que se quedan sin sinopsis y están en la tienda de Microsoft se piden después, en lotes. Cada juego queda marcado con localized_at para no volver a pedirlo hasta dentro de un tiempo. Si Steam deja de responder (StoreError) se para ahí: lo ya pedido queda guardado, se completa con Microsoft y después se vuelve a lanzar el error.

Devuelve {"steam": con sinopsis de Steam, "xbox": con sinopsis de Microsoft, "none": sin sinopsis}.

games/spanish.py, línea 157

games/views.py

```
def trending_games(limit=5)
```

Los juegos que más gente ha añadido a su biblioteca en los últimos TRENDING_DAYS días.

Cada juego trae players (cuántos lo tienen en total) y recent (cuántos lo añadieron en ese tiempo). Si no llegan a limit, se completa con los más populares de IGDB.

games/views.py, línea 28

```
def home(request)
```

Portada: tu resumen (con tus seguidores y seguidos) y tu menú a la izquierda; en el centro la caja para publicar (social.views.composer_context, con los juegos que estás jugando para elegir de cuál hablas) y el feed, y los juegos en tendencia a la derecha.

El feed tiene dos pestañas (?ver=, ver social.views.feed_context): «siguiendo», lo tuyo y lo de la gente que sigues, con sugerencias de gente a la que seguir si no sigues a nadie, y «descubrir», lo de toda la comunidad; ?antes= es el cursor de «Cargar más» para quien no tiene JavaScript. Sin sesión, en el centro van los juegos populares y lo último de la comunidad.

games/views.py, línea 45

```
def catalog(request)
```

Catálogo de juegos.

Filtra por nombre (?q=, que también busca el nombre en España y los otros nombres del juego, como sus siglas), género (?genero=), plataforma (?plataforma=) e idioma (?idioma=, ver LANGUAGE_FILTERS) y pagina de 24 en 24. Si hay sesión, marca en cada tarjeta el estado del juego en la biblioteca del usuario (library_status, su nombre, y library_status_key, su valor, que le da el color). Ofrece importar de IGDB cuando hay búsqueda, sesión y credenciales de IGDB.

games/views.py, línea 83

```
def detail(request, slug)
```

Ficha de un juego: cabecera con su arte, datos, nota de la crítica, valoración de la comunidad, widget de biblioteca, botones de favoritos del perfil y de «Añadir a una lista», las listas en las que está (lists.views.game_page_context), carrusel de imágenes y vídeos, precios y reseñas.

La cabecera y el carrusel salen de games.media: si sus imágenes de IGDB están al día se pintan ya; si no, la ficha carga el carrusel después con HTMX (game_media). «Dónde comprarlo» (_prices_context) se pinta ya si los precios del juego se han buscado alguna vez; si no, la ficha lo carga después con HTMX (game_prices), que antes los busca. Las reseñas de perfiles privados solo las ve su autor.

games/views.py, línea 142

```
def game_media(request, slug)
```

Carrusel de imágenes y vídeos de la ficha (fragmento HTML que la ficha carga con HTMX).

Si nunca se pidieron a IGDB o tienen más de media.MAX_AGE, los pide antes; si IGDB falla, enseña los que hubiera. Si con ellos cambia la cabecera (un juego que no está en Steam y ahora tiene arte), manda también la nueva para que HTMX la cambie en la página.

games/views.py, línea 183

```
@require_POST
@login_required
def import_search(request)
```

Importa de IGDB los resultados de una búsqueda que no estaban en el catálogo local.

Después vuelve al catálogo con la misma búsqueda y avisa de cuántos juegos se importaron, o de que IGDB no respondió.

games/views.py, línea 214

```
def _user_shops(request)
```

Tiendas en las que compra el usuario con sesión (conjunto de ids), o vacío si no ha elegido o no hay sesión.

games/views.py, línea 231

```
def _prices_context(request, game)
```

Variables de «Dónde comprarlo» (games/_prices.html).

Los precios llegan ya repartidos por Game.price_groups(); store_searches() da los enlaces de búsqueda a las tiendas de consola sin precio. Si el usuario ha elegido sus tiendas solo se enseñan esas, salvo con ?tiendas=todas. El historial de precios de PC va ya pintado si está al día; si hay que pedirlo a ITAD, se carga después con HTMX (price_history), y su gráfica elige tienda con ?tienda=.

games/views.py, línea 238

```
def game_prices(request, slug)
```

«Dónde comprarlo» de la ficha (fragmento HTML que la ficha carga con HTMX si los precios del juego nunca se han buscado).

Si siguen sin buscarse, antes los pide a todas las tiendas: las de PC a IsThereAnyDeal y también Xbox y Nintendo (prices.fetch_missing con consoles), y la ficha dice que se acaban de buscar (just_searched). Si IsThereAnyDeal falla, enseña los que haya y avisa de que no se han podido buscar (prices_failed); se volverá a intentar la próxima vez que se abra la ficha.

games/views.py, línea 266

```
def _history_context(request, game, user_shops)
```

Variables de la sección del historial: el resumen de las tiendas elegidas con ?tienda= (por defecto, las del usuario), las opciones del selector y la elegida.

games/views.py, línea 297

```
def price_history(request, slug)
```

Historial de precios de PC del juego con el veredicto «¿compro ahora o espero?» (fragmento HTML).

Si el guardado tiene más de un día, lo pide antes a IsThereAnyDeal; si ITAD falla, enseña el que hubiera. La ficha lo carga con HTMX para no esperar a ITAD, y el selector de tienda lo vuelve a pedir con ?tienda=.

games/views.py, línea 309

games/xbox.py

Precios de consola Xbox sacados del catálogo público de la tienda de Microsoft (sin documentar, sin clave).

Clase XboxError

Fallo al consultar la tienda de Xbox (red o respuesta no válida).

```
def _market()
```

País de la tienda, el mismo que ITAD_COUNTRY (p. ej. «ES»).

games/xbox.py, línea 24

```
def _locale()
```

Idioma y país de la tienda, p. ej. «es-ES».

games/xbox.py, línea 29

```
def fetch_products(product_ids)
```

Devuelve {ID de producto: (precio, precio normal, moneda)} de los productos que se pueden comprar en consola Xbox.

Pide los productos de 20 en 20; los que no se venden en consola no aparecen en el resultado.

games/xbox.py, línea 34

```
def fetch_descriptions(product_ids)
```

Descripción en español de los productos de la tienda de Microsoft: {ID de producto: texto}.

Es la descripción corta o, si no tiene, el primer párrafo de la larga. Pide los productos de 20 en 20 en el idioma de la tienda (es-ES); los que no tienen descripción no aparecen.

games/xbox.py, línea 59

```
def _console_offer(product)
```

Oferta de compra más barata de un producto que funciona en Xbox, o None si no hay.

Ignora las de 0,00, que son Game Pass, pruebas o copias incluidas en packs.

games/xbox.py, línea 88

```
def _normalize(title)
```

Título para comparar: sin ™ ® ©, en minúsculas y con los espacios simplificados.

games/xbox.py, línea 114

```
def search_games(name)
```

IDs de producto de los juegos de Xbox que se llaman exactamente como name (títulos en inglés, como en IGDB).

games/xbox.py, línea 119

```
def store_url(product_id)
```

Enlace a la página del producto en xbox.com.

games/xbox.py, línea 146

```
@transaction.atomic
def _store(game, product_id, offer)
```

Sustituye el precio de Xbox del juego por offer y calcula el % de descuento; sin offer lo deja sin precio.

games/xbox.py, línea 152

```
def update_xbox_prices(games, sleep=time.sleep)
```

Guarda el precio de Xbox de cada juego y devuelve cuántos acabaron con precio.

Usa el primer ID de producto (el suyo y después sus ediciones) que se venda en consola. Como IGDB a menudo solo tiene productos que ya no se venden (p. ej. la versión original de un relanzamiento), los juegos de Xbox One/Series sin precio se buscan también por su título exacto, salvo que otro juego del catálogo se llame igual o que la búsqueda dé más de un producto.

games/xbox.py, línea 170

App «library»

Biblioteca de cada usuario: qué juegos tiene, en qué estado, su valoración y su reseña.

library/admin.py

Clase ReviewCommentInline

Comentarios de una reseña, para moderarlos desde ella.

Clase ReviewImageInline

Imágenes de una reseña, con su miniatura, para quitar las que no deban estar.

```
@admin.display(description='imagen')
def preview(self, image)
```

Miniatura de la imagen, que lleva a la grande.

library/admin.py, línea 45

Clase ReportInline

Denuncias de una reseña o un comentario.

Clase ReviewAdmin

```
def get_queryset(self, request)
```

Las reseñas con sus denuncias pendientes contadas.

library/admin.py, línea 70

```
@admin.display(description='denuncias pendientes', ordering='pending')
def pending_reports(self, review)
```

Cuántas denuncias de la reseña quedan por revisar.

library/admin.py, línea 77

Clase PendingFilter

Filtro de las denuncias por si ya se han revisado.

```
def lookups(self, request, model_admin)
```

Las dos opciones del filtro.

library/admin.py, línea 96

```
def queryset(self, request, queryset)
```

Las denuncias sin revisar o las ya revisadas, según la opción elegida.

library/admin.py, línea 100

Clase ReportAdmin

Denuncias de reseñas y comentarios. Desde cada una se va a lo denunciado (en la web o en el admin) para borrarlo si hace falta, y después se marca como revisada.

```
def has_add_permission(self, request)
```

Las denuncias solo las crean los usuarios desde la web.

library/admin.py, línea 122

```
@admin.display(description='qué es')
def kind(self, report)
```

Qué se ha denunciado: «reseña» o «comentario».

library/admin.py, línea 127

```
@admin.display(description='denunciado')
def target_link(self, report)
```

Enlaces a lo denunciado: su página en la web y su ficha en el admin; «borrado» si ya no existe.

library/admin.py, línea 132

```
@admin.action(description='Marcar como revisadas')
def mark_resolved(self, request, queryset)
```

Marca las denuncias elegidas como revisadas ahora.

library/admin.py, línea 145

```
@admin.action(description='Volver a dejar pendientes')
def mark_pending(self, request, queryset)
```

Vuelve a dejar pendientes las denuncias elegidas.

library/admin.py, línea 151

library/apps.py

Clase LibraryConfig

```
def ready(self)
```

Registra las señales de la app al arrancar Django (borran del disco las imágenes de las reseñas que se quitan).

library/apps.py, línea 7

library/autofill.py

Plataforma y fechas de una partida sacadas de las importaciones, para que una reseña nueva se rellene sola.

Cada título importado (accounts.ImportedTitle) guarda sus minutos, la última partida, la primera (solo la da PlayStation) y las plataformas en las que se jugó (PS4 o PS5 en PlayStation, los dispositivos de Xbox y el PC en Steam, que además dice cuántos minutos fueron en una Steam Deck). Con eso el editor elige la plataforma en la que más ha jugado el usuario y propone las fechas, diciendo de dónde sale cada dato para que se pueda cambiar.

```
def _platforms(row)
```

Plataformas (nombres de IGDB) en las que se jugó un título importado: las guardadas o, si no hay, las que se deducen.

library/autofill.py, línea 23

```
def _played_on(row)
```

Dónde se jugó un título importado y cuántos minutos en cada sitio: [(nombre de plataforma, minutos)].

Lo jugado en una Steam Deck se separa de lo del PC. En lo demás no se sabe cómo se reparten las horas entre sus plataformas (una consola y su sucesora, por ejemplo) y cada una cuenta con todos los minutos.

library/autofill.py, línea 34

```
def _duration(minutes)
```

Tiempo jugado corto para las notas: «12 h» o «40 min».

library/autofill.py, línea 48

```
def play_details(user, game, entry)
```

Plataforma y fechas de la partida de user en game según sus importaciones, para el editor de reseñas.

Devuelve {"initial": {campo: valor}, "notes": {campo: de dónde sale}}: la plataforma del juego en la que más minutos lleva (la Steam Deck cuenta como una más en los juegos de PC); «Lo empecé el» con su primera partida (si alguna importación la da) y, si tiene el juego como Completado, «Lo terminé el» con su última partida. Vacíos si el juego no viene de ninguna importación o sus plataformas no son las del juego.

library/autofill.py, línea 53

library/forms.py

```
def stars_label(value)
```

Texto de una nota de 1 a 10 en estrellas, como se lee en España: «3,5 estrellas», «1 estrella».

library/forms.py, línea 13

Clase PointsWidget

MAX_POINTS casillas de texto para los puntos de «Lo mejor» o «Lo peor» (se llaman <campo>_0, <campo>_1...).

```
def __init__(self, attrs=None)
```

Una casilla por punto, con el largo máximo de un punto.

library/forms.py, línea 23

```
def decompress(self, value)
```

La lista guardada repartida en las casillas; las que sobran, vacías.

library/forms.py, línea 27

Clase PointsField

Lista de puntos de «Lo mejor» o «Lo peor»: hasta MAX_POINTS frases de hasta POINT_MAX_LENGTH caracteres.

```
def __init__(self, **kwargs)
```

Un CharField opcional por casilla; la lista entera también es opcional.

library/forms.py, línea 38

```
def compress(self, data_list)
```

Los puntos escritos, en orden, sin espacios de sobra, vacíos ni repetidos.

library/forms.py, línea 43

Clase ReviewForm

Reseña de un juego: nota en medias estrellas, texto con formato, lo mejor y lo peor, spoilers, plataforma, fechas de la partida y si es una rejugada. La plataforma solo puede ser una de las del juego.

```
def __init__(self, *args, game, **kwargs)
```

Limita la plataforma a las del juego (y la Steam Deck si sale en PC, ver Game.review_platforms).

library/forms.py, línea 97

```
def clean_text(self)
```

Quita los espacios del principio y del final: una reseña en blanco no vale.

library/forms.py, línea 103

```
def point_slots(self, name)
```

Casillas de «Lo mejor» (name="pros") o «Lo peor» ("cons") para pintarlas una a una: [(nombre, id, valor)].

Llevan lo que se acaba de enviar o, si no, lo guardado.

library/forms.py, línea 110

```
@property
def pro_slots(self)
```

Casillas de «Lo mejor» (ver point_slots).

library/forms.py, línea 120

```
@property
def con_slots(self)
```

Casillas de «Lo peor» (ver point_slots).

library/forms.py, línea 125

```
def clean(self)
```

Comprueba que las fechas no sean del futuro y que no se termine antes de empezar.

library/forms.py, línea 129

Clase MultipleImageInput

Selector de archivos que deja elegir varias imágenes a la vez.

Clase MultipleImageField

Varias imágenes subidas en el mismo campo: devuelve la lista de archivos (vacía si no se sube ninguna).

```
def __init__(self, **kwargs)
```

Con el selector múltiple y opcional.

library/forms.py, línea 151

```
def clean(self, data, initial=None)
```

Comprueba cada archivo como lo haría un FileField y los devuelve en una lista.

library/forms.py, línea 156

Clase ReviewImagesForm

Imágenes de una reseña, como mucho MAX_IMAGES: las que ya tiene (se pueden marcar como spoiler o quitar), las que se suben (pasadas a WebP y sin EXIF por accounts.images) y las capturas y artes oficiales del juego, que se eligen sin subir nada. Las nuevas van detrás de las que había: primero las subidas y después las oficiales.

```
def __init__(self, data=None, files=None, *, review, official)
```

review es la reseña que se edita y official, las imágenes oficiales de su juego (games.media.official_images).

En el elector solo salen las oficiales que la reseña aún no tiene.

library/forms.py, línea 176

```
@property
def free_slots(self)
```

Cuántas imágenes más caben en la reseña sin quitar ninguna.

library/forms.py, línea 191

```
def clean(self)
```

Comprueba que no pasen de MAX_IMAGES entre las que se quedan y las nuevas y prepara las subidas.

Cada subida se convierte a WebP en dos tamaños (la grande para el visor y la miniatura para la reseña); las que no son una imagen válida, pesan demasiado o tienen demasiados píxeles dan un error con su nombre.

library/forms.py, línea 195

```
def save(self, review)
```

Aplica los cambios a las imágenes de review, que ya debe estar guardada.

Quita las marcadas (sus archivos se borran con ellas, ver library.signals), pone o quita el spoiler de las demás y añade las nuevas detrás.

library/forms.py, línea 216

Clase ReportForm

Denuncia de una reseña o un comentario: el motivo y, si se quiere, algo más para el administrador.

```
def clean_detail(self)
```

Los detalles sin espacios de sobra al principio ni al final.

library/forms.py, línea 254

Clase CommentForm

Comentario en una reseña, con el mismo formato sencillo que las reseñas.

El campo quita solo los espacios del principio y del final, así que uno en blanco da el error de «required».

library/markup.py

Formato sencillo y seguro de las reseñas y los comentarios.

Se escribe como en muchos foros: **negrita**, *cursiva*, ||spoiler|| y [texto](https://enlace); una línea en blanco separa párrafos. Primero se escapa todo el HTML y después se cambian solo esas marcas, así que no se puede colar ninguna etiqueta ni script. Los enlaces solo pueden ser http o https y llevan rel="nofollow ugc noopener" para que no sirvan de spam.

```
def _inline(text)
```

Un párrafo ya escapado con sus marcas cambiadas por HTML.

library/markup.py, línea 23

```
def render(text)
```

HTML seguro de un texto con formato: un <p> por párrafo,
 en cada salto de línea y las marcas cambiadas.

library/markup.py, línea 48

```
def render_inline(text)
```

HTML seguro de un texto corto de una sola línea (un punto de «Lo mejor» o «Lo peor»), sin párrafo alrededor.

library/markup.py, línea 55

```
def plain(text)
```

El texto sin las marcas de formato y con los spoilers tapados («[spoiler]»), en una sola línea.

Sirve para resúmenes, la portada y la descripción de la página.

library/markup.py, línea 61

library/models.py

Clase LibraryEntry

```
def __str__(self)
```

Texto de la entrada en el admin: «usuario · juego · estado».

library/models.py, línea 80

```
@classmethod
def imported_status(cls, minutes, last_played)
```

Estado de un juego que llega de una importación (Steam, PlayStation o Xbox) según cuánto y cuándo se jugó.

Sin minutos ni partida: «Pendiente». Jugado en los últimos IMPORT_RECENT_DAYS días: «Jugando». Si no: «Jugado».

library/models.py, línea 85

```
def update_import(self, platform, minutes, last_played)
```

Guarda los minutos y la última partida de una plataforma ("steam", "psn" o "xbox") en una entrada existente.

Conserva el estado que puso el usuario, salvo «Pendiente» cuando ya hay horas jugadas: entonces lo cambia a «Jugando» o «Jugado». Devuelve True si ha cambiado algo.

library/models.py, línea 97

```
@property
def playtime_minutes(self)
```

Minutos jugados sumando Steam, PlayStation y Xbox, o None si no viene de ninguna.

library/models.py, línea 118

```
@property
def playtime_hours(self)
```

Horas jugadas (Steam + PlayStation + Xbox) redondeadas a una décima, o None si no hay datos.

library/models.py, línea 126

```
@property
def hours_by_platform(self)
```

Horas de cada plataforma de la que vienen, de la que tiene más a la que menos: [{"key", "name", "hours", "last_played"}]. key es la clave de HOURS_SOURCES («steam», «psn» o «xbox»), que da su logo (library/_platform_mark.html). Solo salen las que llegan a una décima de hora: abrir un juego no es jugarlo.

library/models.py, línea 132

```
@property
def playtime_sources(self)
```

Plataformas de las que vienen las horas jugadas, como claves de HOURS_SOURCES: p. ej. ["steam", "psn"].

library/models.py, línea 144

```
@cached_property
def achievements(self)
```

Cómo va con los logros de este juego en cada plataforma de la que se han importado (lo que da accounts.achievements.progress_by_game), o una lista vacía si no se conocen.

library/models.py, línea 149

```
@property
def stars(self)
```

Nota en estrellas (de 0,5 a 5), o None sin nota.

library/models.py, línea 155

Clase PlaySession

Lo que ha jugado un usuario a un juego en una plataforma un día: lo que ha subido su tiempo entre dos sincronizaciones, apuntado el día de su última partida (varias el mismo día se suman).

Lo apunta la importación (accounts.matching.update_library) solo si ya se conocían las horas de esa plataforma: lo que trae la primera importación es de antes y no se sabe cuándo se jugó. Es lo que miden los retos de horas y, más adelante, el diario de partidas.

```
def __str__(self)
```

Texto en el admin: «usuario · juego · día · minutos».

library/models.py, línea 191

```
@classmethod
def add(cls, user, game_id, platform, day, minutes)
```

Suma minutes a lo que user ha jugado a game_id en platform el día day (la crea si es la primera de ese día).

library/models.py, línea 196

Clase ReviewQuerySet

```
def published(self)
```

Reseñas publicadas, sin los borradores.

library/models.py, línea 214

```
def visible_to(self, viewer)
```

Reseñas publicadas que viewer puede leer, de la más reciente a la más antigua, con sus «me gusta» (like_count) y comentarios (comment_count) contados y sus imágenes ya cargadas.

Las de perfiles privados solo las ve su autor, igual que su biblioteca (Profile.visible_filter).

library/models.py, línea 218

Clase Review

Reseña de un juego. Cada usuario puede escribir varias del mismo juego (una por partida, por ejemplo al rejugarlo); se ven en la ficha, en su perfil, en la portada y en su propia página.

Al publicarla se guardan las horas que llevaba jugadas (de las importaciones de Steam, PlayStation y Xbox) y el estado del juego en su biblioteca, para enseñar «escrita a las 12 h», filtrar las de quien lo ha terminado y plegar las que van más adelante que quien las lee. La nota va de 1 a 10 medias estrellas. Los borradores solo los ve su autor.

```
def __str__(self)
```

Texto en el admin: «usuario · juego».

library/models.py, línea 283

```
def get_absolute_url(self)
```

URL de la página de la reseña: /u/<usuario>/resenas/<id>/.

library/models.py, línea 287

```
@property
def stars(self)
```

Nota en estrellas (de 0,5 a 5), o None sin nota.

library/models.py, línea 292

```
@property
def finished(self)
```

Indica si el autor lo tenía como «Completado» al publicarla.

library/models.py, línea 297

```
@property
def status_note(self)
```

Cómo iba el autor con el juego al publicarla, para la reseña: «Lo terminó», «Aún jugándolo»...; vacía si no se sabe.

library/models.py, línea 302

```
@property
def hours_source_names(self)
```

Nombres de las plataformas de las que salen las horas: «Steam» o «Steam y PlayStation».

library/models.py, línea 307

```
@property
def hours_source_logos(self)
```

Plataformas de las que salen las horas como [(nombre, plantilla de su logo)], para el sello de horas verificadas.

library/models.py, línea 313

```
def hours_since(self, entry)
```

Horas que ha jugado el autor desde que la publicó, si son bastantes para proponerle actualizarla; si no, None.

entry es su entrada de biblioteca del juego. Son bastantes si lleva al menos el doble que al escribirla y como mínimo UPDATE_MIN_EXTRA_HOURS más («Llevas 40 h más: ¿la actualizas?»). Los borradores y las reseñas escritas sin horas importadas no cuentan.

library/models.py, línea 317

```
def snapshot(self, entry)
```

Apunta el estado y las horas que tiene ahora la entrada de biblioteca entry (o nada si no hay).

library/models.py, línea 330

Clase ReviewLike

«Me gusta» de un usuario en una reseña (uno por usuario y reseña).

```
def __str__(self)
```

Texto en el admin: «usuario → reseña».

library/models.py, línea 350

Clase ReviewComment

Comentario en una reseña. Lo pueden borrar quien lo escribió y el autor de la reseña.

```
def __str__(self)
```

Texto en el admin: «usuario en reseña: comienzo del comentario».

library/models.py, línea 370

```
def get_absolute_url(self)
```

URL del comentario dentro de la página de su reseña.

library/models.py, línea 374

```
def can_delete(self, user)
```

Indica si user puede borrar el comentario: es suyo o es el autor de la reseña.

library/models.py, línea 378

```
def attached_image_upload_to(instance, filename)
```

Ruta de una imagen adjunta: la carpeta la decide cada modelo (upload_folder) y el nombre aleatorio, accounts.images.

library/models.py, línea 383

Clase AttachedImage

Imagen adjunta a algo que escribe un usuario: de momento, las reseñas; más adelante, también las publicaciones.

O la ha subido él (image, pasada a WebP y sin EXIF por accounts.images, con su miniatura en thumb) o es una captura o un arte oficial del juego de IGDB (igdb_image_id), que no se sube. Cada una se puede marcar como spoiler: entonces va borrosa hasta que se pulsa.

```
def upload_folder(self)
```

Carpeta de MEDIA_ROOT en la que se guardan las imágenes subidas; la pone cada modelo.

library/models.py, línea 407

```
@property
def is_official(self)
```

Indica si es una imagen oficial del juego (de IGDB) y no una subida por el usuario.

library/models.py, línea 412

```
@property
def url(self)
```

URL de la imagen grande, para el visor a pantalla completa.

library/models.py, línea 417

```
@property
def thumb_url(self)
```

URL de la miniatura: la de IGDB o la reducida al subirla (la grande si no la hay).

library/models.py, línea 422

```
def delete_files(self)
```

Borra del disco la imagen subida y su miniatura (las oficiales no tienen archivo).

library/models.py, línea 428

Clase ReviewImage

Imagen de una reseña (como mucho MAX_IMAGES por reseña). Las subidas se guardan en resenas/<id del autor>/.

```
def __str__(self)
```

Texto en el admin: «reseña · imagen n».

library/models.py, línea 444

```
def upload_folder(self)
```

Las imágenes de las reseñas van en resenas/<id del autor>.

library/models.py, línea 448

Clase Report

Denuncia de un usuario sobre algo que ha escrito otro (una reseña o un comentario) para que la revise un administrador.

Va por contenttypes para que sirva también para lo que se añada después (publicaciones, grupos...). Hay una por usuario y cosa denunciada: volver a denunciarla cambia el motivo y la deja otra vez pendiente.

```
def __str__(self)
```

Texto en el admin: «denunciante → lo denunciado: motivo».

library/models.py, línea 487

```
@property
def target_kind(self)
```

Qué se ha denunciado, en palabras: «reseña» o «comentario» (el nombre del modelo de lo denunciado).

library/models.py, línea 492

Clase WishlistItem

Juego que el usuario quiere («Lo quiero»), con un precio por debajo del cual avisarle (opcional).

Los avisos los crea library.wishlist.check_alerts después de actualizar los precios; también avisa cuando el juego está rebajado a su mínimo histórico, aunque no haya precio objetivo.

```
def __str__(self)
```

Texto en el admin: «usuario · juego».

library/models.py, línea 527

```
@property
def on_steam(self)
```

Indica si el juego está también en la lista de deseos de Steam del usuario (según la última importación).

library/models.py, línea 532

library/points.py

«Lo mejor» y «Lo peor» de las reseñas de un juego, sumados: las sugerencias del editor y lo que más se repite.

El editor sugiere los puntos que ya han escrito otros en el mismo juego. Si la gente los reutiliza, salen las mismas frases y la ficha puede enseñar «Lo que más destacan» con cuántas reseñas lo dicen. Dos puntos cuentan como el mismo si solo cambian las mayúsculas, los espacios o la puntuación del principio y del final.

```
def key(point)
```

Forma de comparar un punto: sin formato, en minúsculas, con los espacios simplificados y sin puntuación en los extremos.

library/points.py, línea 24

```
def _tally(rows)
```

Cuenta los puntos de rows ([(pros, cons)]): {"pros": [(punto, veces)], "cons": [...]}, de más a menos repetido.

Cada reseña cuenta una vez por punto; de cada punto se enseña cómo lo escribió el primero.

library/points.py, línea 29

```
def _visible(game, viewer)
```

Reseñas publicadas del juego que viewer puede leer (las de perfiles públicos y las suyas), de la más antigua a la más nueva: así cada punto se enseña como lo escribió el primero.

library/points.py, línea 44

```
def suggestions(game, viewer)
```

Puntos que más han escrito otros en las reseñas del juego, para sugerirlos en el editor: {"pros": [...], "cons": [...]}.

library/points.py, línea 50

```
def consensus(game, viewer)
```

Lo que más destacan y lo que menos gusta del juego: {"pros": [(punto, reseñas)], "cons": [...]}.

Solo los puntos que dicen al menos CONSENSUS_MINIMUM reseñas, como mucho CONSENSUS_SHOWN de cada lista.

library/points.py, línea 59

library/reminders.py

Avisos a los autores de reseñas que se han quedado atrás: «Llevas 40 h más en Hades: ¿actualizas tu reseña?».

Cada reseña guarda las horas que llevaba su autor al publicarla (Review.hours). Cuando una importación de Steam, PlayStation o Xbox le sube mucho las horas de ese juego (ver Review.hours_since), se le avisa en la campana para

que la actualice. Solo cuenta la última reseña publicada de cada juego: las anteriores son de otras partidas y se quedan como estaban.

```
def latest_reviews(user, games=None)
```

Última reseña publicada de user de cada juego (de esos juegos, si se dan), como {id del juego: reseña}.

library/reminders.py, línea 16

```
def remind_outdated_reviews(user)
```

Avisa a user en la campana de sus reseñas que se han quedado atrás y devuelve cuántos avisos ha creado.

Un aviso por reseña hasta que la vuelva a guardar: si sigue jugando sin tocarla, no se repite.

library/reminders.py, línea 27

library/signals.py

```
@receiver(post_delete, sender=ReviewImage)
def delete_review_image_files(sender, instance, **kwargs)
```

Borra del disco la imagen subida y su miniatura al quitarla de una reseña o al borrar la reseña o la cuenta.

library/signals.py, línea 8

library/templatetags/reviews.py

Filtros de plantilla de las reseñas: el formato del texto, las notas en estrellas, los permisos de los comentarios y los motivos de las denuncias.

```
@register.simple_tag
def report_reasons()
```

Motivos que se pueden elegir al denunciar una reseña o un comentario: [(valor, texto)].

library/templatetags/reviews.py, línea 14

```
@register.filter
def review_html(text)
```

HTML seguro de una reseña o un comentario con su formato (ver library.markup).

library/templatetags/reviews.py, línea 20

```
@register.filter
def review_inline(text)
```

HTML seguro de un punto de «Lo mejor» o «Lo peor» con su formato, sin párrafo.

library/templatetags/reviews.py, línea 26

```
@register.filter
def review_plain(text)
```

Una reseña sin formato y con los spoilers tapados, para resúmenes.

library/templatetags/reviews.py, línea 32

```
@register.filter
def stars_label(rating)
```

Nota de 1 a 10 escrita en estrellas: «3,5 estrellas»; vacía sin nota.

library/templatetags/reviews.py, línea 38

```
@register.filter
def can_delete(comment, user)
```

Indica si user puede borrar el comentario (es suyo o la reseña es suya).

library/templatetags/reviews.py, línea 44

library/views.py

```
def mark_liked(reviews, user)
```

Apunta en cada reseña de reviews (liked) si user le ha dado «me gusta»; devuelve la misma lista.

library/views.py, línea 49

```
def review_context(request, game, entry)
```

Variables de la sección de reseñas de la ficha de game para quien la ve (entry: su juego en la biblioteca).

Las reseñas que puede leer, ordenadas con ?resenas= (recientes o más gustadas) y filtradas con ?filtro= (las de quien lo ha terminado o las escritas con más de LONG_PLAY_HOURS horas); las suyas del juego, borradores incluidos, con las horas que lleva de más desde la última (extra_hours, ver Review.hours_since) para proponerle actualizarla; lo mejor y lo peor que más se repite (consensus, ver library.points); y en cada reseña si le ha dado «me gusta» (liked) y si va más adelante que él en el juego (ahead, ver AHEAD_HOURS) para plegarla.

library/views.py, línea 59

```
def _respond(request, game, entry)
```

Respuesta común de las acciones de biblioteca.

Si la petición viene de HTMX devuelve el widget de biblioteca ya actualizado junto con el recuadro de valoración de la comunidad y la sección de reseñas (que HTMX cambia en su sitio); si no, redirige a la ficha del juego.

library/views.py, línea 120

```
@require_POST
@login_required
def set_status(request, game_id)
```

Añade el juego a la biblioteca del usuario con el estado enviado, o cambia el que tenía.

Si el estado es nuevo, lo apunta en su actividad (social.activity) y, al pasarlo a «Completado», apunta cuándo (completed_at, que se vacía si después cambia de estado) y mira si con eso ha conseguido algún reto (challenges.progress.check). Responde 400 si el estado no es uno de los permitidos.

library/views.py, línea 147

```
@require_POST
@login_required
def remove(request, game_id)
```

Quita el juego de la biblioteca del usuario (no da error si no estaba) y olvida su actividad con él, salvo las reseñas.

library/views.py, línea 172

```
@require_POST
@login_required
def rate(request, game_id)
```

Guarda la valoración del usuario (de 1 a 10 medias estrellas) para un juego de su biblioteca.

Con rating=0 borra la valoración. Si cambia, lo apunta en su actividad (social.activity). Responde 404 si el juego no está en su biblioteca y 400 si la valoración no es un número del 0 al 10.

library/views.py, línea 183

```
@login_required
def review_new(request, game_id)
```

Editor de una reseña nueva de un juego de la biblioteca del usuario.

Empieza con la nota que ya le había puesto y, si ya tiene alguna reseña publicada del juego, marcada como rejugada. Si el juego no está en su biblioteca, vuelve a la ficha avisando.

library/views.py, línea 206

```
@login_required
def review_edit(request, review_id)
```

Editor de una reseña ya escrita (o de un borrador) del usuario; las de otros dan 404.

library/views.py, línea 222

```
def _review_editor(request, review, entry)
```

Muestra y guarda el editor de review. «Publicar» la publica y «Guardar borrador» la guarda sin que la vea nadie más.

Al publicarla se apuntan las horas y el estado que tiene ahora en la biblioteca (Review.snapshot) y, si lleva nota, pasa a ser también la nota del juego en su biblioteca, que es la que cuenta en la media de la comunidad. La fecha de publicación es la de la primera vez. En una reseña nueva, la plataforma y las fechas vienen ya rellenas con lo de las importaciones (library.autofill) y «Lo mejor» y «Lo peor» sugieren lo que han escrito otros (library.points). Las imágenes (subidas o capturas oficiales del juego) van en un formulario aparte, ReviewImagesForm, que se guarda a la vez. La primera vez que se publica se apunta en la actividad del autor (social.activity); al editarla, no.

library/views.py, línea 229

```
@require_POST
@login_required
def review_delete(request, review_id)
```

Borra una reseña del usuario (con sus «me gusta» y comentarios) y vuelve a las reseñas del juego.

library/views.py, línea 287

```
@require_POST
@login_required
def review_preview(request)
```

Vista previa del texto de una reseña o un comentario con su formato (fragmento HTML para el editor).

library/views.py, línea 298

```
def _readable_review(request, review_id)
```

Reseña publicada review_id que el usuario puede leer; 404 si es un borrador o de un perfil privado ajeno.

library/views.py, línea 303

```
def review_detail(request, username, review_id)
```

Página de una reseña: el juego, la nota, los datos de la partida, sus imágenes, sus «me gusta» y sus comentarios.

Los borradores y las reseñas de perfiles privados solo los ve su autor; a los demás les da 404. A su autor se le propone actualizarla si ha jugado mucho más desde que la escribió (solo en la última que ha publicado del juego); los demás pueden denunciarla (con ?denunciar, el formulario llega abierto).

library/views.py, línea 311

```
def _comments_context(review, viewer)
```

Variables de library/_comments.html para los comentarios de la reseña (ver comment_thread).

library/views.py, línea 344

```
def comment_thread(parent, comments, viewer, *, comment_url, delete_url_name,
report_url_name, count_id)
```

Variables de library/_comments.html para los comentarios de parent: una reseña o una publicación (social.Post).

comments son sus comentarios (con su parent ya cargado, para saber quién puede borrarlos); salen con sus autores, en orden, y en cada uno si viewer ya lo ha denunciado (reported). comment_url es la dirección del formulario para comentar; delete_url_name y report_url_name, los nombres de las URL para borrar y denunciar un comentario; y count_id, el id del contador de comentarios que HTMX cambia al comentar o borrar.

library/views.py, línea 354

```
def _notify(review, actor, kind)
```

Avisa al autor de la reseña de que actor le ha dado «me gusta» o la ha comentado.

No se repite: un «me gusta» de la misma persona solo avisa una vez aunque lo quite y lo vuelva a dar, y un comentario no avisa si ya hay uno igual sin leer.

library/views.py, línea 384

```
@require_POST
@login_required
def review_like(request, review_id)
```

Da o quita el «me gusta» del usuario a una reseña de otro (avisando a su autor la primera vez).

Con HTMX devuelve el botón con el contador nuevo; si no, vuelve a la página de la reseña. Las propias no se pueden marcar (400).

library/views.py, línea 403

```
@require_POST
@login_required
def review_comment(request, review_id)
```

Publica un comentario en una reseña (avisando a su autor si no es él) y vuelve a sus comentarios.

library/views.py, línea 424

```
@require_POST
@login_required
def comment_delete(request, comment_id)
```

Borra un comentario: solo quien lo escribió o el autor de la reseña (si no, 403).

library/views.py, línea 446

```
@require_POST
@login_required
def review_report(request, review_id)
```

Denuncia una reseña de otro para que la revise un administrador; las propias no se pueden denunciar (400).

library/views.py, línea 462

```
@require_POST
@login_required
def comment_report(request, comment_id)
```

Denuncia un comentario de otro para que lo revise un administrador; los propios no se pueden denunciar (400).

Como con la reseña, da 404 si la reseña del comentario es un borrador o de un perfil privado ajeno.

library/views.py, línea 472

```
def _report(request, target, kind, game, back_url)
```

Guarda la denuncia del usuario sobre target (una reseña, un comentario, una publicación o una lista, según kind) y avisa a los administradores.

Si ya lo había denunciado, cambia el motivo y la vuelve a dejar pendiente. Con HTMX devuelve el botón de denunciar (library/_report.html) ya como «Gracias» o, si falta el motivo, abierto con el error; si no, vuelve a back_url con un mensaje.

library/views.py, línea 484

```
def _notify_staff(report, what, game)
```

Avisa en la campana a los administradores de la denuncia de what («una reseña de ana») en game, sin repetirlo mientras tengan uno igual sin leer.

game puede ser None (una publicación que no habla de ningún juego). El aviso lleva a la denuncia en el admin, desde donde se puede ir a lo denunciado y borrarlo.

library/views.py, línea 509

```
def _wish_response(request, game, wish, error='')
```

Respuesta de las acciones de la lista de deseos: el botón «Lo quiero» actualizado si es HTMX; si no, vuelve a la página de la que vino (la ficha por defecto) avisando del error si lo hay.

library/views.py, línea 526

```
@require_POST
@login_required
def wish_toggle(request, game_id)
```

Añade el juego a la lista de deseos del usuario o lo quita si ya estaba.

Si lo quita estando en su lista de deseos de Steam, se apunta para que la importación de Steam no lo vuelva a traer; si lo vuelve a añadir él, se olvida.

library/views.py, línea 541

```
@require_POST
@login_required
def wish_target(request, game_id)
```

Guarda el precio por debajo del cual avisar (vacío = solo avisar del mínimo histórico).

Al cambiarlo se olvida el último aviso, para volver a comparar con el precio nuevo. Responde 404 si el juego no está en su lista.

library/views.py, línea 564

```
def _steam_order(row)
```

Clave de orden «Como en Steam»: primero los que ha ordenado allí, por su puesto; después los demás de su lista de Steam, del añadido más recientemente al más antiguo; al final, los que no están en Steam.

library/views.py, línea 585

```
@login_required
def wishlist(request)
```

Lista de deseos del usuario con el mejor precio de hoy de cada juego (en sus tiendas, si las ha elegido), su mínimo histórico, si está por debajo de su límite y si está en su lista de Steam. Se ordena con ?orden= (recientes por defecto, descuento, precio, nombre o como en Steam) y se filtra por idioma con ?idioma= (ver LANGUAGE_FILTERS).

library/views.py, línea 606

library/wishlist.py

Avisos de la lista de deseos: comparan los precios de hoy con lo que quiere cada usuario.

Se lanzan al terminar update_prices. Un juego avisa cuando su mejor precio (PC, Xbox o Nintendo, solo en las tiendas del usuario si las ha elegido) baja del precio que puso el usuario o cuando está rebajado a su mínimo histórico. No se repite el aviso hasta que baje más, o hasta que vuelva a subir y baje otra vez. Los juegos que ya están en la biblioteca del usuario no avisan (ya lo tiene).

También está aquí el resumen que se enseña después de traer la lista de deseos de Steam (con los precios de los juegos nuevos ya pedidos, ver games.prices.fetch_missing): cuántos están hoy en su mínimo histórico y cuántos son más baratos fuera de Steam.

```
def at_historical_low(game, best)
```

Indica si best (la oferta más barata del juego) está rebajada a su mínimo histórico o por debajo.

Solo cuenta si está rebajada: un juego que nunca baja siempre está en su mínimo.

library/wishlist.py, línea 28

```
def steam_deals(user)
```

De los juegos de «Lo quiero» de user que están en su lista de deseos de Steam: (cuántos son, cuántos están hoy en su mínimo histórico, cuántos son más baratos en otra tienda de PC que en Steam).

Cuentan solo las tiendas del usuario si las ha elegido; el precio de Steam con el que se compara es siempre el de Steam.

library/wishlist.py, línea 37

```
def alert_reason(item, best)
```

Por qué avisar del precio best (un GamePrice) de un juego de la lista: «objetivo», «minimo» o None.

«minimo» solo cuenta si está rebajado: un juego que nunca baja siempre está en su mínimo.

library/wishlist.py, línea 63

```
def alert_text(item, best, reason)
```

Texto del aviso: el juego, el precio, la tienda y el motivo.

library/wishlist.py, línea 77

```
def check_alerts(games=None)
```

Crea los avisos de bajada de precio de las listas de deseos (de esos juegos, o de todos) y manda un correo por usuario con los suyos si lo tiene activado. Devuelve cuántos avisos ha creado.

library/wishlist.py, línea 85

```
def _send_email(user, notifications)
```

Manda al usuario un correo con sus avisos de precio; si falla, lo apunta en el log y sigue.

library/wishlist.py, línea 121

App «social»

Lo social: seguir a otros jugadores, las listas de seguidores y seguidos y la actividad de cada jugador.

social/activity.py

La actividad de cada jugador: lo que hace con los juegos de su biblioteca, lo que importa, lo que publica y sus listas.

Se apunta desde lo que hace el jugador (cambiar el estado de un juego, valorarlo, publicar una reseña, quitarlo de la biblioteca, crear una lista o añadirle juegos) y desde las importaciones de Steam, PlayStation y Xbox, nunca con señales: la sincronización de cada noche también guarda juegos y les cambia el estado, y eso no lo ha hecho el jugador.

Lo que hace con un mismo juego en poco rato se junta en una sola actividad («Terminó Hades □□□□½» con su reseña debajo), y cada importación es una sola aunque traiga cientos de juegos. La lista de deseos no sale: es privada, y no se enseña en ningún otro sitio.

```
def _recent(user)
```

Lo último que ha hecho user, si fue hace menos de MERGE_WINDOW (para juntar con ello lo siguiente); si no, None.

social/activity.py, línea 25

```
def _game_activity(user, game, merge_only=False, **changes)
```

Junta changes (status, rating o review) con lo último que hizo user si fue con game hace poco; si no, crea otra.

Con merge_only no crea ninguna: sirve para deshacer algo recién hecho (quitar la nota que acaba de poner). Si se queda vacía, se borra. Devuelve la actividad, o None si no hay ninguna.

social/activity.py, línea 33

```
def status_changed(user, game, status)
```

Apunta que user ha pasado game a status (o que lo ha añadido a su biblioteca con ese estado).

social/activity.py, línea 56

```
def rated(user, game, rating)
```

Apunta la nota que user le ha puesto a game. Si la ha quitado (None), solo se deshace la que acababa de poner.

social/activity.py, línea 61

```
def review_published(review)
```

Apunta que su autor ha publicado review, con su nota si la lleva.

Solo la primera vez: si la volvió a dejar en borrador y la publica otra vez, ya tenía su actividad.

social/activity.py, línea 68

```
def game_removed(user, game)
```

user ha quitado game de su biblioteca: se olvida lo que hizo con él, salvo sus reseñas, que siguen publicadas.

social/activity.py, línea 81

```
def post_published(post)
```

Apunta la publicación post en la actividad de su autor. Cada una va sola: no se junta con nada.

social/activity.py, línea 88

```
def games_imported(user, platform, entries)
```

Apunta como una sola actividad los juegos que acaban de entrar en la biblioteca de user con una importación.

platform es la clave de la plataforma ("steam", "psn" o "xbox") y entries, las entradas nuevas de la biblioteca; se guardan cuántas son y los ids de los IMPORT_COVERS juegos más jugados. Una importación de la misma plataforma justo después (al volver a sincronizar, por ejemplo) se suma a la anterior.

social/activity.py, línea 96

```
def _list_activity(game_list, user)
```

Lo último que ha hecho user si fue con game_list hace poco (MERGE_WINDOW); si no, None.

social/activity.py, línea 126

```
def list_created(game_list)
```

Apunta que su dueño ha creado game_list. Los juegos que le añada enseguida se juntan con esto (list_games_added).

Si la lista es privada (o lo es después), la actividad solo la ve su dueño; mientras no tenga juegos no se enseña.

social/activity.py, línea 134

```
def list_games_added(game_list, user, count=1)
```

Apunta que user (su dueño o quien colabora en ella) ha añadido count juegos a game_list: se suman a lo último que hizo si fue con esta lista hace poco (crearla o añadirle otros); si no, es una actividad nueva («añadió 3 juegos a la lista…»).

social/activity.py, línea 146

```
def list_game_removed(game_list, user)
```

user ha quitado un juego de game_list: si él le había añadido juegos hace poco, se descuenta de esa actividad (y si solo era eso, se borra), como quitar una nota recién puesta. Lo de antes no se toca.

social/activity.py, línea 160

social/admin.py

Clase PostImageInline

Imágenes de una publicación, para quitar las que no deban estar.

Clase PostCommentInline

Comentarios de una publicación, para moderarlos desde ella.

Clase PostAdmin

```
def get_queryset(self, request)
```

Las publicaciones con sus denuncias pendientes contadas.

social/admin.py, línea 41

```
@admin.display(description='denuncias pendientes', ordering='pending')
def pending_reports(self, post)
```

Cuántas denuncias de la publicación quedan por revisar.

social/admin.py, línea 48

social/apps.py

Clase SocialConfig

```
def ready(self)
```

Registra las señales de la app al arrancar Django: borran la actividad que se queda vacía al borrar una reseña y los archivos de las imágenes de las publicaciones que se borran.

social/apps.py, línea 7

social/forms.py

Clase PostCommentForm

Comentario en una publicación: el mismo que el de las reseñas (texto con su formato sencillo, sin espacios de sobra).

Clase PostForm

Publicación nueva: el texto (con el formato sencillo de las reseñas), el juego del que habla si quiere, si es un spoiler y hasta POST_MAX_IMAGES imágenes, que se pasan a WebP y sin EXIF con su miniatura (accounts.images). Tiene que llevar texto o alguna imagen.

```
def clean_text(self)
```

Quita los espacios del principio y del final.

social/forms.py, línea 35

```
def clean(self)
```

Comprueba que lleve texto o imágenes y que no pasen de POST_MAX_IMAGES, y prepara las subidas.

Las que no son una imagen válida, pesan demasiado o tienen demasiados píxeles dan un error con su nombre.

social/forms.py, línea 39

```
def save(self, user)
```

Guarda la publicación de user con sus imágenes en el orden en que llegaron y la devuelve.

social/forms.py, línea 58

social/models.py

Clase FollowManager

```
def follow(self, follower, followed)
```

Hace que follower siga a followed y avisa a followed en la campana; devuelve True si no lo seguía ya.

El aviso llega solo la primera vez: si follower deja de seguir a followed y vuelve, no se repite. Lleva a la página de follower o, si followed no puede verla (perfil privado), a la lista de seguidores de followed, donde sale sin enlace. No comprueba si followed se puede seguir (eso lo mira social.views.follow).

social/models.py, línea 17

```
def unfollow(self, follower, followed)
```

Hace que follower deje de seguir a followed; devuelve True si lo seguía.

social/models.py, línea 38

Clase Follow

Un usuario (follower) sigue a otro (followed) para ver lo que hace.

Va en un solo sentido y sin solicitudes: followed no tiene que aceptar ni seguir de vuelta. No se puede seguir dos veces a la misma persona ni seguirse a uno mismo (también lo impide la base de datos). Los perfiles privados no se pueden seguir, pero sus dueños sí pueden seguir a otros.

```
def __str__(self)
```

Texto en el admin: «seguidor → seguido».

social/models.py, línea 71

Clase Post

Publicación de un jugador: lo que quiera contar, con hasta POST_MAX_IMAGES imágenes y, si quiere, sobre un juego.

Sale en el feed de la portada y en la actividad de su perfil (social.activity.post_published) y tiene su propia página. La ve quien puede ver su perfil (Profile.visible_filter). El texto lleva el mismo formato sencillo que las reseñas (library.markup) y, si es un spoiler, el texto y las imágenes van tapados hasta que se pulsan.

```
def __str__(self)
```

Texto en el admin: «usuario: comienzo del texto».

social/models.py, línea 100

```
def get_absolute_url(self)
```

URL de la página de la publicación: /u/<usuario>/publicaciones/<id>/.

social/models.py, línea 104

Clase PostImage

Imagen de una publicación (como mucho POST_MAX_IMAGES). Se guardan en publicaciones/<id del autor>/.

```
def __str__(self)
```

Texto en el admin: «publicación · imagen n».

social/models.py, línea 118

```
def upload_folder(self)
```

Las imágenes de las publicaciones van en publicaciones/<id del autor>.

social/models.py, línea 122

Clase PostLike

«Me gusta» de un usuario en una publicación (uno por usuario y publicación).

```
def __str__(self)
```

Texto en el admin: «usuario → publicación».

social/models.py, línea 139

Clase PostComment

Comentario en una publicación, como los de las reseñas. Lo pueden borrar quien lo escribió y el autor de la publicación.

```
def __str__(self)
```

Texto en el admin: «usuario en publicación: comienzo del comentario».

social/models.py, línea 159

```
def get_absolute_url(self)
```

URL del comentario dentro de la página de su publicación.

social/models.py, línea 163

```
def can_delete(self, user)
```

Indica si user puede borrar el comentario: es suyo o es el autor de la publicación.

social/models.py, línea 167

Clase Activity

Algo que ha hecho un jugador: sale en la pestaña Actividad de su perfil y en el feed de la portada.

La apunta social.activity desde lo que hace el jugador, nunca con señales: las importaciones también guardan juegos y les cambian el estado, y eso no lo ha hecho él. Hay tres tipos:

- GAME: lo que hace con un juego de su biblioteca. Lo que hace con el mismo juego en poco rato se junta en una sola (ver social.activity.MERGE_WINDOW): el último estado al que lo ha pasado (status), la nota que le ha puesto (rating) y la reseña que ha publicado (review), las tres o solo alguna. - IMPORT: los juegos nuevos de una importación de Steam, PlayStation o Xbox, todos juntos. En data van la plataforma («platform», clave de HOURS_SOURCES), cuántos son («count») y los ids de los más jugados («games»), para enseñar sus portadas. - POST: una publicación (post), con su juego si lo tiene. Se borra con ella. - LIST: una lista (game_list): que la ha creado («created» en data) y cuántos juegos le ha añadido («added»), juntos si van seguidos (MERGE_WINDOW). Se borra con ella; si es privada, solo la ve su dueño.

updated_at es la hora de lo último que se ha juntado: la que se enseña y por la que se ordena.

```
def __str__(self)
```

Texto en el admin: «usuario · lo que hizo».

social/models.py, línea 240

```
@property
def is_empty(self)
```

Indica si una actividad de un juego se ha quedado sin nada (sin estado, nota ni reseña) y sobra.

social/models.py, línea 245

```
@property
def has_content(self)
```

Indica si hay algo que enseñar de ella a quien la mira: los juegos de una importación, la publicación, la lista (si puede verla y tiene juegos) o, en las de un juego, el estado, la nota o la reseña (si su reseña ya no se puede leer, puede que nada).

social/models.py, línea 250

```
@property
def list_created(self)
```

Indica si una actividad de una lista cuenta que la ha creado (si no, solo que le ha añadido juegos).

social/models.py, línea 267

```
@property
def list_added(self)
```

Cuántos juegos ha añadido a la lista en esta actividad.

social/models.py, línea 272

```
@property
def verb(self)
```

Qué hizo con el juego, en minúscula: (lo que va antes del nombre del juego, lo que va después).

Con un estado, lo de STATUS_VERBS; si no, «reseñó» si publicó una reseña que se puede leer, o «valoró».

social/models.py, línea 277

```
@property
def shown_rating(self)
```

Nota que se enseña: la que le puso o, si no, la de su reseña (si la reseña se puede leer); o None.

social/models.py, línea 287

```
@property
def platform_name(self)
```

Nombre de la plataforma de una importación («Steam», «PlayStation» o «Xbox»).

social/models.py, línea 295

```
@property
def import_count(self)
```

Cuántos juegos nuevos trajo una importación.

social/models.py, línea 300

```
@property
def summary(self)
```

Lo que hizo, en una línea y sin formato, para el admin: «terminó Hades», «importó 12 juegos de Steam», «publicó» o «creó la lista Indies».

social/models.py, línea 305

social/signals.py

```
@receiver(post_delete, sender=Review)
def forget_empty_activity(sender, instance, **kwargs)
```

Al borrar una reseña, borra la actividad que solo era esa reseña; la que tenía también estado o nota se queda sin ella.

social/signals.py, línea 10

```
@receiver(post_delete, sender=PostImage)
def delete_post_image_files(sender, instance, **kwargs)
```

Borra del disco la imagen subida y su miniatura al borrar la publicación o la cuenta de su autor.

social/signals.py, línea 19

social/views.py

```
def follow_context(viewer, user)
```

Variables del botón de seguir y de los contadores de user para quien mira (viewer).

Cuántos seguidores tiene user (follower_count) y a cuántos sigue (following_count), si viewer ya sigue a user (is_following) y si user sigue a viewer (follows_you). Las dos últimas son falsas sin sesión y en el propio perfil.

social/views.py, línea 47

```
@require_POST
@login_required
def follow(request, username)
```

Sigue a username (seguir=1) o deja de seguirlo (seguir=0).

Seguir a quien ya se sigue, o dejar a quien no se sigue, no hace nada: pulsar dos veces no deshace lo hecho. Seguirse a uno mismo da 400. Un perfil privado ajeno da 404, como su página, salvo para dejar de seguirlo (pudo hacerse privado después). Al seguir se avisa en la campana (Follow.objects.follow). Con HTMX devuelve el botón nuevo, del tamaño de donde estaba (desde: «perfil» o «lista»), y en el perfil también el número de seguidores de la cabecera; sin HTMX vuelve a la página de «next» o al perfil.

social/views.py, línea 70

```
def followers(request, username)
```

Seguidores de username, del más reciente al más antiguo (ver _people).

social/views.py, línea 103

```
def following(request, username)
```

Gente a la que sigue username, de la más reciente a la más antigua (ver _people).

social/views.py, línea 108

```
def _people(request, username, view)
```

Página con los seguidores de username (view «seguidores») o con la gente a la que sigue («siguiendo»).

Va de PEOPLE_PER_PAGE en PEOPLE_PER_PAGE (?page=). Un perfil privado da 404 a cualquiera que no sea su dueño, como su página. En las listas salen también los perfiles privados (a quién sigue cada uno se ve, como lo que comenta), pero sin enlace a su página (profile_visible) ni botón para seguirlos, solo para dejar de seguirlos. En cada persona se apunta si quien mira la sigue (viewer_follows) y si ella sigue a quien mira (follows_viewer).

social/views.py, línea 113

```
def _batch(activities, viewer, cursor)
```

Una tanda de ACTIVITY_PAGE actividades de activities (ya filtradas) tal como las ve viewer, de lo más reciente a lo más antiguo.

Devuelve (actividades, cursor de la tanda siguiente o "" si no hay más). cursor dice desde dónde seguir: lo da la tanda anterior, y vacío (o roto) es desde el principio. Cada actividad trae su reseña solo si viewer puede leerla (si su autor la volvió a dejar en borrador, no), con sus imágenes y sus «me gusta» y comentarios contados; su lista, solo si viewer puede verla, con su mosaico, sus «me gusta» y cuánto ha jugado de ella viewer; y las importaciones traen sus juegos (import_games). Las que se quedan sin nada que enseñar no salen. No mira de quién son: eso lo decide quien la llama.

social/views.py, línea 155

```
def activity_context(user, viewer, cursor='')
```

Variables de la pestaña Actividad del perfil de user tal como la ve viewer, desde cursor (ver _batch).

La tanda que toca (activities), el cursor de la siguiente (next_cursor) y, si quedan más, las direcciones de «Cargar más»: la que pide HTMX (more_url) y la de la pestaña desde ahí, para quien no tenga JavaScript (more_href). No mira si viewer puede ver el perfil: eso lo hace quien la llama.

social/views.py, línea 190

```
def feed_context(viewer, tab='', cursor='')
```

Variables del feed de la portada para viewer en la pestaña tab, desde cursor (ver _batch).

«siguiendo» (la de por defecto) es lo de viewer y la gente que sigue; «descubrir», lo de toda la comunidad; sin sesión solo hay «descubrir». En las dos sale solo lo de perfiles que viewer puede ver (Profile.visible_filter): si alguien a quien sigue se hace privado, deja de ver lo suyo. Además de la tanda y de «Cargar más» (como activity_context), lleva la pestaña (feed_tab) y las que hay (feed_tabs); si viewer no sigue a nadie (follows_nobody), en la primera tanda va también gente a la que seguir (suggestions).

social/views.py, línea 207

```
def suggested_people(viewer, limit=SUGGESTIONS)
```

Gente a la que viewer podría seguir: perfiles públicos a los que aún no sigue, primero los que tienen más juegos en común con viewer y después los que más han hecho en los últimos SUGGESTION_DAYS días.

Solo salen quienes tienen algún juego en común o actividad reciente. Cada uno trae cuántos juegos tiene en común (shared) y si sigue a viewer (follows_viewer), y lo que necesitan su foto y el botón de seguir. Se cuenta con subconsultas para no cruzar la biblioteca con la actividad de cada uno.

social/views.py, línea 247

```
def _cursor(activity)
```

Cursor de «Cargar más» para seguir después de activity: «microsegundos desde 1970 (UTC)-id».

Se cuenta en microsegundos enteros, sin decimales, para que la fecha vuelva exacta al leerlo.

social/views.py, línea 285

```
def _parse_cursor(cursor)
```

Lo contrario de _cursor: (fecha, id), o None si cursor está vacío o no tiene esa forma.

social/views.py, línea 293

```
def activity(request, username)
```

Tanda siguiente de la actividad de username (?antes=, el cursor de la anterior) para «Cargar más».

Con HTMX devuelve solo las actividades nuevas y el botón para la siguiente; sin HTMX lleva a la pestaña Actividad del perfil desde ese punto. Un perfil privado da 404 a cualquiera que no sea su dueño.

social/views.py, línea 302

```
def feed(request)
```

Tanda siguiente del feed de la portada (?ver=, la pestaña, y ?antes=, el cursor de la anterior) para «Cargar más».

Con HTMX devuelve solo las actividades nuevas y el botón para la siguiente; sin HTMX lleva a la portada desde ese punto.

social/views.py, línea 317

```
def _posts()
```

Publicaciones con lo que enseñan sus tarjetas ya cargado: autor (con su foto), juego, imágenes, «me gusta» (like_count) y comentarios (comment_count).

social/views.py, línea 329

```
def mark_liked_posts(posts, user)
```

Apunta en cada publicación de posts (liked) si user le ha dado «me gusta»; devuelve la misma lista.

social/views.py, línea 339

```
def composer_context(user, game=None, form=None)
```

Variables de la caja de publicar de user (social/_composer.html).

El formulario (post_form: form o uno vacío), los juegos que ofrece para elegir de cuál va la publicación (composer_games: game, si se da, y los últimos COMPOSER_GAMES que está jugando), el elegido (composer_game, su id o None) y las medidas con las que el navegador reduce las imágenes muy grandes.

social/views.py, línea 349

```
@login_required
def post_new(request)
```

Publica lo que escriba el usuario (POST) o enseña la página para hacerlo (GET).

Con ?juego= (desde la ficha de un juego) la publicación ya va sobre ese juego. Al publicar se apunta en su actividad (social.activity.post_published) y vuelve a la portada, donde sale la primera de Siguiendo; si falta algo, enseña esta página con los errores y lo que había escrito.

social/views.py, línea 370

```
def _readable_post(request, post_id)
```

Publicación post_id con sus tarjetas cargadas, si el usuario puede verla; 404 si es de un perfil privado ajeno.

social/views.py, línea 390

```
def post_detail(request, username, post_id)
```

Página de una publicación: su autor, el juego del que habla, el texto entero, sus imágenes, sus «me gusta» y sus comentarios.

La ve quien puede ver el perfil de su autor (si no, 404). Su autor puede borrarla; los demás, denunciarla (con ?denunciar llega abierto el formulario).

social/views.py, línea 398

```
def _post_comments(post, viewer)
```

Variables de library/_comments.html para los comentarios de post (ver library.views.comment_thread).

social/views.py, línea 420

```
@require_POST
@login_required
def post_comment(request, post_id)
```

Publica un comentario en una publicación (avisando a su autor si no es él) y vuelve a sus comentarios.

Con HTMX devuelve los comentarios ya con el nuevo (o el formulario con el error) y el contador de la página.

social/views.py, línea 432

```
def _notify_post_comment(post, actor)
```

Avisa al autor de post de que actor la ha comentado, salvo si ya tiene un aviso igual sin leer.

social/views.py, línea 455

```
@require_POST
@login_required
def post_comment_delete(request, comment_id)
```

Borra un comentario de una publicación: solo quien lo escribió o el autor de la publicación (si no, 403).

social/views.py, línea 470

```
@require_POST
@login_required
def post_comment_report(request, comment_id)
```

Denuncia un comentario de otro en una publicación; los propios no (400) y los de publicaciones que no se pueden ver dan 404.

social/views.py, línea 486

```
@require_POST
@login_required
def post_delete(request, post_id)
```

Borra una publicación del usuario, con sus imágenes, sus «me gusta» y su actividad, y vuelve a la portada; las de otros dan 404.

social/views.py, línea 497

```
@require_POST
@login_required
def post_like(request, post_id)
```

Da o quita el «me gusta» del usuario a una publicación de otro, avisando a su autor la primera vez.

Con HTMX devuelve el botón con el contador nuevo; si no, vuelve a la página de la publicación. Las propias no se pueden marcar (400).

social/views.py, línea 506

```
def _notify_post_like(post, actor)
```

Avisa al autor de post de que a actor le ha gustado, solo la primera vez aunque lo quite y lo vuelva a dar.

social/views.py, línea 525

```
@require_POST
@login_required
def post_report(request, post_id)
```

Denuncia una publicación de otro para que la revise un administrador (library.views._report); las propias no (400).

social/views.py, línea 538

App «lists»

Listas de juegos de cada usuario: crearlas, ordenarlas y comentarlas, y verlas en su página, en el perfil y en la ficha de cada juego.

lists/admin.py

Clase ListItemInline

Juegos de una lista, en su orden, para corregirlos o quitarlos desde ella.

Clase ListCollaboratorInline

Gente invitada a editar la lista y si ha aceptado.

Clase ListCommentInline

Comentarios de una lista, para moderarlos desde ella.

Clase GameListAdmin

```
def get_queryset(self, request)
```

Las listas con cuántos juegos tiene cada una y cuántas denuncias suyas quedan por revisar.

lists/admin.py, línea 44

```
@admin.display(description='Juegos', ordering='game_count')
def game_count(self, obj)
```

Cuántos juegos tiene la lista.

lists/admin.py, línea 52

```
@admin.display(description='denuncias pendientes', ordering='pending')
def pending_reports(self, obj)
```

Cuántas denuncias de la lista quedan por revisar.

lists/admin.py, línea 57

lists/forms.py

Clase ListCommentForm

Comentario en una lista: el mismo que el de las reseñas y las publicaciones (texto con su formato sencillo).

Clase GameListForm

Datos de una lista: el título, la descripción (con el formato sencillo de las reseñas), si la ven los demás y su tipo: lista, numerada o tier list. Los juegos se añaden aparte, uno a uno (lists.views.add).

Para las tier lists lleva también sus niveles: MAX_TIERS filas con nombre (nivel_<n>) y color (color_<n>), de mejor a peor. La fila n es el nivel con id n, así que renombrar uno no cambia los juegos que tiene; los que se dejan sin nombre se quitan y sus juegos quedan sin clasificar.

```
def __init__(self, *args, **kwargs)
```

Añade las filas de los niveles, rellenas con los de la lista (o los de una tier list nueva).

Sin tipo en lo enviado, la lista se queda con el que tenía (una nueva, lista sin más).

lists/forms.py, línea 60

```
@property
def switches(self)
```

Los campos de sí o no (visible para los demás), que la plantilla pinta como interruptores.

lists/forms.py, línea 82

```
@property
def tier_fields(self)
```

Las filas de los niveles para la plantilla: [(nombre, color)] de cada uno, de mejor a peor.

lists/forms.py, línea 87

```
def clean_title(self)
```

Deja un solo espacio entre palabras (el campo ya quita los del principio y el final, y sin título da error).

lists/forms.py, línea 91

```
def clean_description(self)
```

Quita los espacios del principio y del final.

lists/forms.py, línea 95

```
def clean(self)
```

Arma los niveles con las filas que tienen nombre (cleaned_data["tiers"]); un color que no sea «#rrggbb» da error. Una tier list necesita al menos un nivel; las demás guardan los niveles por si vuelven a serlo.

lists/forms.py, línea 99

```
def save(self, commit=True)
```

Guarda la lista con sus niveles.

lists/forms.py, línea 118

Clase NewListForm

GameListForm de «Nueva lista»: una lista sin más o numerada. Las tier lists se crean en su página (TierListForm), aunque una lista ya creada puede pasar a serlo en su editor.

```
def __init__(self, *args, **kwargs)
```

Quita «Por niveles» de los tipos.

lists/forms.py, línea 133

Clase TierListForm

GameListForm de «Nueva tier list»: el tipo va fijo y oculto, y sus niveles se ven siempre.

```
def __init__(self, *args, **kwargs)
```

Empieza con una tier list nueva (con los niveles de serie), oculta el tipo y pone ejemplos de tier list.

lists/forms.py, línea 143

```
def clean_mode(self)
```

Siempre tier list, aunque se mande otro tipo.

lists/forms.py, línea 151

lists/models.py

```
def default_tiers()
```

Niveles con los que empieza una tier list: [{"id", "label", "color"}] de DEFAULT_TIERS.

lists/models.py, línea 28

Clase GameListQuerySet

```
def visible_to(self, viewer)
```

Listas que viewer puede ver: las suyas, las que le han invitado a editar y las públicas de perfiles públicos.

Es la regla de GameList.visible_to como condición de consulta: tienen que decir lo mismo.

lists/models.py, línea 34

```
def editable_by(self, user)
```

Listas cuyos juegos puede cambiar user: las suyas y en las que colabora (ha aceptado la invitación).

lists/models.py, línea 45

```
def with_preview(self)
```

Listas con lo que enseñan sus tarjetas ya cargado: su autor (con su foto), cuántos juegos tienen (game_count), sus «me gusta» (like_count) y comentarios (comment_count) y los MOSAIC_COVERS primeros juegos para el mosaico (preview_items).

Van de la más reciente a la más antigua: con los recuentos, Django ya no aplica el orden de Meta.

lists/models.py, línea 50

Clase GameList

Lista de juegos de un usuario («Mejores metroidvanias», «Para jugar en pareja»), con su descripción.

Los juegos van en el orden que elija (ListItem.position), cada uno con su comentario si quiere. Según su tipo (mode) es una lista sin más, una numerada (cada juego lleva su puesto, para rankings) o una tier list (cada juego va en uno de sus niveles: S, A, B...). Las tier lists tienen su propia sección (su página para crearlas y su pestaña del perfil), pero son listas como las demás: una puede cambiar de tipo en su editor y conserva sus niveles. Su dueño puede invitar a otros a editar sus juegos (ListCollaborator). La ven los demás si es pública y su perfil también (visible_to); las privadas solo las ven su dueño y los invitados. Los demás pueden darle «me gusta», comentarla y denunciarla, y sale en la actividad de su dueño (social.activity.list_created).

```
def __str__(self)
```

Texto en el admin: «título (usuario)».

lists/models.py, línea 111

```
def get_absolute_url(self)
```

URL de la página de la lista: /u/<usuario>/listas/<id>/.

lists/models.py, línea 115

```
def visible_to(self, viewer)
```

Indica si viewer puede ver la lista: siempre su dueño y quien está invitado a editarla (aunque aún no haya aceptado: tiene que verla para hacerlo); los demás, si es pública y pueden ver el perfil de su dueño.

En las consultas la aplica GameListQuerySet.visible_to, que tiene que decir lo mismo.

lists/models.py, línea 119

```
def can_edit_games(self, user)
```

Indica si user puede añadir, quitar, ordenar y comentar los juegos: su dueño y quien colabora en ella.

lists/models.py, línea 129

```
@property
def is_ranked(self)
```

Indica si es numerada: cada juego lleva su puesto.

lists/models.py, línea 136

```
@property
def is_tiered(self)
```

Indica si es una tier list: cada juego va en uno de sus niveles.

lists/models.py, línea 141

```
@property
def kind_note(self)
```

Lo que se dice de su tipo junto a cuántos juegos tiene: «numerada», «tier list» o nada.

lists/models.py, línea 146

```
@property
def noun(self)
```

Cómo se la llama en los textos y los avisos: «tier list» o «lista».

lists/models.py, línea 151

```
@property
def profile_tab(self)
```

Pestaña del perfil de su dueño en la que sale (?vista=): las tier lists tienen la suya, aparte de las demás.

lists/models.py, línea 156

```
@property
def tier_list(self)
```

Sus niveles, de mejor a peor, cada uno con el color de su texto (text, ver accounts.models.text_on).

lists/models.py, línea 161

```
def tier_rows(self, items, keep_loose=False)
```

Los juegos items (ListItem, en su orden) repartidos por niveles: [(nivel, juegos)] en el orden de los niveles y, al final, los que no están en ninguno con el nivel None, si hay alguno (con keep_loose, aunque no haya ninguno: en el tablero que se puede editar es donde se sueltan los que se sacan de su nivel).

lists/models.py, línea 165

```
@property
def mosaic_gaps(self)
```

Huecos vacíos del mosaico cuando tiene menos de MOSAIC_COVERS juegos (para recorrerlos en la plantilla).

lists/models.py, línea 176

```
def touch(self)
```

Da la lista por actualizada ahora (sin tocar nada más).

lists/models.py, línea 180

```
def add(self, game, user=None)
```

Añade game al final de la lista, apuntando quién lo añade (user); devuelve True si no estaba ya.

No mira LIST_MAX_GAMES ni si user puede editarla: eso lo hace quien la llama.

lists/models.py, línea 184

```
def remove(self, game)
```

Quita game de la lista y corre los de detrás para no dejar hueco; devuelve True si estaba.

lists/models.py, línea 197

```
def move(self, game, position)
```

Lleva game al puesto position (desde 0; si se pasa, al principio o al final) y corre los demás.

Devuelve True si game está en la lista.

lists/models.py, línea 205

```
def place(self, game, level, row)
```

Pone game en el nivel level (su id; None, sin clasificar) y lo deja en ese nivel donde lo han soltado al arrastrarlo en el tablero: row son los ids de los juegos de esa fila en su nuevo orden, con el suyo.

Queda justo detrás del juego que tiene delante en la fila o, si es el primero, justo delante del que le sigue; solo en la fila, sin nadie más, conserva su puesto. Los demás juegos no cambian de orden entre sí. De row solo cuentan los juegos que están de verdad en ese nivel (quien arrastra puede tener la página sin actualizar). Devuelve True si game está en la lista; no mira si level es uno de sus niveles: eso lo hace quien la llama.

lists/models.py, línea 220

```
@staticmethod
def _save_order(items)
```

Guarda como puestos seguidos (0, 1, 2...) el orden de items, solo en los que cambian.

lists/models.py, línea 251

Clase ListItem

Un juego de una lista, en su puesto y con el comentario de quien la hace («por qué está aquí»), si lo tiene.

```
def __str__(self)
```

Texto en el admin: «lista · juego».

lists/models.py, línea 283

Clase ListCollaborator

Alguien a quien el dueño de una lista invita a editar sus juegos: añadirlos, quitarlos, ordenarlos y comentarlos.

Hasta que acepta (accepted_at) solo puede verla, aunque sea privada, para aceptar o rechazar la invitación. Los datos de la lista, quién colabora y borrarla siguen siendo cosa de su dueño. Quien colabora puede irse cuando quiera; lo que añadió se queda.

```
def __str__(self)
```

Texto en el admin: «usuario en lista (pendiente)».

lists/models.py, línea 309

Clase ListLike

«Me gusta» de un usuario en una lista de otro (uno por usuario y lista).

```
def __str__(self)
```

Texto en el admin: «usuario → lista».

lists/models.py, línea 326

Clase ListComment

Comentario en una lista, como los de las reseñas y las publicaciones. Lo pueden borrar quien lo escribió y el dueño de la lista.

```
def __str__(self)
```

Texto en el admin: «usuario en lista: comienzo del comentario».

lists/models.py, línea 347

```
def get_absolute_url(self)
```

URL del comentario dentro de la página de su lista.

lists/models.py, línea 351

```
def can_delete(self, user)
```

Indica si user puede borrar el comentario: es suyo o es el dueño de la lista.

lists/models.py, línea 355

lists/presets.py

Plantillas de tier list que se rellenan solas con la biblioteca del usuario: «Jugados en 2026», sus más jugados, su pendiente…

Cada plantilla (PRESETS) elige juegos de la biblioteca en un orden y da el título y los niveles de la tier list; lists.views.from_library la crea con esos juegos en «Sin clasificar», o ya colocados por la nota que les ha puesto (RATING_TIERS), para que solo haya que arrastrarlos. Las de un año son del año en curso: Steam solo da la última partida y el historial de partidas (library.models.PlaySession) es reciente, así que en un año pasado faltarían los juegos que se han vuelto a jugar después.

```
def library(user)
```

La biblioteca de user (LibraryEntry, con su juego) con los minutos jugados en todas las plataformas (minutes): de más a menos horas y, a igualdad, de lo añadido hace menos a lo que más.

lists/presets.py, línea 34

```
def year_bounds(year)
```

Principio de year y del año siguiente, en la zona horaria del sitio.

lists/presets.py, línea 44

```
def played_in(user, year)
```

Lo jugado en year: lo que tiene ese año su última partida de Steam, PlayStation o Xbox, partidas apuntadas (PlaySession) o una reseña empezada o terminada; lo completado ese año; y, si es el año en curso, lo que está jugando.

lists/presets.py, línea 49

```
def completed_in(user, year)
```

Lo terminado en year: lo pasado a «Completado» ese año o con una reseña terminada ese año.

lists/presets.py, línea 65

```
def most_played(user, year)
```

Los MOST_PLAYED juegos con más horas, sumando las de todas las plataformas.

lists/presets.py, línea 72

```
def full_achievements(user, year)
```

Lo que tiene con el platino de PlayStation o con todos los logros en alguna plataforma (accounts.achievements).

lists/presets.py, línea 77

```
def played_on(platform)
```

Cómo elige sus juegos la plantilla de una plataforma: lo que tiene horas en platform (clave de HOURS_SOURCES), de más a menos horas en ella.

lists/presets.py, línea 87

```
def backlog(user, year)
```

Lo pendiente, de lo añadido hace menos a lo que más.

lists/presets.py, línea 99

Clase Preset

Una plantilla: su clave (la que manda el formulario), su nombre y su explicación en «Nueva lista» (label y help), el título de la tier list que crea (label y title pueden llevar {year}), cómo elige los juegos (entries: del usuario y el año a sus LibraryEntry, en orden), sus niveles ([(nombre, color)], de mejor a peor) y si coloca de entrada los juegos que tienen nota (by_rating, solo con los niveles de serie).

```
def label_for(self, year)
```

Su nombre en «Nueva lista» en year.

lists/presets.py, línea 119

```
def title_for(self, year)
```

Título de la tier list que crea en year.

lists/presets.py, línea 123

```
def choose(self, user, year)
```

Los juegos de la biblioteca de user que entran en year, en su orden (LibraryEntry), como mucho LIST_MAX_GAMES.

lists/presets.py, línea 127

```
def tier_list(self)
```

Sus niveles como los guarda GameList.tiers: [{"id", "label", "color"}].

lists/presets.py, línea 131

```
def tier_of(self, entry)
```

Id del nivel en el que empieza el juego de entry: con by_rating, el de su nota (RATING_TIERS); si no, o sin nota, ninguno (sin clasificar).

lists/presets.py, línea 135

```
def choices(user)
```

Las plantillas del bloque «Desde tu biblioteca» de «Nueva tier list», en el año en curso, con lo que enseña cada una: su clave, su nombre y su explicación (key, label y help), cuántos juegos metería (count), las portadas de los primeros (preview_items y mosaic_gaps, como el mosaico de una lista), si se puede crear (ready: al menos MIN_GAMES) y, si user ya tiene una lista con el título que le pondría, esa lista (existing). Las que no tienen ningún juego no salen.

lists/presets.py, línea 171

lists/views.py

```
def mark_progress(lists, viewer)
```

Apunta en cada lista de lists cuántos de sus juegos ha jugado viewer (viewer_played; None sin sesión).

Jugado es cualquier estado de la biblioteca menos Pendiente (PLAYED_STATUSES). Devuelve la misma lista.

lists/views.py, línea 56

```
def mark_liked_lists(lists, viewer)
```

Apunta en cada lista de lists (liked) si viewer le ha dado «me gusta»; devuelve la misma lista.

lists/views.py, línea 74

```
def profile_lists(owner, viewer, public_only=False)
```

Listas de owner y en las que colabora que puede ver viewer, de la más reciente a la más antigua, con lo que enseñan sus tarjetas (GameListQuerySet.with_preview) y cuántos de sus juegos ha jugado viewer (mark_progress). Las que son de otro llevan from_other, para decir de quién son.

Con public_only salen solo las públicas, como las vería cualquiera (la vista previa de Editar perfil).

lists/views.py, línea 84

```
def mark_boards(lists)
```

Apunta en cada tier list de lists su tablero en miniatura (lists/_tier_card.html): board, con [(nivel, juegos, más)] de cada nivel en su orden (sus BOARD_COVERS primeros juegos y cuántos más tiene), y loose_count, cuántos están sin clasificar (también los de un nivel que ya no existe, como en GameList.tier_rows).

Son dos consultas para todas: los primeros juegos de cada nivel y cuántos tiene cada uno. Devuelve la misma lista.

lists/views.py, línea 103

```
def _lists_with(game, viewer)
```

Listas que viewer puede ver en las que está game, de la más reciente a la más antigua, con sus tarjetas cargadas.

lists/views.py, línea 129

```
def picker_context(user, game)
```

Variables del desplegable «Añadir a una lista» de la ficha de game (lists/_picker.html).

Las listas que user puede editar (las suyas y en las que colabora), de la más reciente a la más antigua, cada una con su dueño, cuántos juegos tiene (game_count) y si game ya está en ella (has_game), y en cuántas está (in_lists).

lists/views.py, línea 138

```
def game_page_context(request, game)
```

Variables de las listas en la ficha de game: con sesión, las del desplegable «Añadir a una lista» (picker_context); y para todos, «En N listas»: las GAME_PAGE_LISTS más recientes que puede ver quien mira (game_lists) y cuántas son (game_lists_count).

lists/views.py, línea 153

```
def game_lists(request, slug)
```

Todas las listas en las que está un juego que puede ver quien mira, de la más reciente a la más antigua, de LISTS_PER_PAGE en LISTS_PER_PAGE (?page=).

lists/views.py, línea 164

```
def detail(request, username, list_id)
```

Página de una lista: su mosaico, el título, el autor (y quien colabora), la descripción y los juegos, en portadas (en las tier lists, por niveles) o con el comentario de cada uno (?vista=; sin ella, con los comentarios si alguno tiene), y debajo sus comentarios. En el tablero de una tier list, quien puede editarla arrastra las portadas de un nivel a otro (lists/_tier_board.html y tier).

La ve quien puede verla (GameList.visible_to); si no, 404. A quien está invitado a editarla le pregunta si acepta. Con sesión, enseña cuánto ha jugado quien mira de la lista, el estado de cada juego en su biblioteca y, de lo que no tiene, cuánto cuesta (list_progress). Los demás pueden darle «me gusta» y denunciarla (con ?denunciar, el formulario llega abierto).

lists/views.py, línea 174

```
def _readable_list(request, list_id)
```

Lista list_id con su dueño cargado, si el usuario puede verla; si no, 404.

lists/views.py, línea 227

```
@require_POST
@login_required
def like(request, list_id)
```

Da o quita el «me gusta» del usuario a una lista de otro, avisando a su dueño la primera vez.

Con HTMX devuelve el botón con el contador nuevo; si no, vuelve a la lista. Las propias no se pueden marcar (400).

lists/views.py, línea 237

```
def _notify_like(game_list, actor)
```

Avisa al dueño de game_list de que a actor le ha gustado, solo la primera vez aunque lo quite y lo vuelva a dar.

lists/views.py, línea 257

```
def _list_comments(game_list, viewer)
```

Variables de library/_comments.html para los comentarios de game_list (ver library.views.comment_thread).

lists/views.py, línea 268

```
@require_POST
@login_required
def comment(request, list_id)
```

Publica un comentario en una lista (avisando a su dueño si no es él) y vuelve a sus comentarios.

Con HTMX devuelve los comentarios ya con el nuevo (o el formulario con el error) y el contador de la página.

lists/views.py, línea 280

```
def _notify_comment(game_list, actor)
```

Avisa al dueño de game_list de que actor la ha comentado, salvo si ya tiene un aviso igual sin leer.

lists/views.py, línea 303

```
@require_POST
@login_required
def comment_delete(request, comment_id)
```

Borra un comentario de una lista: solo quien lo escribió o el dueño de la lista (si no, 403).

lists/views.py, línea 318

```
@require_POST
@login_required
def comment_report(request, comment_id)
```

Denuncia un comentario de otro en una lista; los propios no (400) y los de listas que no se pueden ver dan 404.

lists/views.py, línea 334

```
@require_POST
@login_required
def report(request, list_id)
```

Denuncia una lista de otro para que la revise un administrador (library.views._report); las propias no (400).

lists/views.py, línea 345

```
def list_progress(games, viewer)
```

Cómo va viewer con los juegos games de una lista (con sus precios ya cargados).

Cuántos son (total), cuántos tiene en su biblioteca (owned), cuántos ha jugado (played: cualquier estado menos Pendiente; percent, en porcentaje), terminado (completed) y tiene en Pendiente (backlog), las horas que lleva en ellos (hours: las de Steam, PlayStation y Xbox), el estado de cada uno (statuses: id del juego → nombre del estado; status_keys: id del juego → valor del estado, para su color) y lo que le falta (ver _missing).

lists/views.py, línea 353

```
def _missing(games, viewer)
```

Lo que le falta a viewer de una lista: games, los juegos que no están en su biblioteca, y lo que cuestan.

Cuántos son (missing); cuántos tienen precio en sus tiendas (priced) y lo que suman hoy (today) y si espera a que cada uno baje a su mínimo histórico de PC (at_lows; el de hoy si no lo tiene o si hoy está más barato), con saving si así sale más barato; cuántos no tienen precio (unpriced: los de consolas sin tienda); y cuántos están ya en su lista de deseos (wished) y cuántos no (to_wish), para el botón de pasarlos a «Lo quiero».

lists/views.py, línea 378

```
@require_POST
@login_required
def wish_missing(request, list_id)
```

Pasa a la lista de deseos del usuario los juegos de una lista que ni tiene en su biblioteca ni quiere ya, y vuelve a la lista. Como «Lo quiero» en la ficha: si alguno lo había quitado de su lista de deseos de Steam, se olvida, para que la importación de Steam no lo vuelva a quitar.

lists/views.py, línea 412

```
def _own_list(request, list_id)
```

Lista list_id del usuario; las de otros dan 404.

lists/views.py, línea 442

```
def _editable_list(request, list_id)
```

Lista list_id si el usuario puede editar sus juegos (su dueño o quien colabora en ella); si no, 404.

lists/views.py, línea 447

```
def editor_context(game_list)
```

Variables de los juegos del editor (lists/_editor_items.html): la lista, sus juegos en orden (con quién añadió cada uno), si es una tier list sus niveles (tiers) y el tablero en el que se arrastran sus portadas (tier_rows, siempre con la fila «Sin clasificar»), y los máximos.

lists/views.py, línea 455

```
def board_context(game_list, viewer)
```

Variables del tablero de la página de una tier list (lists/_tier_board.html) para quien puede editarla, que es lo que se le devuelve al soltar una portada en él: sus juegos por niveles, con la fila «Sin clasificar» aunque esté vacía, y el estado de cada uno en la biblioteca de viewer (viewer_status), como en detail.

lists/views.py, línea 470

```
def collaborators_context(game_list, owner)
```

Variables de la sección «Quién puede editarla» del editor (lists/_collaborators.html): la gente invitada, con si ha aceptado; cuántos caben; y, para escribir su nombre, la gente a la que sigue owner que aún no está invitada (suggested, como mucho SUGGESTED_COLLABORATORS).

lists/views.py, línea 482

```
def search_context(game_list, query)
```

Variables del buscador del editor: la búsqueda (search_query; no query, que es la del buscador de la cabecera) y, si la hay, los SEARCH_RESULTS juegos más populares que la cumplen (results, como el buscador del catálogo), cada uno marcado si ya está en la lista (in_list).

lists/views.py, línea 498

```
def _create(request, form, game=None)
```

Guarda como del usuario la lista nueva de form (ya validado), que empieza con game si se da; la apunta en su actividad (social.activity.list_created) y lleva a su editor, para buscar y añadir sus juegos.

lists/views.py, línea 511

```
@login_required
def new(request)
```

Crea una lista del usuario (POST) o enseña el formulario para hacerlo (GET): una lista sin más o numerada; las tier lists tienen su propia página (new_tier_list).

Con juego= (desde «Nueva lista con este juego» en la ficha) la lista empieza con ese juego. Se apunta en su actividad y lleva a su editor, para buscar y añadir más (_create).

lists/views.py, línea 529

```
@login_required
def new_tier_list(request)
```

Página «Nueva tier list»: las plantillas que la crean ya rellena con la biblioteca del usuario (lists.presets; se crean con from_library) y el formulario para empezar una en blanco con sus niveles (POST), que la crea como new.

lists/views.py, línea 545

```
@require_POST
@login_required
def from_library(request)
```

Crea una tier list del usuario con una de las plantillas de su biblioteca (plantilla=, ver lists.presets) y lleva a su página, para arrastrar sus juegos en el tablero.

Los juegos entran en el orden de la plantilla, sin clasificar o ya colocados por su nota, y la lista se crea privada: recién hecha está sin colocar, y su dueño la hace visible cuando quiera. Se apunta en su actividad como cualquier lista (social.activity). Si ya tiene una lista con ese título, lleva a ella en vez de repetirla; con menos de presets.MIN_GAMES juegos, vuelve a «Nueva tier list» con un aviso.

lists/views.py, línea 558

```
@login_required
def edit(request, list_id)
```

Editor de una lista: sus datos (título, descripción, si la ven los demás, su tipo y sus niveles), que guarda su dueño con el formulario; sus juegos, que se buscan (?q=, ver search), se añaden, se ordenan, se ponen en un nivel, se comentan y se quitan uno a uno (add, move, tier, note y remove); y quién puede editarla (invite y collaborator_remove). Quien colabora solo ve los juegos (y puede irse, leave); a los demás les da 404.

El formulario trabaja con una copia de la lista para que, si no es válido, la página siga con lo guardado.

lists/views.py, línea 600

```
@require_POST
@login_required
def delete(request, list_id)
```

Borra una lista del usuario (los juegos siguen en su biblioteca y en el catálogo) y vuelve a sus listas, o a sus tier lists si lo era.

lists/views.py, línea 630

```
@login_required
def search(request, list_id)
```

Busca en el catálogo juegos para añadir a una lista que el usuario puede editar (?q=, ver search_context).

Con HTMX devuelve solo los resultados; sin HTMX lleva al editor con la búsqueda hecha.

lists/views.py, línea 641

```
@require_POST
@login_required
def add(request, list_id, game_id)
```

Añade un juego al final de una lista que el usuario puede editar (si ya estaba, no hace nada); caben LIST_MAX_GAMES.

Se apunta quién lo añade y su actividad (social.activity.list_games_added), y responde según desde dónde se pulsa (ver _item_response).

lists/views.py, línea 655

```
@require_POST
@login_required
def remove(request, list_id, game_id)
```

Quita un juego de una lista que el usuario puede editar (si no estaba, no hace nada) y responde como add. Si lo acababa de añadir él, también se descuenta de su actividad (social.activity.list_game_removed).

lists/views.py, línea 673

```
@require_POST
@login_required
def move(request, list_id, game_id)
```

Cambia de puesto un juego de una lista que el usuario puede editar: uno arriba o abajo (a=arriba o a=abajo) o al puesto que se escriba (puesto=, desde 1; si se pasa, al principio o al final). Responde como add.

Da 404 si el juego no está en la lista y 400 si el puesto no es un número.

lists/views.py, línea 685

```
@require_POST
@login_required
def note(request, list_id, game_id)
```

Guarda el comentario de quien hace la lista sobre uno de sus juegos (vacío lo quita); caben NOTE_MAX_LENGTH caracteres.

Lo pueden escribir su dueño y quien colabora. Con HTMX devuelve solo su formulario, marcado como guardado o con el error, para no rehacer los demás juegos del editor; sin HTMX vuelve al editor, a ese juego.

lists/views.py, línea 710

```
@require_POST
@login_required
def tier(request, list_id, game_id)
```

Pone un juego de una tier list en uno de sus niveles (nivel=, su id; vacío lo deja sin clasificar). Con orden= (los ids de los juegos de esa fila separados por comas, como han quedado al soltarlo en el tablero) además lo deja en su sitio dentro del nivel (GameList.place). Lo pueden hacer su dueño y quien colabora. Responde como add; 404 si el juego no está en la lista y 400 si el nivel no existe.

lists/views.py, línea 738

```
def _collaborators_response(request, game_list, error='')
```

Respuesta de invitar o quitar a alguien de una lista: con HTMX, la sección «Quién puede editarla» del editor (con el error, si lo hay); sin HTMX, vuelve a ella avisando del error.

lists/views.py, línea 759

```
@require_POST
@login_required
def invite(request, list_id)
```

Invita a alguien (usuario=, su nombre de usuario, con o sin @) a editar los juegos de una lista del usuario y se lo avisa en la campana. Caben MAX_COLLABORATORS; no puede invitarse a sí mismo ni a quien ya está invitado.

lists/views.py, línea 773

```
@require_POST
@login_required
def collaborator_remove(request, list_id, user_id)
```

Quita a alguien de los que pueden editar una lista del usuario, o retira su invitación; lo que añadió se queda.

lists/views.py, línea 799

```
@require_POST
@login_required
def accept(request, list_id)
```

Acepta la invitación del usuario a editar una lista y lleva a su editor; 404 si no le han invitado.

lists/views.py, línea 808

```
@require_POST
@login_required
def leave(request, list_id)
```

Rechaza la invitación del usuario a editar una lista o deja de colaborar en ella (lo que añadió se queda).

Vuelve a la lista si aún puede verla y, si no (era privada), a sus listas (o a sus tier lists, si lo era); 404 si no le habían invitado.

lists/views.py, línea 820

```
def _item_response(request, game_list, game, error='')
```

Respuesta de añadir, quitar, mover o poner en un nivel un juego de una lista, según desde dónde se pulsa (desde=).

Con HTMX: desde la ficha («ficha»), el desplegable «Añadir a una lista» abierto; desde el buscador del editor («buscador»), los juegos del editor y, fuera de su sitio, el resultado ya marcado como añadido; desde el tablero de la página de la lista («tablero»), el tablero; desde el editor, sus juegos. Sin HTMX vuelve a next (si es de esta web) o al editor, avisando del error si lo hay.

lists/views.py, línea 837

App «challenges»

Retos: crearlos, unirse y contar el progreso de cada uno con su biblioteca (juegos completados y horas jugadas).

challenges/admin.py

Clase ParticipationInline

Quién se ha unido al reto y si lo ha conseguido.

challenges/forms.py

Clase ChallengeEditForm

Lo que se puede cambiar de un reto ya creado: el título, la descripción (con el formato sencillo de las reseñas) y si es público. Sus reglas no, para que sea justo con quien ya se ha unido.

```
def clean_title(self)
```

Deja un solo espacio entre palabras (el campo ya quita los del principio y el final, y sin título da error).

challenges/forms.py, línea 31

```
def clean_description(self)
```

Quita los espacios del principio y del final.

challenges/forms.py, línea 35

Clase ChallengeForm

Un reto nuevo: lo de ChallengeEditForm y sus reglas, según su tipo.

De juegos concretos: los de una lista (source_list: una de las que puede editar quien lo crea o, desde «Convertir en reto», la que estaba mirando) y cuántos de ellos hay que terminar (target; vacío, todos). De número de juegos o de horas: cuántos (target) y, si se quiere, de qué género, plataforma y años. Para todos, las fechas, que son opcionales.

```
def __init__(self, *args, user, game_list=None, **kwargs)
```

Prepara las listas que se pueden elegir (las que puede editar user y, si se da, game_list, desde «Convertir en reto») y los géneros y plataformas del catálogo.

challenges/forms.py, línea 76

```
def clean(self)
```

Comprueba lo que pide cada tipo y que las fechas y los años vayan en orden.

challenges/forms.py, línea 95

```
def save(self, user)
```

Guarda el reto de user. Uno de juegos concretos se queda con los que tiene su lista ahora (aunque la lista cambie después) y sin filtros; los demás, sin lista.

challenges/forms.py, línea 123

challenges/models.py

```
def long_date(day)
```

Una fecha como se escribe en España: «1 de octubre de 2026».

challenges/models.py, línea 15

Clase ChallengeQuerySet

```
def visible_to(self, viewer)
```

Retos que viewer puede ver: los públicos y los suyos. Es la regla de Challenge.visible_to en una consulta.

challenges/models.py, línea 21

```
def with_counts(self)
```

Retos con su creador cargado y cuánta gente se ha unido (participant_count).

challenges/models.py, línea 28

Clase Challenge

Un reto que crea un jugador y al que se puede unir cualquiera: terminar unos juegos concretos, terminar un número de juegos (de un género, una plataforma o unos años, si se quiere) o jugar un número de horas, entre unas fechas o sin ellas.

El progreso de cada uno se cuenta solo con su biblioteca (challenges.progress): los juegos que pasa a «Completado» entre las fechas (LibraryEntry.completed_at) o lo que juega en ellas (library.models.PlaySession). Sus reglas no cambian después de crearlo, para que sea justo con quien ya se ha unido. Los públicos los ve cualquiera; los privados, solo quien los crea (un objetivo propio, como «20 juegos en 2027»).

```
def __str__(self)
```

Texto en el admin: su título.

challenges/models.py, línea 86

```
def get_absolute_url(self)
```

URL de la página del reto: /retos/<id>/.

challenges/models.py, línea 90

```
def visible_to(self, viewer)
```

Indica si viewer puede ver el reto: los públicos, cualquiera; los privados, solo quien los crea.

challenges/models.py, línea 94

```
@property
def goal(self)
```

Lo que hay que llegar a tener, en la unidad de challenges.progress: juegos terminados o minutos jugados.

challenges/models.py, línea 99

```
@property
def is_upcoming(self)
```

Indica si aún no ha empezado.

challenges/models.py, línea 108

```
@property
def is_over(self)
```

Indica si ya ha terminado: desde entonces no se puede unir nadie.

challenges/models.py, línea 113

```
@property
def days_left(self)
```

Días que le quedan si está en marcha y tiene fecha de fin (hoy incluido); si no, None.

challenges/models.py, línea 118

```
@property
def filters_text(self)
```

De qué juegos va, en palabras: «de Rol de PC lanzados entre 2000 y 2010»; vacío si de cualquiera.

challenges/models.py, línea 125

```
@property
def rules_text(self)
```

Lo que hay que hacer, en una frase: «Termina 5 juegos de Terror», «Juega 50 horas», «Termina los 6 juegos».

challenges/models.py, línea 142

```
@property
def dates_text(self)
```

Cuándo cuenta, en palabras: «del 1 de octubre al 31 de octubre de 2026», «desde…», «hasta…» o «sin fechas».

challenges/models.py, línea 156

Clase Participation

Un jugador que se ha unido a un reto. Su progreso se calcula con su biblioteca (challenges.progress); cuando llega al objetivo se apunta cuándo (completed_at), una vez.

```
def __str__(self)
```

Texto en el admin: «usuario en reto».

challenges/models.py, línea 182

challenges/progress.py

El progreso de cada jugador en los retos, contado con su biblioteca.

En los de juegos (concretos o un número) cuentan los que ha pasado a «Completado» entre las fechas del reto (LibraryEntry.completed_at); sin fecha de inicio cuentan también los de antes, aunque no se sepa cuándo los terminó. En los de horas se suma lo jugado entre las fechas (library.models.PlaySession), que se apunta desde que existen las sesiones. done_by lo cuenta de golpe para muchos jugadores, para la clasificación de la página del reto.

```
def _start(day)
```

El principio del día day en la hora de España, para compararlo con fechas y horas.

challenges/progress.py, línea 19

```
def _game_filters(challenge, prefix='game__')
```

Condiciones sobre el juego de un reto de número de juegos o de horas: su género, su plataforma y sus años.

challenges/progress.py, línea 24

```
def _completed(challenge, users)
```

Juegos de la biblioteca de users que cuentan en un reto de juegos: los completados entre sus fechas que son de los suyos (juegos concretos) o que cumplen sus filtros (número de juegos).

challenges/progress.py, línea 38

```
def done_by(challenge, users)
```

Cuánto lleva cada uno de users en challenge: {id del usuario: juegos terminados o minutos jugados}. Los que no llevan nada no salen.

challenges/progress.py, línea 53

```
def summary(challenge, done)
```

Cómo va alguien que lleva done (lo de done_by) en challenge, para enseñarlo: lo hecho y el objetivo en juegos o en horas (value y goal_value), el porcentaje (percent, como mucho 100) y si ha llegado (reached).

challenges/progress.py, línea 66

```
def finished_games(challenge, user)
```

Ids de los juegos de un reto de juegos concretos que user ha terminado y cuentan.

challenges/progress.py, línea 80

```
def mark_completed(challenge, done)
```

Apunta que lo han conseguido (Participation.completed_at, una sola vez) los participantes de challenge que, según done (lo que da done_by), han llegado al objetivo. Devuelve a cuántos se les ha apuntado ahora.

challenges/progress.py, línea 85

```
def check(user)
```

Mira si user ha conseguido alguno de los retos en los que está y que aún no tenía (de los que no han terminado).

Lo llaman la biblioteca al pasar un juego a «Completado» y la importación al apuntar sesiones. Devuelve los retos conseguidos ahora.

challenges/progress.py, línea 97

challenges/views.py

```
def _open(challenges)
```

Los retos de challenges a los que aún se puede unir alguien: los que no han terminado.

challenges/views.py, línea 22

```
def _with_progress(challenges, user)
```

Pone en cada reto de challenges cómo va user en él (mine, ver challenges.progress.summary) y si lo ha conseguido (won); devuelve la misma lista.

challenges/views.py, línea 27

```
def explore(request)
```

Página de retos: con sesión, los tuyos (con cómo vas en cada uno) y los de la gente que sigues; para todos, los más populares que siguen abiertos (con más gente) y los últimos que se han creado. Solo salen los públicos, salvo los tuyos, y con sesión, abajo solo aquellos en los que aún no estás: los tuyos ya salen arriba.

challenges/views.py, línea 40

```
def _visible(request, challenge_id)
```

Reto challenge_id si el usuario puede verlo; si no, 404.

challenges/views.py, línea 59

```
def detail(request, challenge_id)
```

Página de un reto: qué hay que hacer y cuándo, quién lo ha creado, cómo vas tú (en los de juegos concretos, cuáles llevas) y la clasificación de quienes se han unido, de quien más lleva a quien menos.

En la clasificación solo salen quienes tienen el perfil visible para quien mira; los que han llegado al objetivo quedan apuntados como que lo han conseguido (challenges.progress.mark_completed).

challenges/views.py, línea 67

```
@login_required
def new(request)
```

Crea un reto (POST) o enseña el formulario (GET). Quien lo crea queda unido a él.

Con lista= (desde «Convertir en reto» de una lista que puede ver) empieza como uno de juegos concretos con los de esa lista; si no, como uno de número de juegos.

challenges/views.py, línea 113

```
@login_required
def edit(request, challenge_id)
```

Cambia el título, la descripción o si es público un reto del usuario (sus reglas no se cambian); los de otros dan 404.

challenges/views.py, línea 139

```
@require_POST
@login_required
def delete(request, challenge_id)
```

Borra un reto del usuario, con quienes se habían unido, y vuelve a la página de retos.

challenges/views.py, línea 152

```
@require_POST
@login_required
def join(request, challenge_id)
```

Une al usuario a un reto que puede ver (si ya estaba, no hace nada) y mira si ya lo ha conseguido con lo que lleva. A uno que ya ha terminado no se puede unir nadie (400).

challenges/views.py, línea 162

```
@require_POST
@login_required
def leave(request, challenge_id)
```

Saca al usuario de un reto en el que estaba (si no estaba, no hace nada) y vuelve a él si aún puede verlo.

challenges/views.py, línea 177